

Memory Management

— Chapter 8

2023年11月

叶冠华

g.ye@bupt.edu.cn

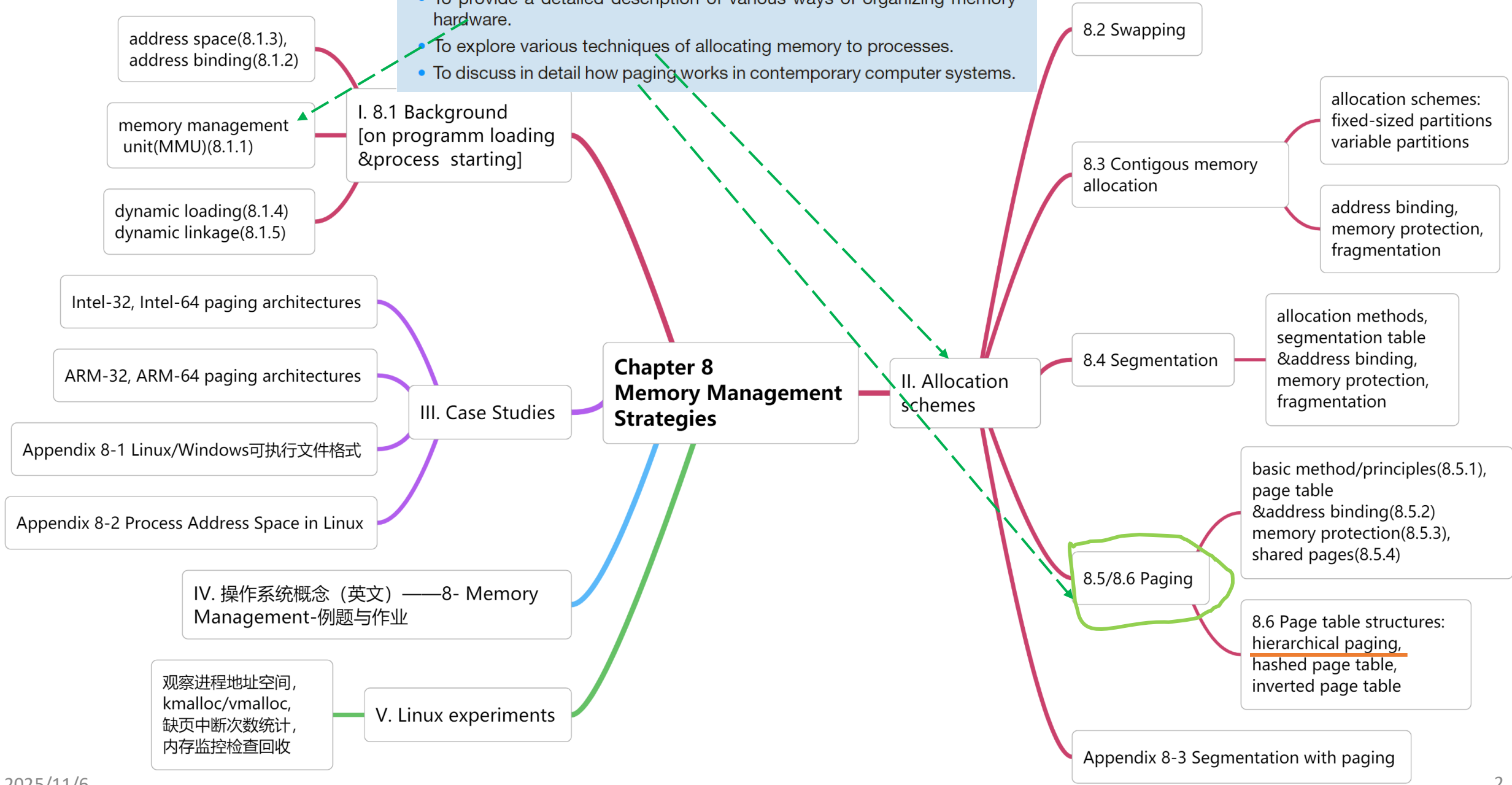
School of Computer Science (National Pilot Software Engineering School)



北京邮电大学

CHAPTER OBJECTIVES

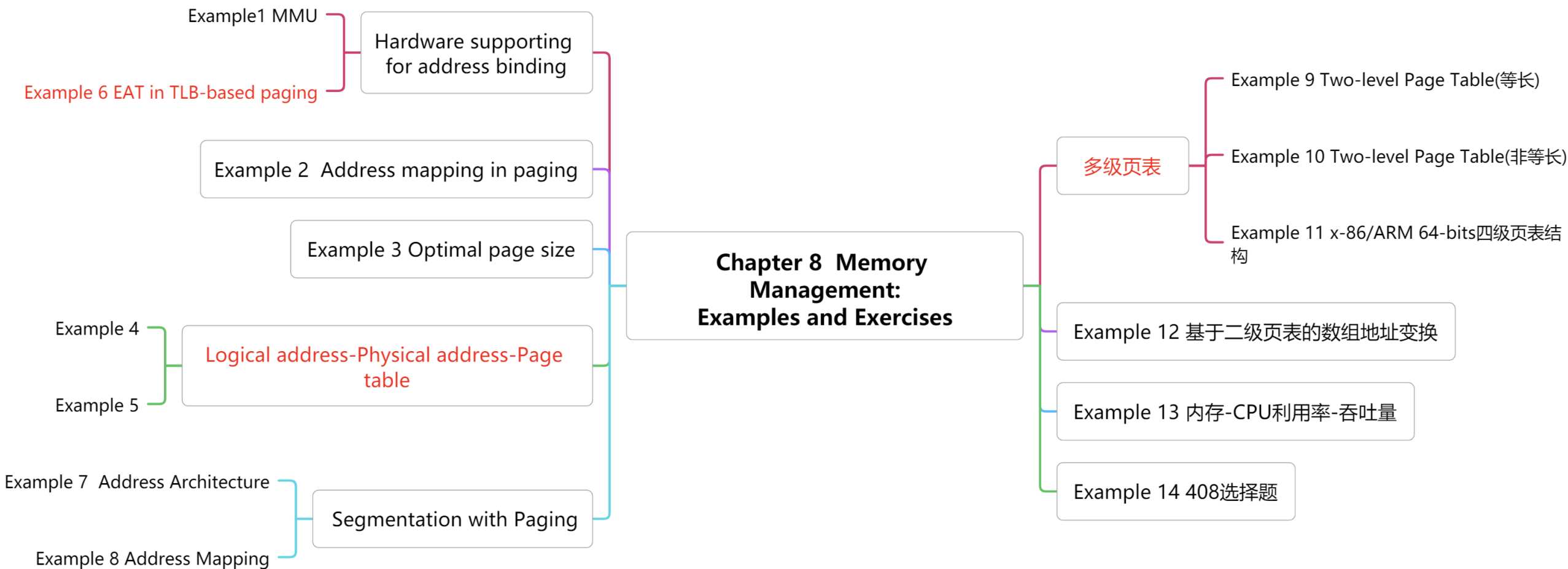
- To provide a detailed description of various ways of organizing memory hardware.
- To explore various techniques of allocating memory to processes.
- To discuss in detail how paging works in contemporary computer systems.



Terminology

memory stall	内存停顿
address space, logical address, physical address, address binding/mapping/relocation/translation	地址空间, 逻辑地址, 物理地址, 地址绑定/映射/重定位/翻译
MMU, relocation registers, base register, limit register	内存管理单元, 重定位寄存器, 基址寄存器, 界限寄存器
loading, linkage/linking, stub	加载, 链接, 存根

swapping contiguous allocation, fixed-sized partition, variable-partition hole, fragmentation, compaction	交换, 连续分配, 固定分区, 可变分区, 内存空洞, 内存碎片, 内存规整
segmentation, segment, segment table	分段/段式内存管理, 段, 段表
paging, page, page table, page number, page offset, frame, frame table, TLB, translation look-aside buffers, EAT, effective access time	分页/页式内存管理, 页, 页表, 页号, 页内偏移, 物理页、页框、页架、页帧, 物理页表, 转址旁路缓存, 有效访问时间
multi-level/hierarchical paging hashed page table, inverted page table	多级/多层分页, 哈希页表, 倒置/逆向页表



8.1 Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
 - CPU访问内存速度慢，导致CPU取不到数据、指令，处于停顿——**CPU暂停**
- Protection of memory required to ensure correct operation
 - avoiding user processes to incorrectly access OS, or avoiding user processes to incorrectly access each other
- Cache sits between main memory and CPU registers
 - to speed up CPU access instructions and data

Address Spaces

- **Address space**

- The total range of memory locations addressable by a computer
- [0x00000000, ..., 0x7FFFFFFF] for 32-bit architecture with 2GB RAM
- [0x0000000000000000, ..., 0xFFFFFFFFFFFFFFFF] for 64-bit architecture (theoretically)

- Programs on disk, ready to be brought into memory to execute, form an **input queue**

- without support, must be loaded into address 0x00000000

- Inconvenient to have first user process physical address always at 0x00000000

- How can it not be?

- Each process in main memory has its address space, i.e. logical address/relocatable space, that can be accessed

Background

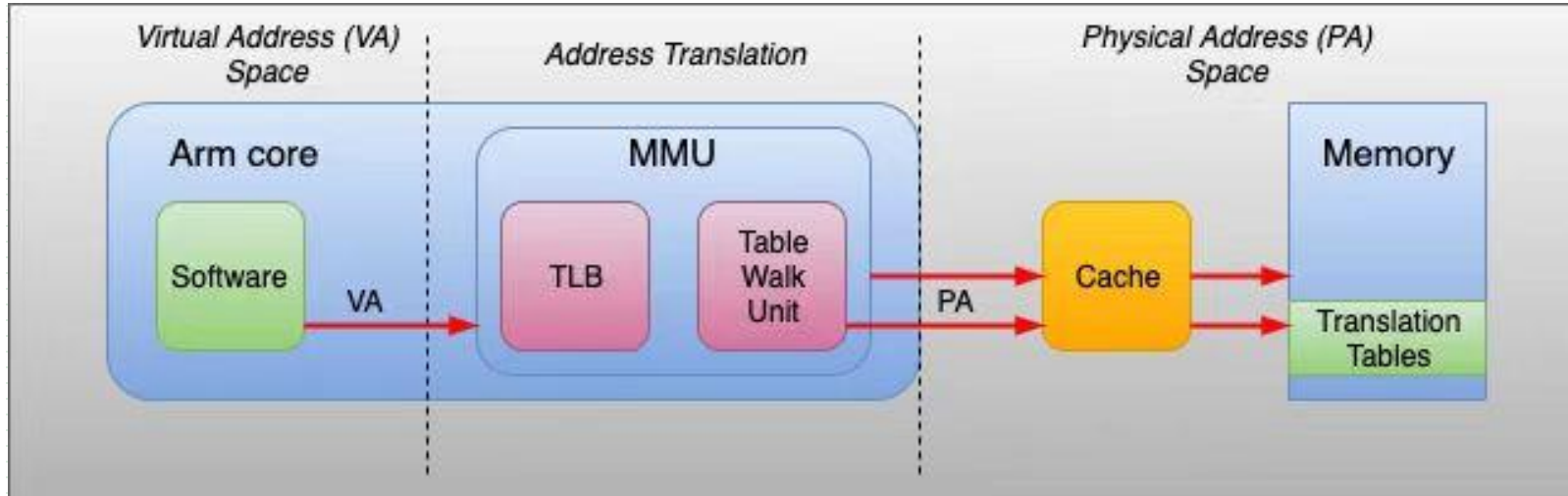
- Three address types, refer to Fig.8.0.0 
 - symbolic address
 - logical address
 - physical address
- Further, addresses represented in different ways *at different stages* of a program's life
 - Source code addresses usually **symbolic**
 - data and codes in programs
 - Compiled code addresses **bind** to **relocatable/logical** addresses
 - i.e. in Fig.8.0.0, **count** located at "14 bytes from beginning of this module"
 - Linker or loader will bind relocatable addresses to **absolute/physical** addresses that are accessed by CPU at instruction execution
 - i.e. **count** located at 70014
 - Each binding/mapping(地址联编/映射) maps one address space to another

Logical vs Physical address space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - ▣ **Logical address**
 - generated by the CPU
 - fetched from the instruction by CPU
 - ▣ **Physical address**
 - address seen by the memory.
- **Logical address space** is the set of all logical addresses generated by a process
- **Physical address space** is the set of all physical addresses generated by a process

Memory Management Unit (MMU) in CPU cores

- For contiguous allocation described in 8.3, MMU includes base registers and limit registers, used for
 - ▣ address binding
 - ▣ protection
- For non-contiguous allocation, e.g. paging or segmentation described in 8.4 and later, MMU includes
 - ▣ Translation Lookaside Buffer (TLB), for quick access on page table in Fig. 8.11
 - ▣ address binding hardware
 - e.g. relocation registers in Fig. 8.4, or Table Walk Unit in ARM CPU



Memory Management Unit (MMU) in CPU cores

- A pair of *base* register and *limit* register define the logical **address space**
 - ▣ Fig. 8.1
- With these two registers, CPU must check every memory access generated in user mode to be sure it is between base and limit for that user, protection of memory space is accomplished
 - ▣ Fig. 8.2

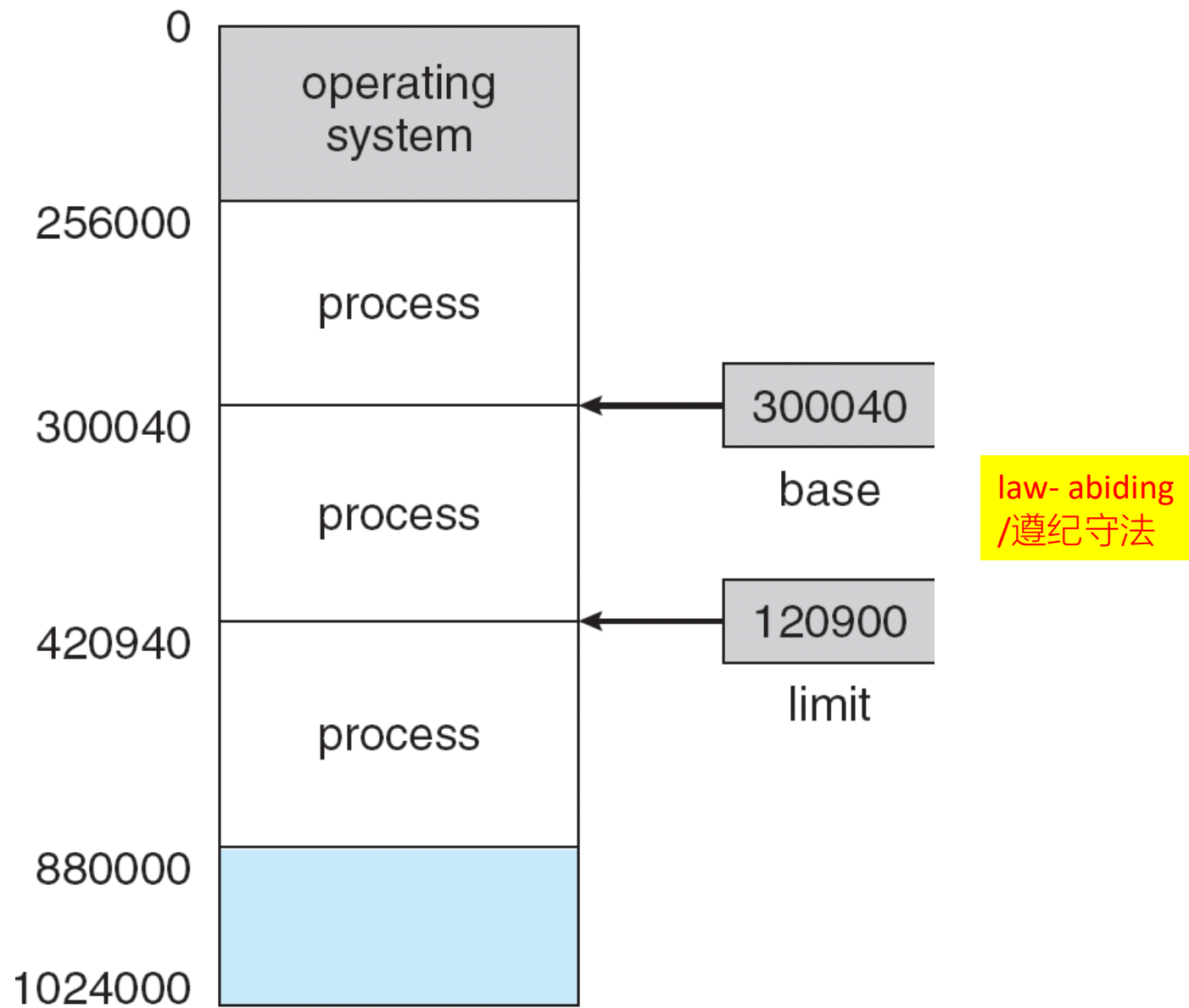


Fig. 8.1 A base and a limit register define a logical address space

LOAD A, X

寄存器A, 内存地址: X=9960

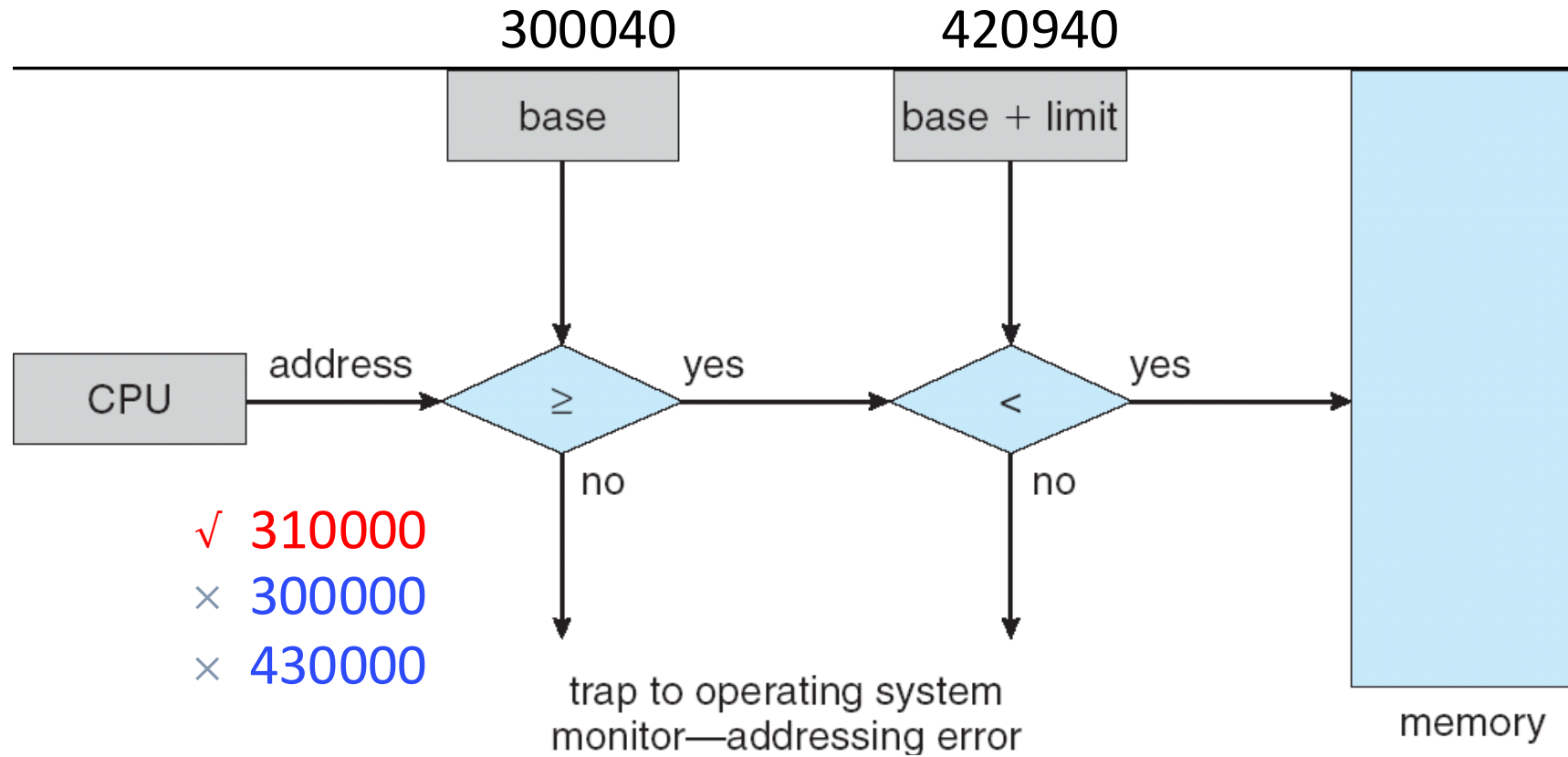


Fig. 8.2 Hardware address protection and binding with base and limit registers

Address Binding

- Logical and physical address spaces are defined with respect to different objects in Fig.8.3, and [address binding/relocating/mapping/translation](#) among these address spaces will happen during compiling, linkage, loading, and executing of programs
 - ▣ programs go through several steps before running in memory
 - ▣ as described in Fig.8.3 and Fig.8.0.0 
- Address binding ([绑定, 联编](#))/translation/relocation/mapping
 - ▣ address translation, i.e.,
 - 1) from symbolic address to relocatable address, or
 - 2) from relocatable address to absolute address 
 - ▣ refer to Fig.8.0.0, e.g. Fig.8.0.1

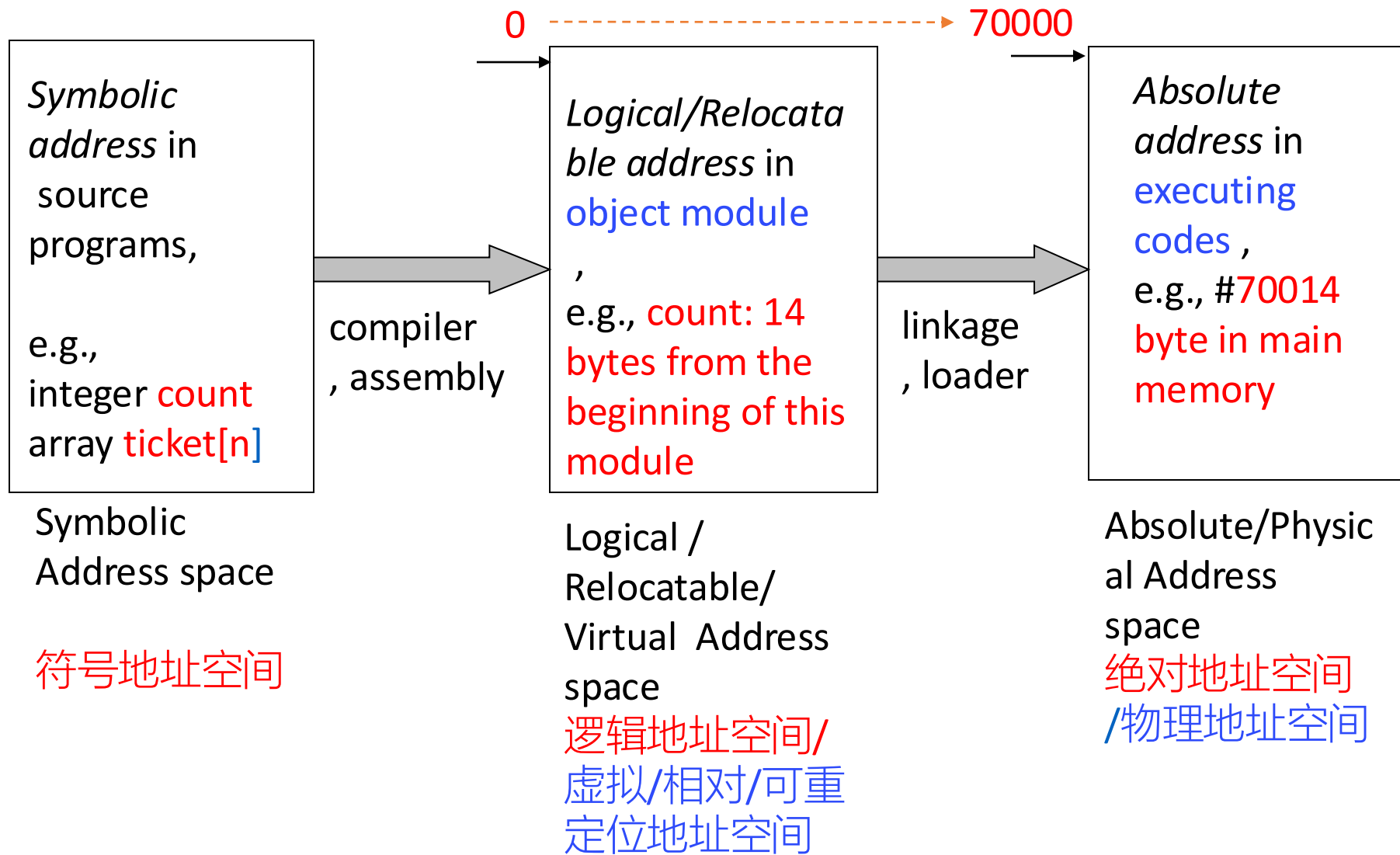
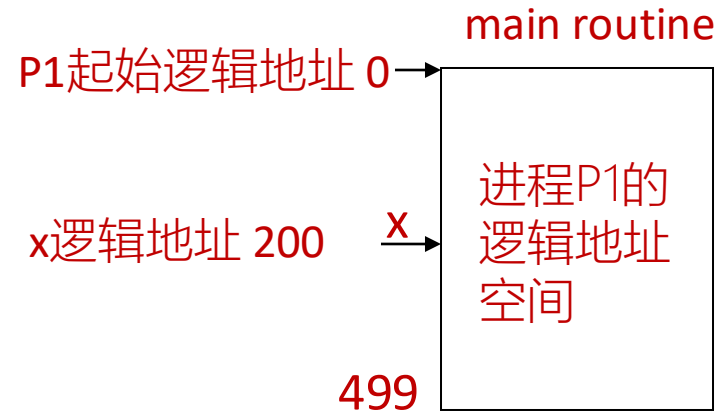


Fig.8.0.0 Address mapping among three address spaces

ADD A, x
STORE B, y



$x = x + 200$
relocation

P1起始物理地址 200

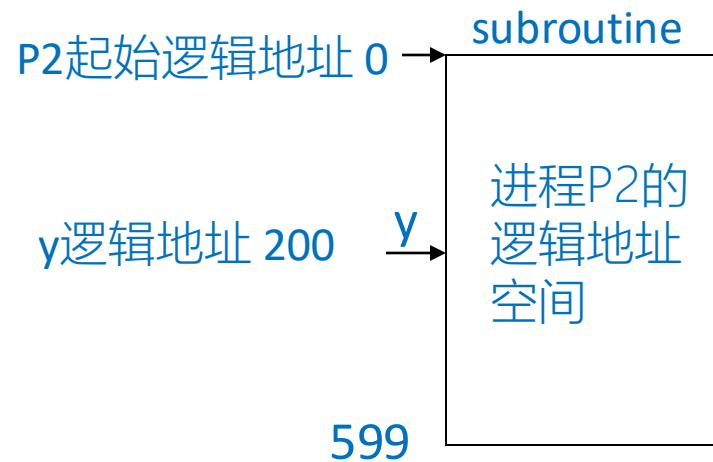
x物理地址 400

P1结束物理地址 699
P2起始物理地址 700

y物理地址 900

P2结束物理地址 1299

$y = y + 700$
relocation



物理地址空间

OS进程

200

进程P1的物理地址空间

500

进程P2的物理地址空间

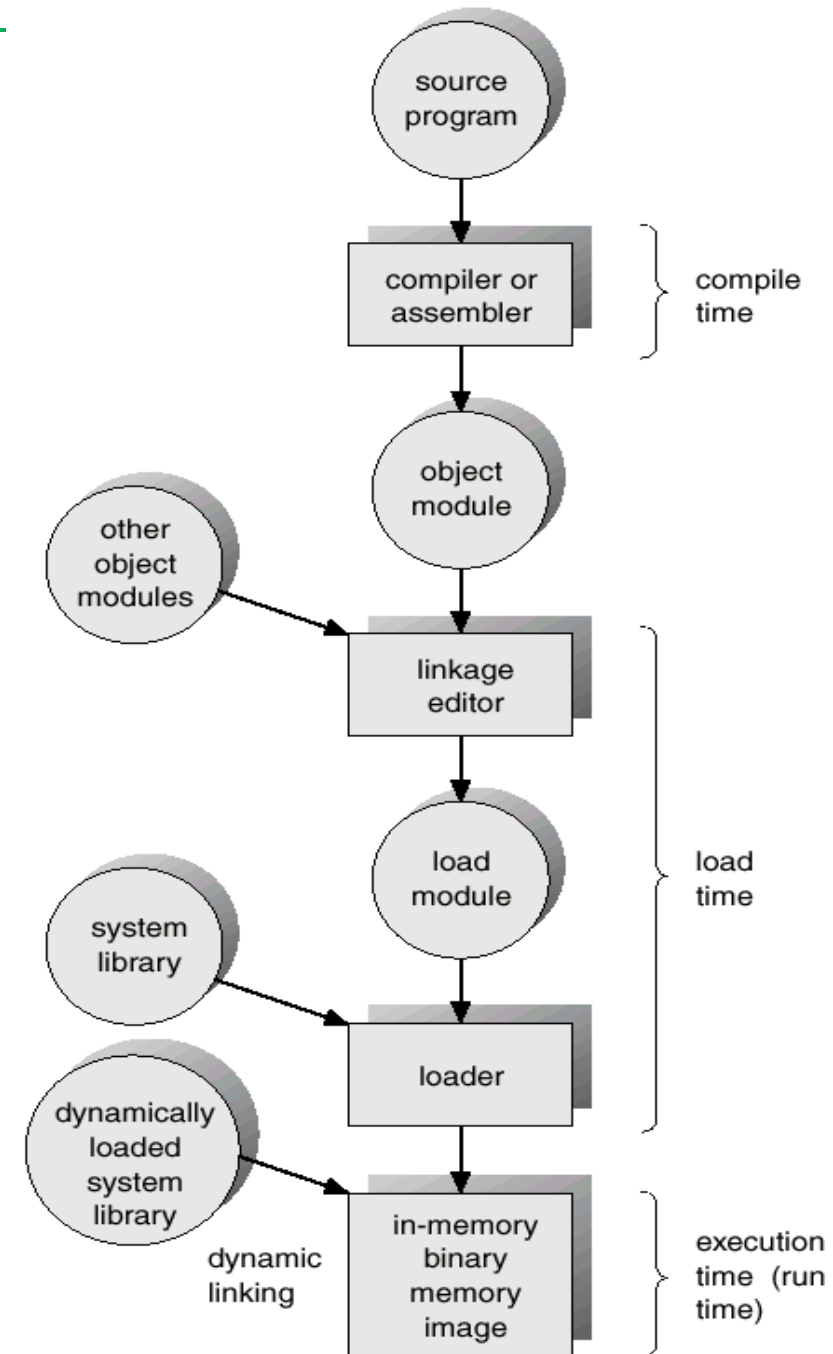
600

Fig. 8.0.1

Address binding/translation/relocation/mapping

Fig.8.3 Multi-steps processing of a user program

e.g. ▶



Address Binding

- Address **binding of instructions and data to memory addresses** can happen at three different stages
- **Compile time** (编译/汇编时产生绝对地址)
 - ▣ if memory location of programs is known a priori, **absolute code** (i.e., program codes with absolute address) can be generated **by compiler**;
 - e.g. ADD A, 400 (**200+200**) in Fig. 8.0.1, generated by compiler 
 - ▣ must recompile code if starting location changes.
 - e.g., MS-DOS.COM-format programs.
- **Load time** (程序进入内存时产生绝对地址) 
 - ▣ compiler must generate relocatable code (e.g. ADD A, 200) if memory location is not known at compile time, and absolute code is generated during load time by loader and linkage (e.g. ADD A, 400)

Address Binding

- Execution time 
 - ▣ binding delayed until run time if the process can be moved during its execution from one memory segment to another
 - 分配给进程的内存空间可动态改变，程序中各指令绝对/物理地址只能在指令执行时由硬件机制MMU动态产生
 - ▣ e.g. ADD A, 200 in memory
 - ▣ address mapping (i.e. absolute code generation) is conducted during program execution by hardware (e.g., base and limit registers, or relocation registers)

Address Binding

- Two types of binding/relocation/translation/mapping
 - ▣ static address relocation in **compile time and load time** (静态地址重定位, 指令执行前已完成了地址变换)
 - e.g. binding at **compile time**
binding at **load time**
 - ▣ dynamic address relocation in **execution time** (动态地址重定位, 指令执行时进行地址变换)
 - e.g. binding at **execution time**

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
 - contiguous, segmentation, paging
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
 - **Base register** now called **relocation register**
 - relocation registers (重定位寄存器) in Fig. 8.4
 - MS-DOS on Intel 80x86 used **4 relocation registers**
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory

Dynamic relocation using a relocation register

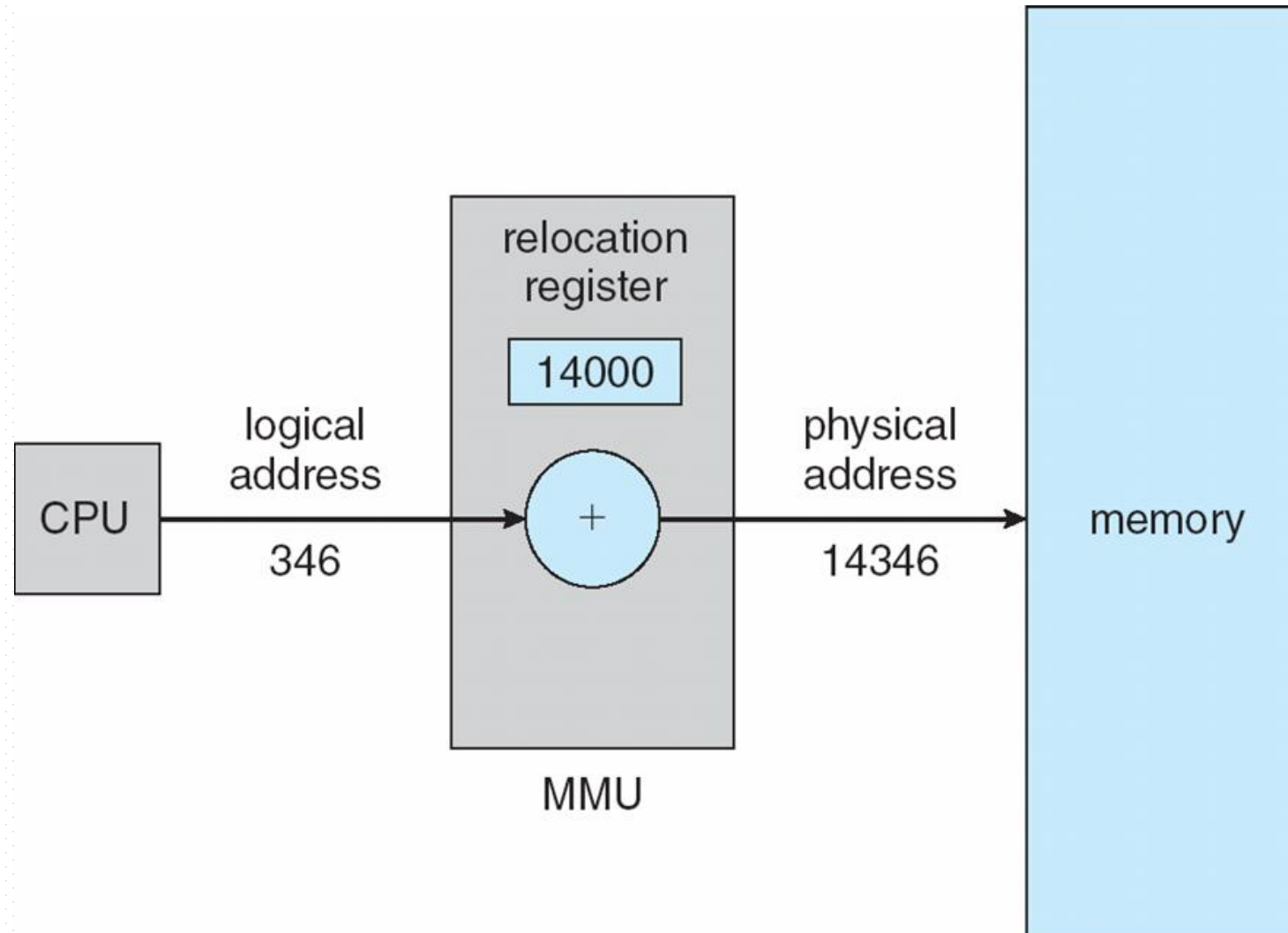


Fig.8.4 Dynamic relocation using a relocation register

Dynamic Loading

- Process consists of, e.g.
 - ▣ main program + routines + procedures + library functions
- The size of a process may be limited to the size of physical memory, it cannot be loaded entirely into main memory
- Dynamic loading
 - ▣ all *routines* are kept on disk in a relocatable load format, *routine is not loaded until it is called*
 - ▣ better memory-space utilization, unused routine is never loaded
 - ▣ supporting schemes of Virtual Memory (VM), e.g. demand paging and demand segmentation, as described in Chapter 9
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
 - ▣ Implemented through program design
 - ▣ OS can help by providing libraries to implement dynamic loading

Dynamic Linking and Shared Libraries

- **Static linking** – system libraries and program code combined by the loader (OS 加载器/装载器) into the binary program image
- **Dynamic linking** – linking postponed until execution time
- **Stub**, small piece of code, used to locate the appropriate memory-resident library routine (存根, 桩基模块)
 - ▣ a **routine** that contains no executable code and that generally consists of comments describing what will eventually be here
 - ▣ It is used as a placeholder for a routine to be written later. Also called *dummy stub*, top-down programming
- Stub replaces itself with the address of the routine, and executes the routine

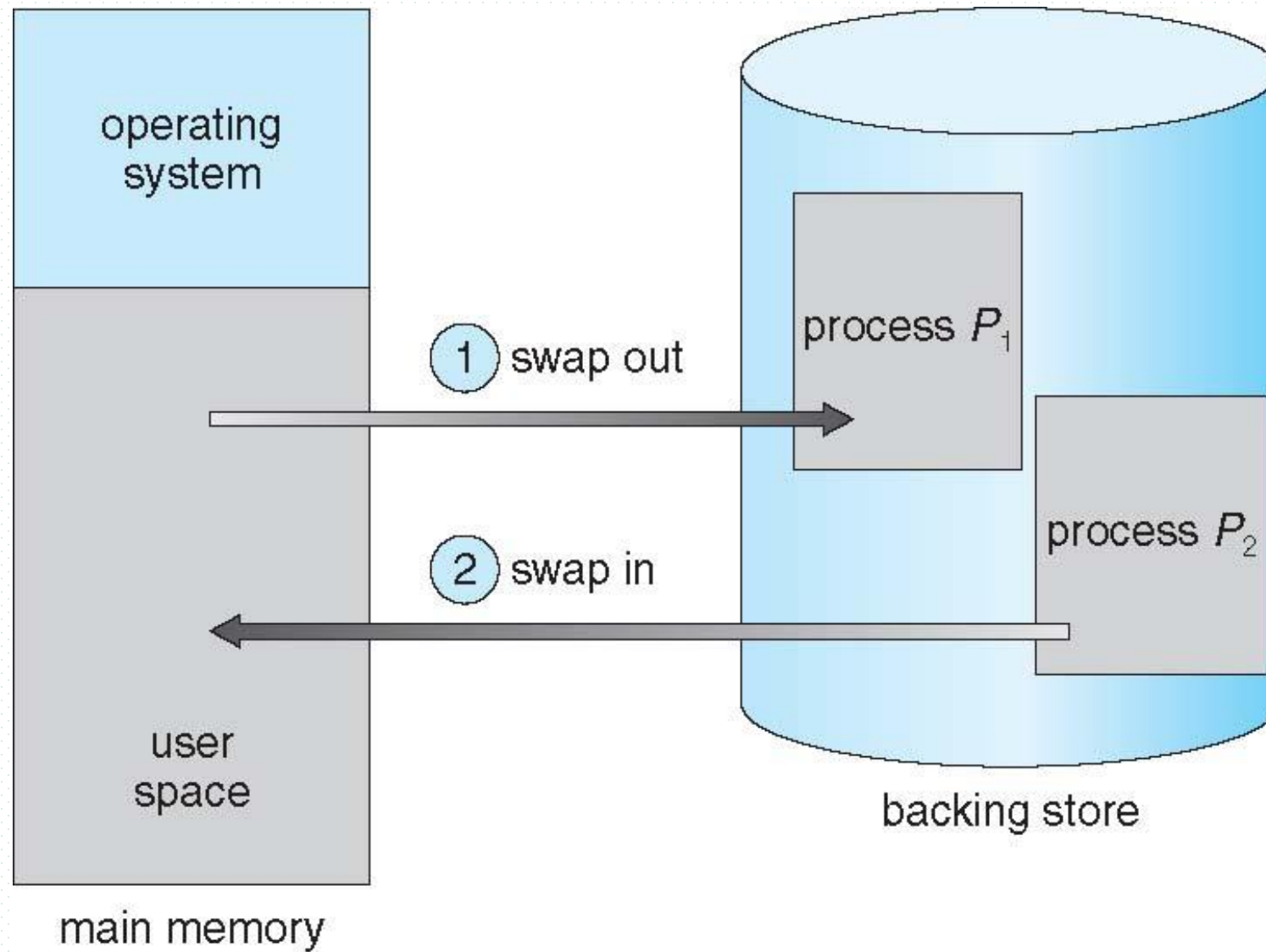
Dynamic Linking and Shared Libraries

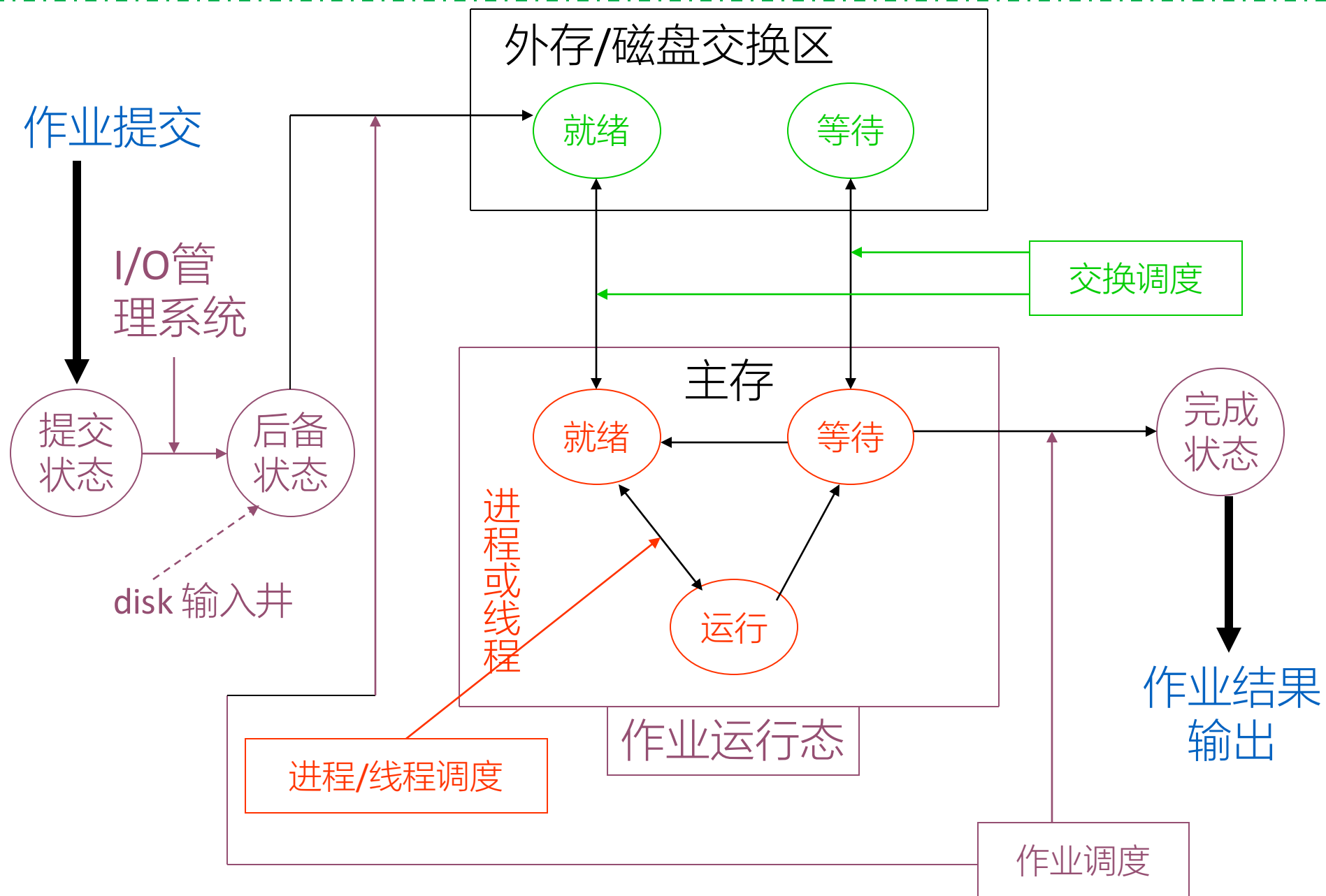
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
 - ▣ If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**

8.2 Swapping

- A process can be **swapped** temporarily out of memory to a **backing store** (e.g. swap file/space on disk), and then brought back into memory for continued execution
 - total physical memory space of processes can exceed physical memory
 - medium-term scheduling (中期调度), swap in, swap out
- **Swapping**
 - a memory extending/enlarging mechanism to accommodate more concurrent processes in systems
 - a process **that is in ready or waiting/blocked states** can be *swapped* temporarily out of memory to a **backing store**, and the memory originally occupied by the swapped out process can then be allocated to other process.
- **Backing store**(备份存储) – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
 - swap files
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms
 - lower-priority process is swapped out so higher-priority process can be loaded and executed

Schematic View of Swapping





Swapping (Cont.)

- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory (i.e. instructions and data in process) swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk
- Does the swapped out process need to swap back in to same physical addresses?
 - ▣ depends on address binding method
 - plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
 - ▣ swapping normally disabled // 正常情况下不允许交换
 - ▣ started if more than threshold amount of memory allocated // 分配内存过多, 超过阈值, 或空闲内存小于阈值, 启动交换
 - ▣ disabled again once memory demand reduced below threshold // 内存需求低于阈值, 或空闲内存数量超过阈值, 停止交换

AI4OS
in openEuler

Context Switch Time including Swapping

- Swapping results in **additional** process context switch time and overhead in kernel mode
- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
 - ▣ e.g. 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Swap out time of 2000 **ms**
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used

//事先知道进程真正需要的内存数量，只交换需要的部分，降低交换数量和交换时间

 - ▣ Processes use system calls to inform OS of memory use via `request_memory()` and `release_memory()`

Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
 - ▣ Pending I/O – can't swap out as I/O would occur to wrong process
 - ▣ Or always transfer I/O to kernel space, then to I/O device
 - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
 - ▣ But modified version common
 - Swap only when free memory extremely low

Swapping on Mobile Systems

- Swapping is generally not typically supported in mobile systems, why?
 - ▣ Flash memory based
 - Small amount of space
 - Limited number of write cycles
 - Poor throughput between flash memory and CPU on mobile platform
- If free memory is below a threshold, iOS **asks** apps to voluntarily relinquish allocated memory——控制degree of multiprogramming
 - ▣ read-only data (e.g. code) is thrown out and reloaded from flash if needed
 - ▣ application programs that fail to free enough memory are terminated by OS
- Android terminates apps if free memory is low, but first writes **application state** to flash for fast restarting the terminated apps
- Both OSes support paging as discussed below

移动设备一般不使用swapping

8.3 Contiguous Allocation

allocation methods, address binding and protection, fragments

- Contiguous allocation is one **early** method
- Main memory must support both OS and user processes
- Main memory usually into two **partitions**
 - Resident operating system, usually held in **low memory** (内存低端) with interrupt vector
 - User processes then held in **high memory** (内存高端)
- Each process is loaded into memory **entirely** before it can execute, and is contained in a **single contiguous** section of memory
 - 全部加载
 - 连续存放

Binding and Memory Protection

- Memory Management Unit MMU, including Base register and Limit register, maps logical address *dynamically* into physical address, i.e. address binding, and implement memory protection
 - ▣ protect user processes from each other, and
 - ▣ from changing operating-system code and data
- Base register contains value of smallest physical address, used for **address binding**
 - ▣ Fig. 8.4
- Limit register contains range of logical addresses, used for **memory protection**
 - ▣ each logical address must be less than the limit register

Memory Binding and Protection

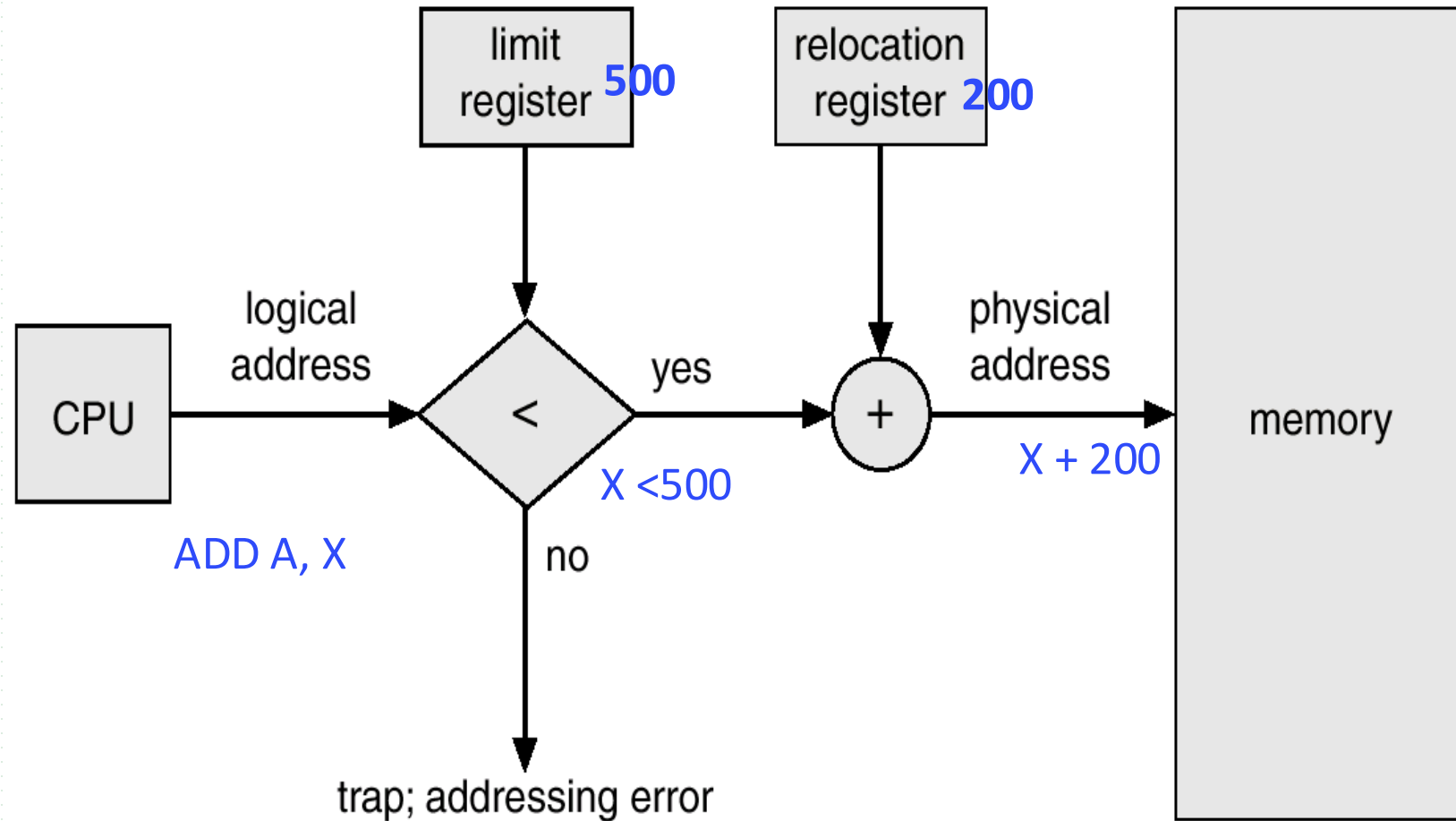
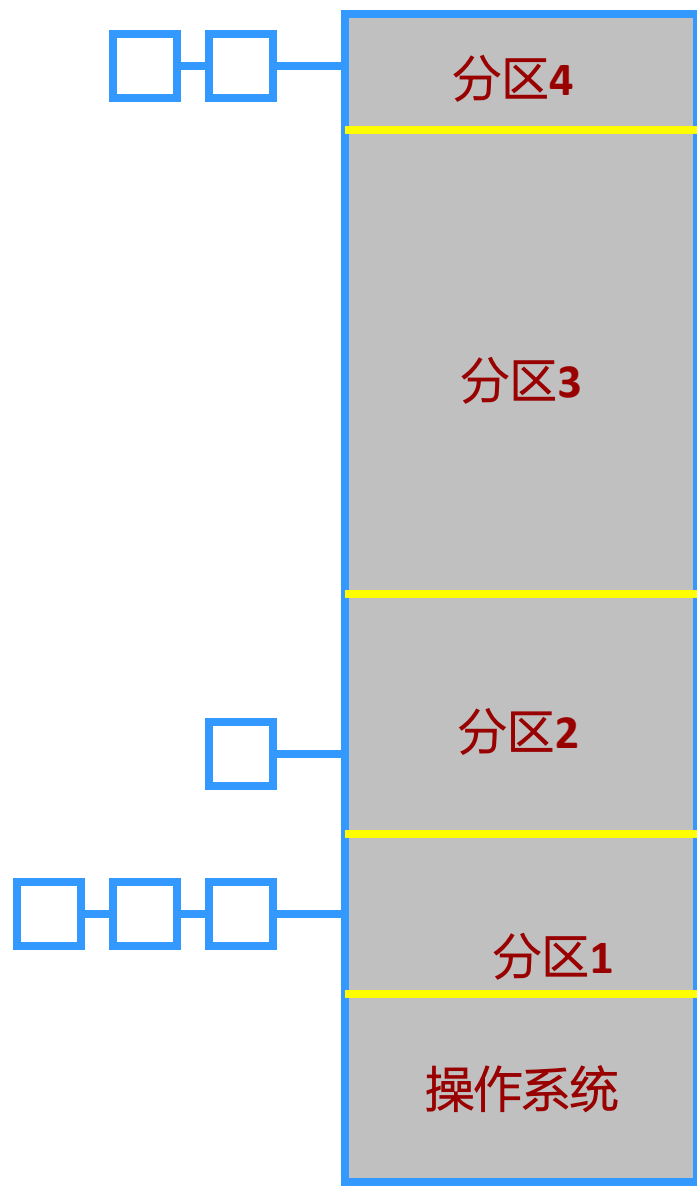


Fig.8.4 Memory protection and address binding

Memory Allocation Methods

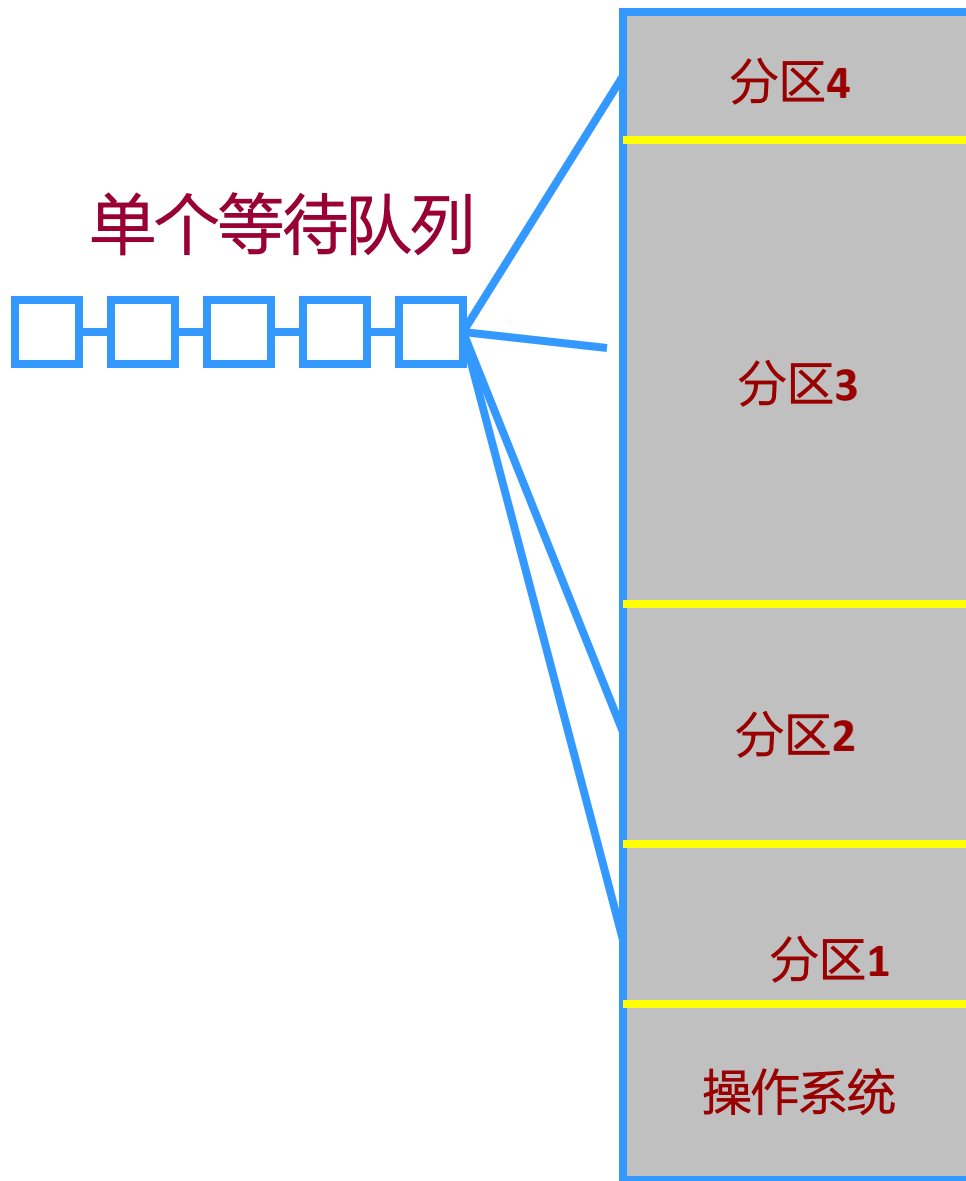
- Fixed-sized partitions (固定分区法)
 - ❑ memory is divided into several fixed-sized partitions, each partition may contains only one process
 - ❑ when a partition is free, a process is selected from input queue and loaded into this free partition.
 - ❑ degree of multiprogramming is bound by the number of partitions.
 - ❑ originally used by IBM OS/360 operating system, no longer in use
 - ❑ e.g. Fig. 8.0.2

多个等待队列



(a)

单个等待队列



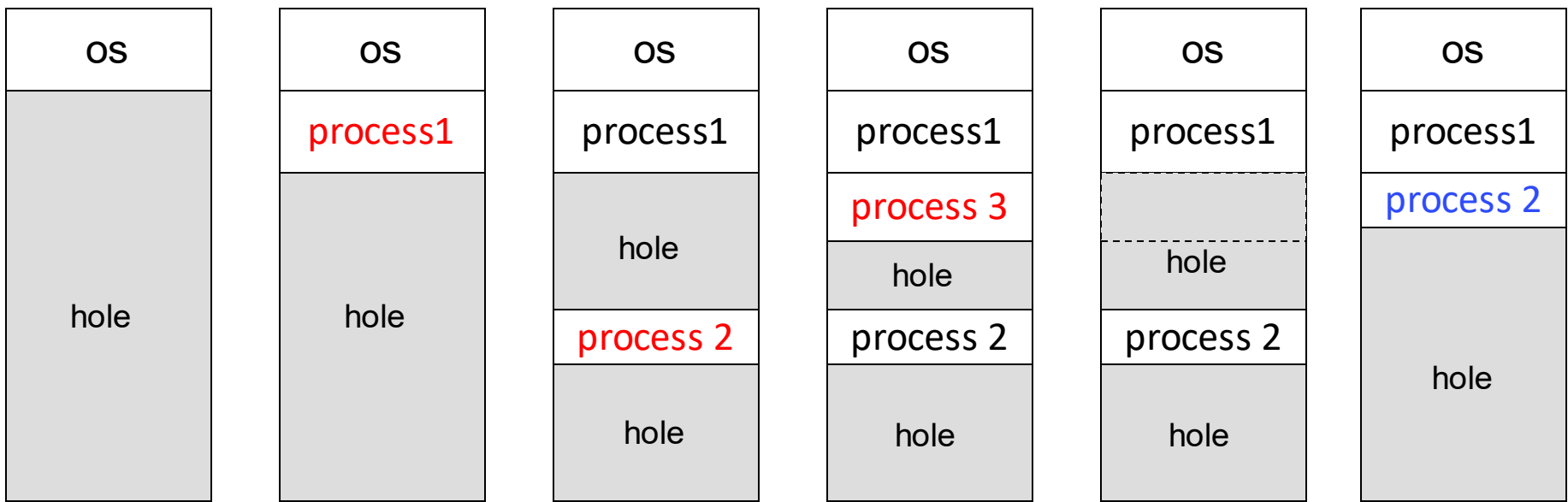
(b)

Fig. 8.0.2 Fixed-sized partitions



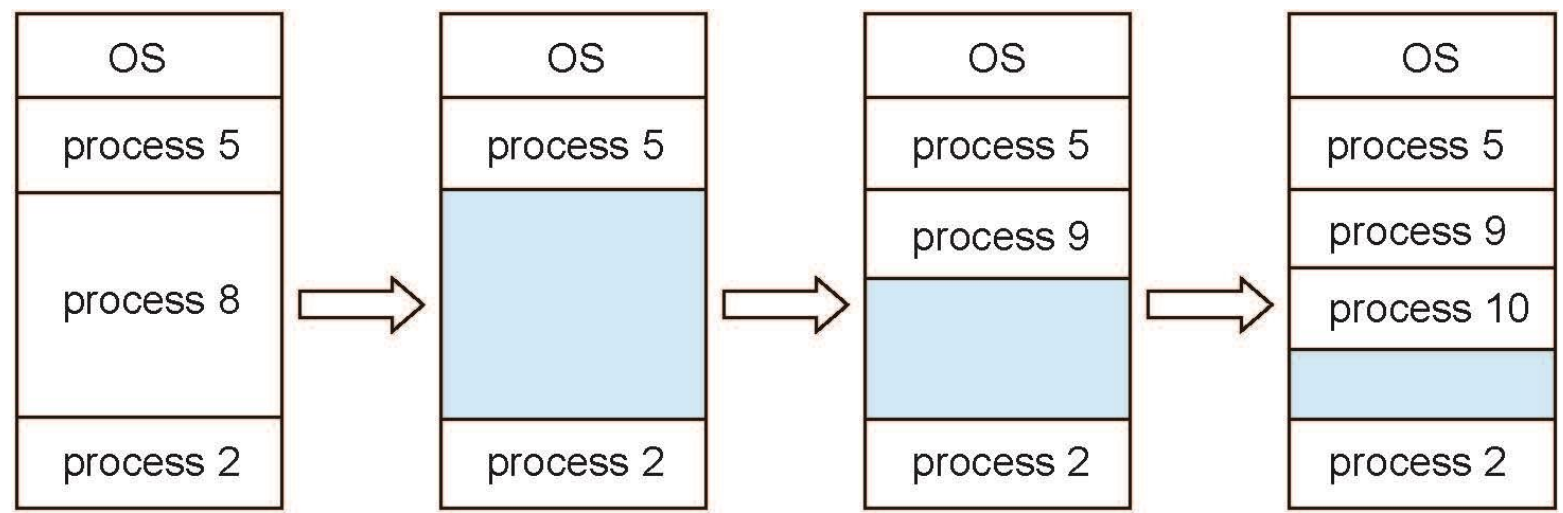
Contiguous Allocation: MVT(可变分区法)

- Memory consists of *allocated* partitions and *free* partitions (named *holes*)
 - ▣ **Hole**
 - block of available/free memory; holes of various size are scattered throughout memory
- OS keeps a data structure (called **partition table**) to indicate which parts of memory are available (holes) and which are occupied
- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- When a process release its memory occupied, a hole occurs, and adjacent holes are merged to form one large hole
- MVT is used primarily in the batch environments, but its ideas also applicable to a time-sharing environment in which pure segmentation is used for memory management (§8.5)



资源共享、按需分配、避免浪费，提高资源利用率

Fig. 8.0.3 An example of MVT



Dynamic Storage-Allocation Problem

- How to satisfy a request of size n from a list of free holes?
 - **First-fit**: Allocate the **first** hole that is big enough
 - used by boot-map allocator in Linux
 - **Best-fit**: Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
 - **Worst-fit**: Allocate the **largest** hole; must also search entire list
 - Produces the largest leftover hole
- 结论: Simulations have shown //仿真验证
 - both first fit and best fit are better than worst fit in terms of decreasing time and storage utilization 【是否有碎片?】
 - first-fit, best-fit存储利用率高, 产生的碎片少于worst-fit
 - neither first fit nor best fit is clearly better than the other in terms of storage utilization 【碎片】, but first fit is generally faster

408-17,B

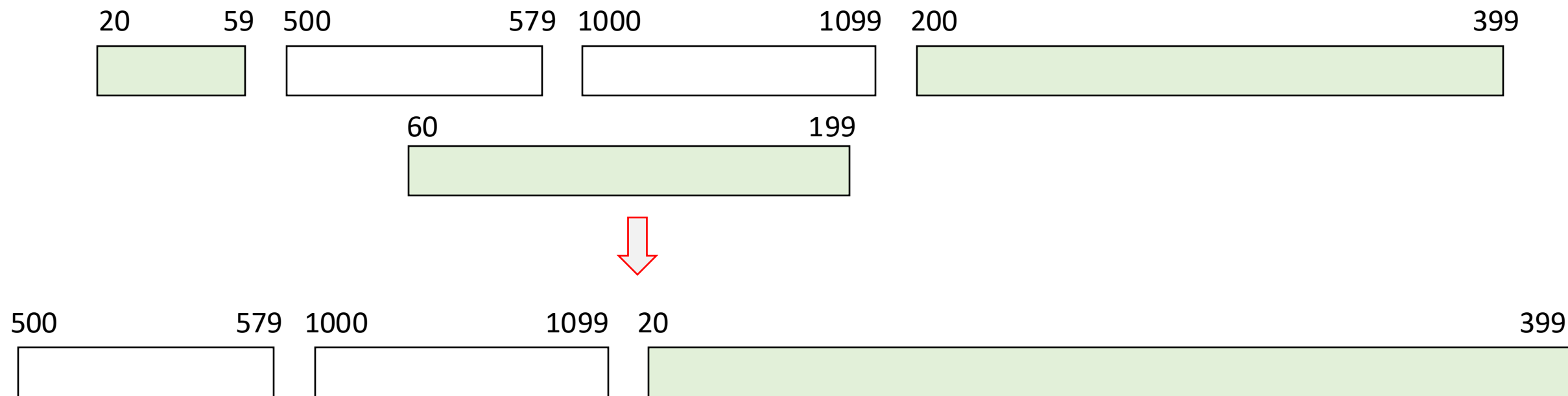
25. 某计算机按字节编址，其动态分区内存管理采用最佳适应算法，每次分配和回收内存后都对空闲分区链重新排序。当前空闲分区信息如下表所示。

分区起始地址	20 K	500 K	1000 K	200 K
分区大小	40 KB	80 KB	100 KB	200 KB

回收起始地址为60 K、大小为140 KB的分区后，系统中空闲分区数量、空闲分区链第一个分区的起始地址和大小分别是

- A. 3、20 K、380 KB B. 3、500 K、80 KB
C. 4、20 K、180 KB D. 4、500 K、80 KB

空闲块hole按照
size从小到大，
链接组织在一起！



例：动态内存分配malloc()/free() in C

动态内存分配空间在程序heap空间中

- `void *malloc(size_t size)`
 - ▣ 分配指定大小的内存空间，返回一个指向该空间的指针；size的单位：byte
 - ▣ 需要强制类型转换后才能引用其中的值
- `Void free(void *ptr)`
 - ▣ 释放由malloc所分配的内存空间，ptr指向要释放的内存块，该指针应当是之前调用malloc()的返回值
- E.g. `int* ptr;`
`ptr = (int*)malloc(10* sizeof(int));`
`free(ptr);`

动态内存分配malloc()/free() in C

- `int brk(void *addr)`
 - ▣ 用于返回堆的顶部地址
- `void sbrk(intptr_t increment)`
 - ▣ 用于扩展堆，提通过参数`increment`，指定要扩展、增加的内存大小；
 - ▣ 如果扩展成功，返回`btr`的旧值。如果`increment`为0，返回`brk`的当前值
- 先通过`sbrk`扩展堆，将这部分空闲内存空间作为缓冲池，然后采用`malloc/free`管理缓冲池中的内存
- `Malloc`使用空闲链表，组织堆中的空闲内存块
- C程序执行`malloc()`， 将从程序所在进程的堆中，采用`first/worst/best/next-fit`算法，为进程分配合适的内存

Fragmentation

- Fragmentation (碎片)
 - wasted/unavailable memory section, demerits for contiguous allocation
- External Fragmentation
 - several *noncontiguous* memory sections/partitions, total memory size of these sections is larger than a allocation request form a process
 - but the process can not be hold *contiguously* in the sections
- Internal Fragmentation
 - wasted memory internal to a partition
 - memory allocated to processes may be slightly larger than requested memory; this size difference is internal to a partition, but not being used
- Fixed-sized partition scheme results in external and internal fragmentation, while MVT leads to external fragmentation

- with respect to a memory request 500KB from incoming process 1, partition1, partition2, and partition4 are external fragments
- In view of process1, which is of size 500KB and resident in **partition3**, the size of *internal fragment* in partition3 is 100KB (= 600KB - 100KB)

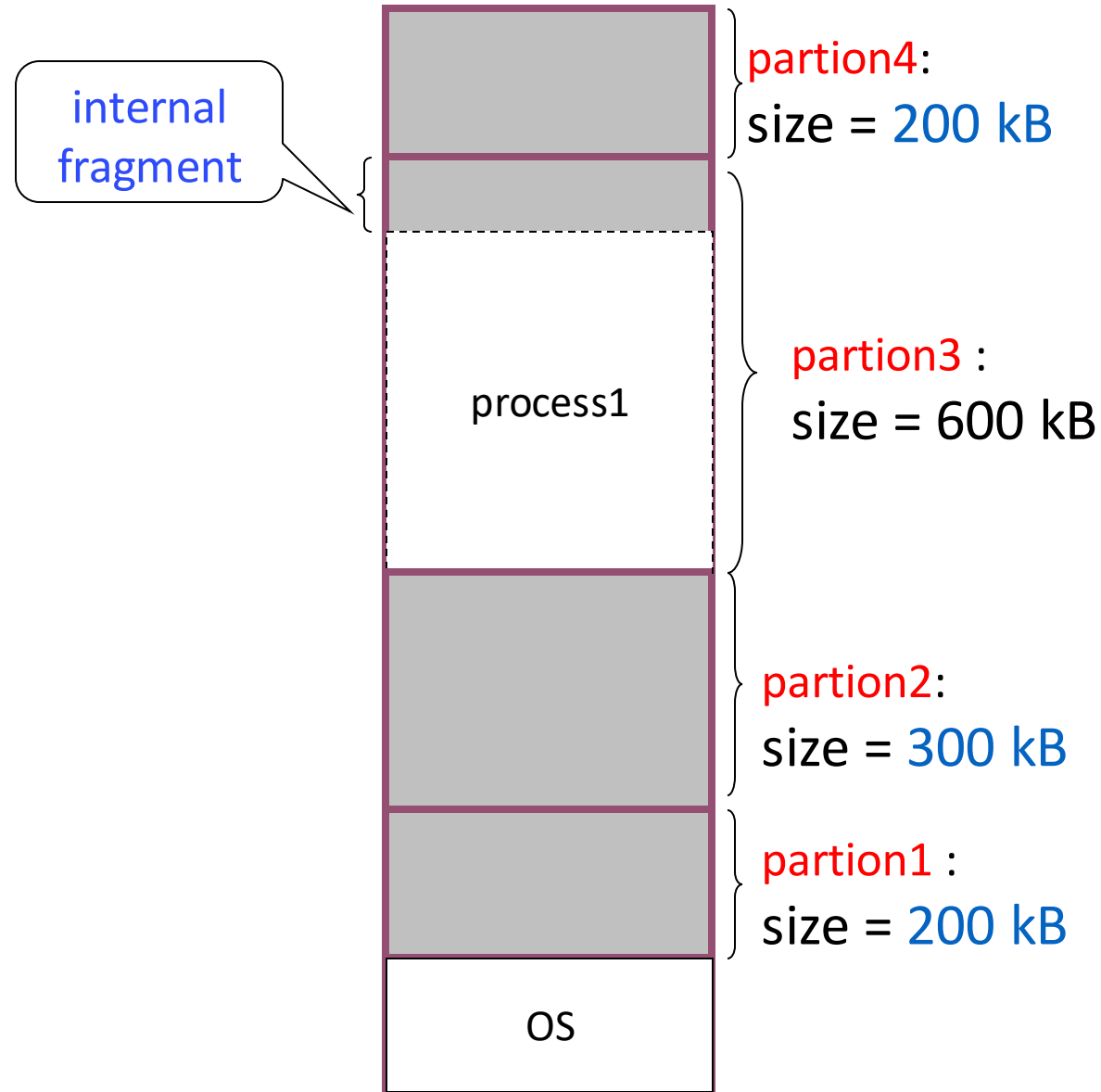


Fig. The external fragment and the internal fragment

Fragmentation

- With respect to the first fit scheme, statistic analysis reveals that given N blocks allocated, there may be $0.5 N$ blocks to be external fragmentation, and $1/3$ memory may be unusable
 - ▣ $1/3$ may be unusable -> **50-percent rule**
- Reduce external fragmentation by **compaction/内存规整** 
 - ▣ Shuffle memory contents to place all free memory together in one large block
 - ▣ Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - ▣ I/O problem
 - Latch job in memory while it is involved in I/O
 - Do I/O only into OS buffers
- Now consider that backing store has same fragmentation problems

8.4 Segmentation

- In view of users, a program is a collection of segments, on the basis of the application program structures
 - ▣ a segment is a logical unit in the program, such as that in Fig.8.18, segments may vary in their size. /*各段的大小可以不同
 - main program
 - procedure
 - function
 - method
 - object
 - local variables, global variables
 - common block
 - stack
 - symbol table
 - arrays
 - ▣ the logical address space is in accordance with the user view of memory
- /*进程的逻辑地址空间按内容或过程 (函数) 关系分成段，每段有自己的名字

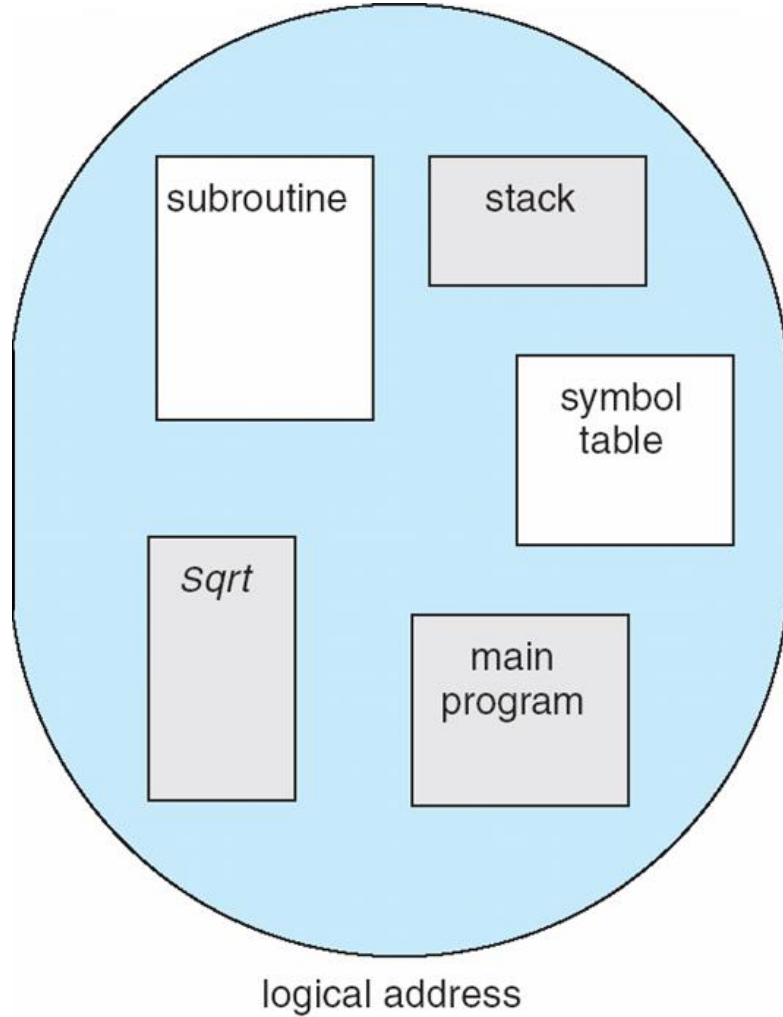


Fig. User's View of a Program

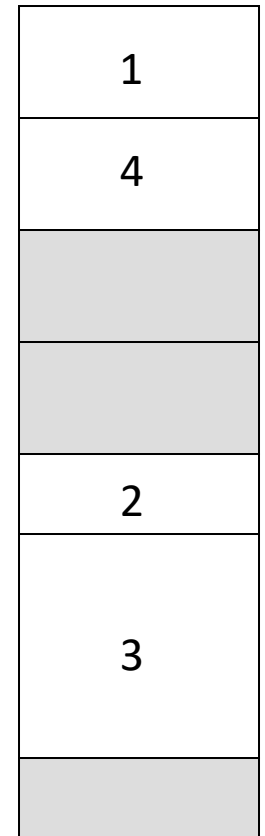
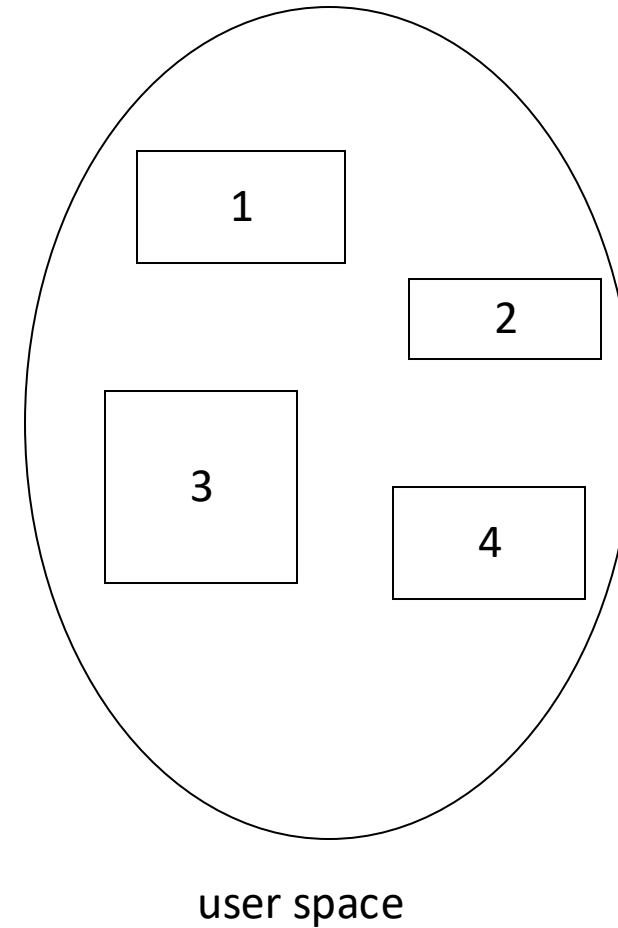


Fig. Logical View of Segmentation

Segmentation

- Logical address consists of a two tuple:

<segment-number, offset>,  逻辑地址是二维的，由段号和段内地址构成

- Each segmentation is allocated in one contiguous memory section, and all memory sections of one program may be noncontiguous
 - ▣ 以段为单位非连续分配内存

进程逻辑地址空间由编程者根据程序的结构 (子程序/模块/数据区等) 划分为多个段, 逻辑地址由段号 + 段内地址构成: (s, w)

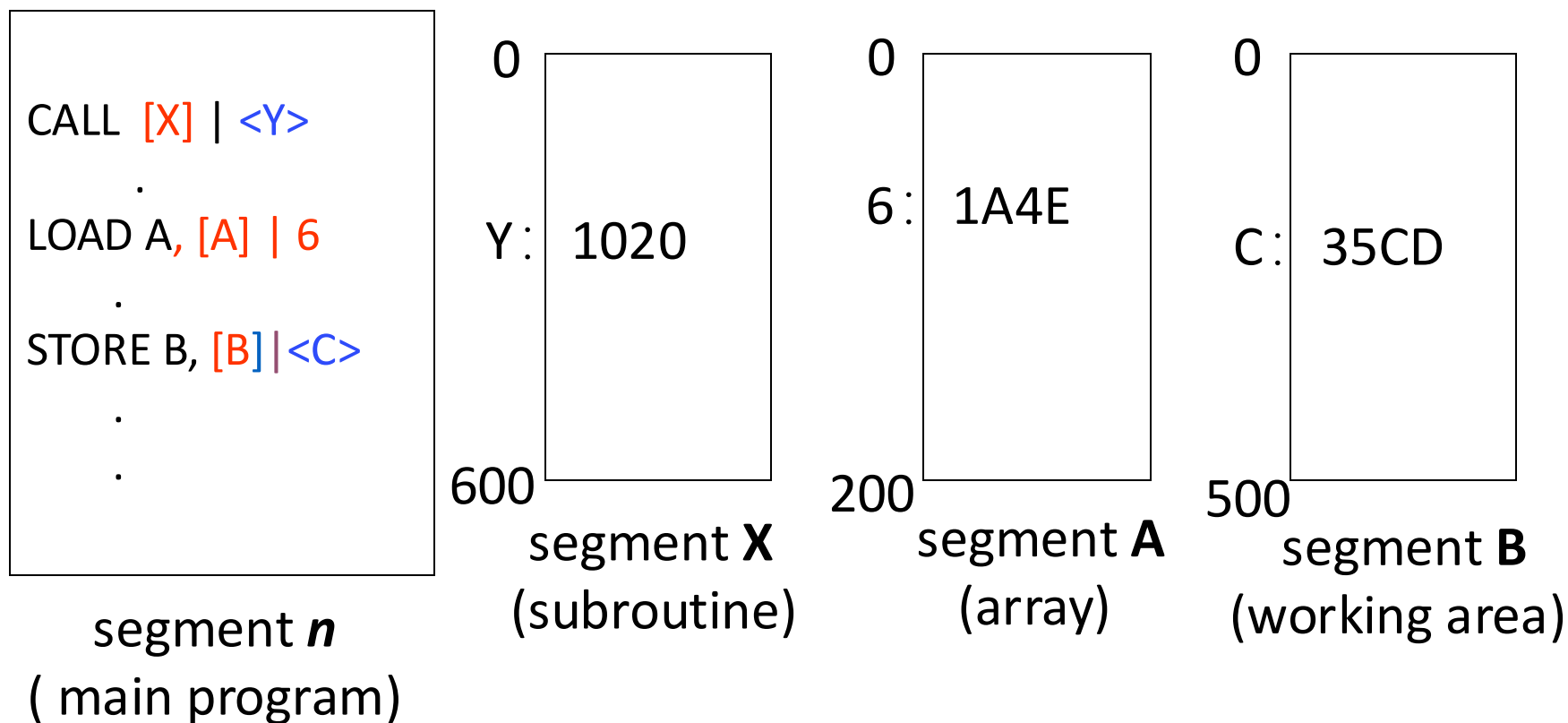
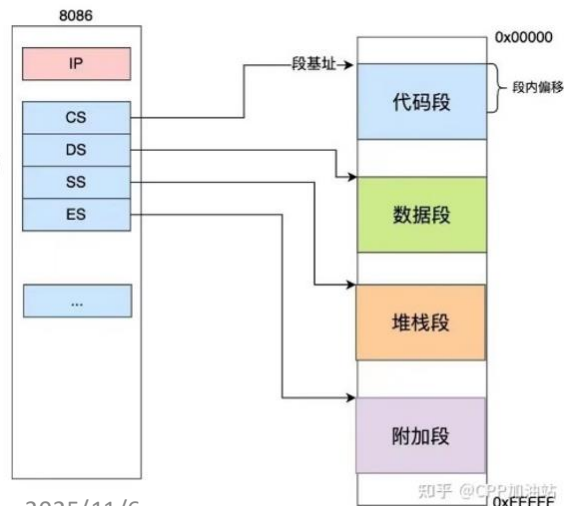


Fig. 8.17.1 A process with 4 segments in the logical space

History of Segmentation and Paging

分段

1978 年英特尔又研究成功了第一款 16 位 CPU - 8086, 这款 CPU 可以说是 x86 系列的鼻祖了, 设计了 16 位的寄存器和 20 位的地址总线, 所以内存地址可以达到 2^{20} Byte 即 1M, 极大地扩展了地址空间, 但是问题来了, 由于寄存器只有 16 位, 那么 16 位的 IP 寄存器如何能寻址 20 位的地址呢, 首先 Intel 工程师设计了一种分段的方法: 1M 内存可以分为 16 个大小为 64 K 的段, 那么内存地址就可以由「段的起始地址 (也叫**段基址**) + **段内偏移** (IP 寄存器中的值)」组成, 对于进程说只需要关心 4 个段, 代码段, 数据段, 堆栈段, 附加段, 这几个段的段基址分别保存在 CS, DS, SS, ES 这四个寄存器中



这四个寄存器也是 16 位, 那怎么访问 20 位的内存地址呢, 实现也很简单, 将每个寄存器左移四位, 然后再加上段内偏移即为寻址, CPU 都是取代码段中的指令来执行的, 我们以代码段内的寻址为例来计算内存地址, 指令的地址 = $CS \ll 4 + IP$, 这种方式做到了 20 位的寻址, 只要改变 CS, IP 的值, 即可实现在 0 到最大地址 0xFFFFF 全部 20 位地址的寻址

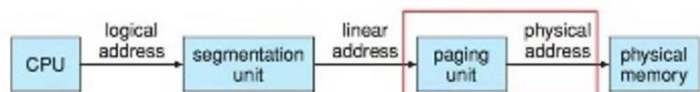
内存分页

1985 年 intel 推出了 32 位处理器 80386, 也是首款支持分页内存的 CPU

和分段这样连续分配一整段的空间给程序相比, 分页是把整个物理空间切成一段段固定尺寸的大小, 当然为了映射, 虚拟地址也需要切成一段段固定尺寸的大小, 这种固定尺寸的大小我们一般称其为页, 在 Linux 中一般每页的大小为 4KB, 这样虚拟地址和物理地址就一页来映射起来了

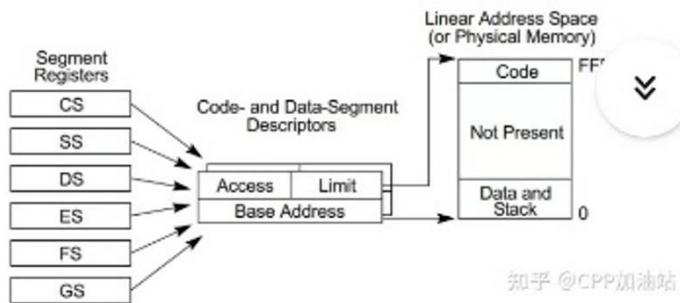
History of Segmentation and Paging

既然分页这么好，那么分段是不是可以去掉了呢，理论上确实可以，但 Intel 的 CPU 严格执行了 backward compatibility（回溯兼容），也就是说最新的 CPU 永远可以运行针对早期 CPU 开发的程序，否则早期的程序就得针对新 CPU 架构重新开发了（早期程序针对的是 CPU 的段式管理进行开发），这无论对用户还是开发者都是不能接受的（别忘了安腾死亡的一大原因就是由于不兼容之前版本的指令），兼容性虽然意味着每款新的 CPU 都得兼容老的指令，所背的历史包袱越来越重，但对程序来说能运行肯定比重新开发好，所以既然早期的 CPU 支持段，那么自从 80386 开始的所 CPU 也都得支持段，而分页反而是可选的，就意味着这些 CPU 的内存管理都是段页式管理，逻辑地址要先经过段式管理单元转成线性地址（也称虚拟地址），然后再经过页式管理单元转成物理内存，如下



分页是可选项

在 Linux 中，虽然也是段页式内存管理，但它统一把 CS, DS, SS, ES 的段基址设置为了 0，段界限也设置为了整个虚拟内存的长度，所有段都分布在同一个地址空间，这种内存模式也叫平坦内存模型（flat memory model）



平坦内存模型

我们知道逻辑地址由段选择子：段内偏移地址组成，既然段选择子指向的段基地址为 0，那也就意味着段内偏移地址即为即为线性地址（也就是虚拟地址），由此可知 Linux 中所有程序的代码都使用了虚拟地址，通过这种方式巧妙地绕开了分段管理，分段只起到了访制和权限的作用（别忘了各种权限检查依赖 DPL, RPL 等特权字段，特权极转移也依赖于段选择子中的 DPL 来切换的）

Segmentation

- Address binding are conducted on the basis of the **segmentation table** for each process
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - ▣ **base** – contains the starting physical address where the segments reside in memory
 - ▣ **limit** – specifies the length of the segment
 - ▣ refer to Fig.8.17.2
- Use Segment table to conduct address mapping
 - ▣ Fig. 8.19
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
segment number **s** is legal if **s** < **STLR**
- A segmentation example is shown in Fig. 8.20

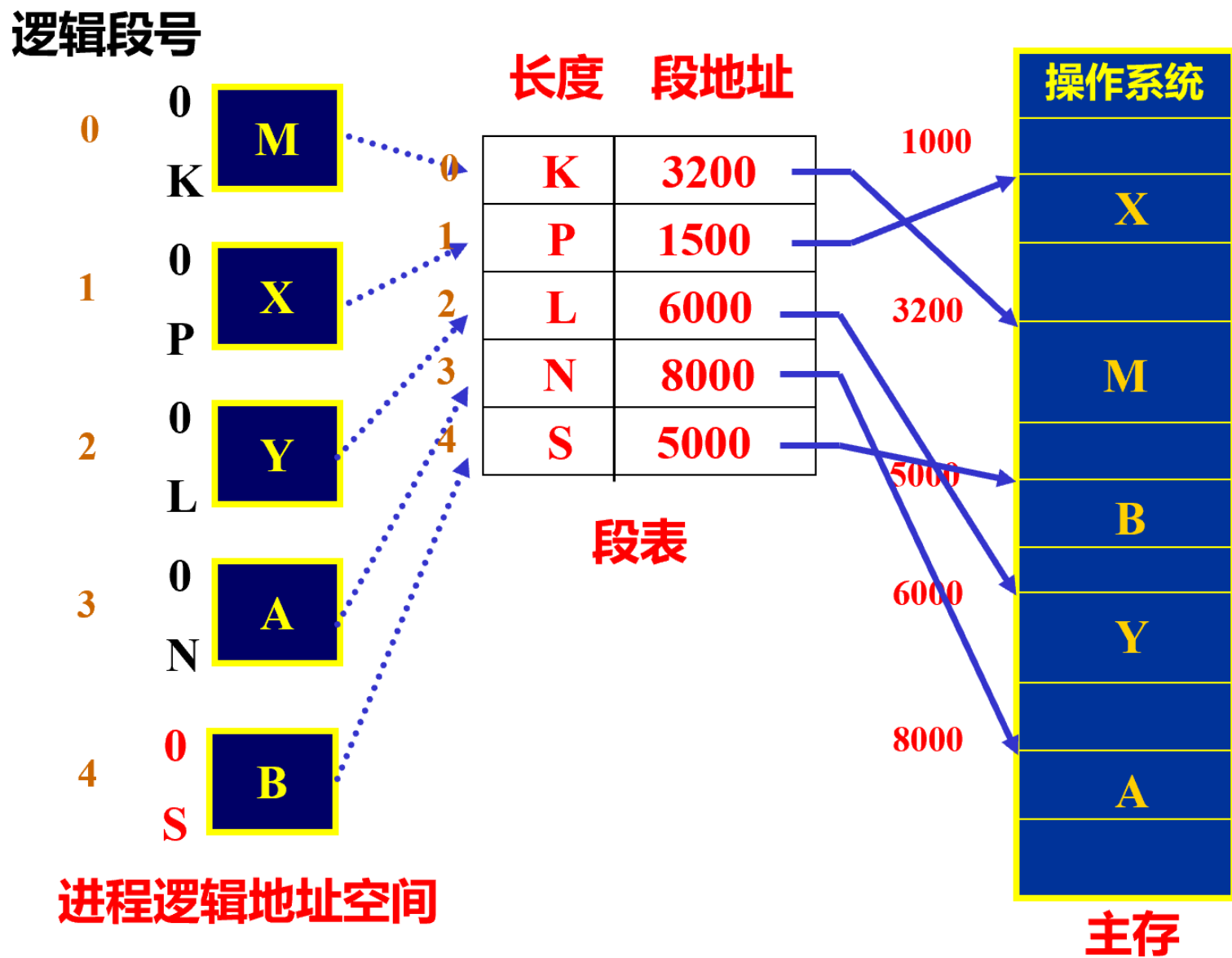


Fig. 8.17.2

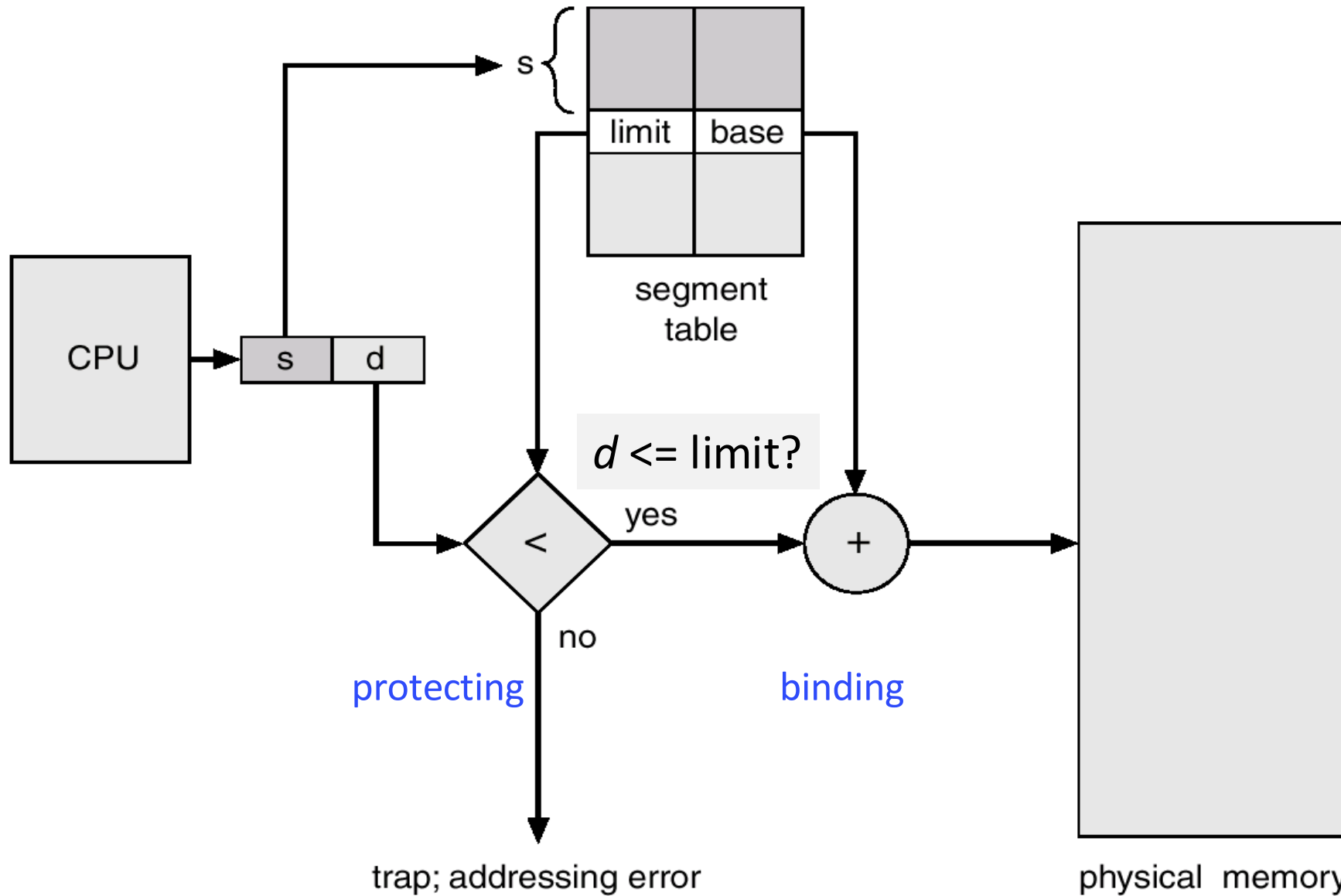
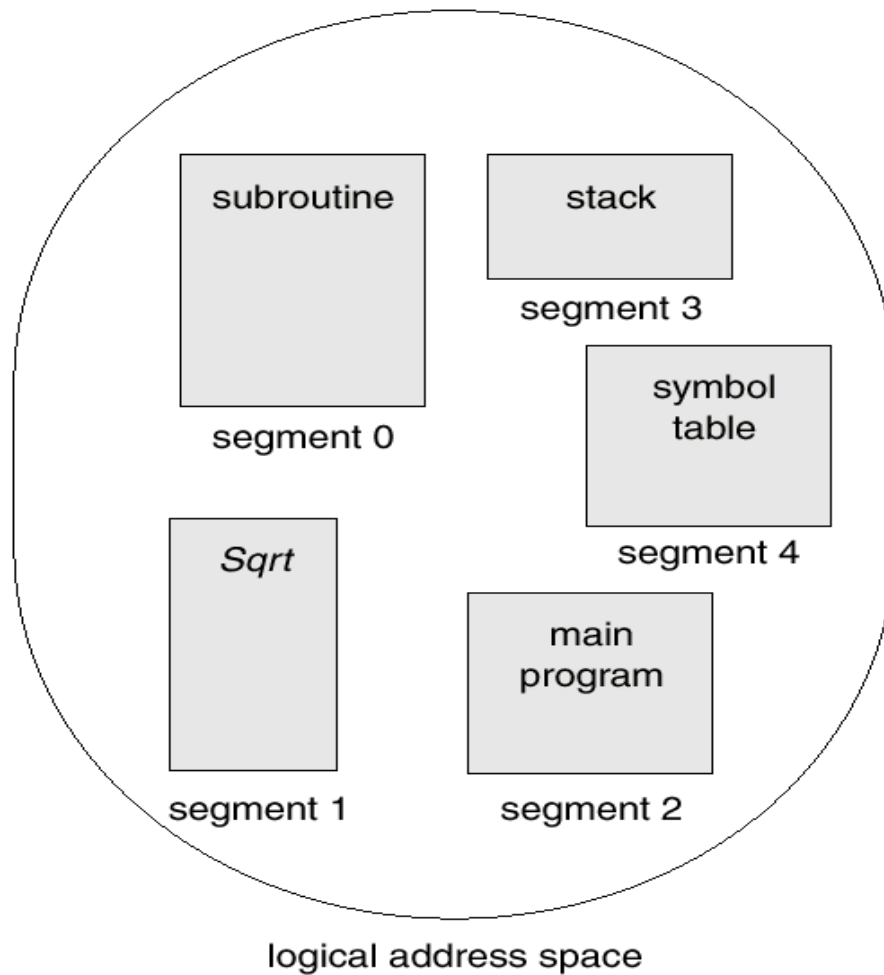


Fig.8.19 Address mapping and protecting via segmentation hardware

Segmentation

- Protection
 - ▣ With each entry in segment table associate:
 - validation bit = 0 $\Rightarrow$ illegal segment
 - read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
 - ▣ first, best, worst fit
- Segmentation may results in external fragmentation
 - ▣ Why? each segment must be allocated to a contiguous memory section



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table

segments vary
in sizes

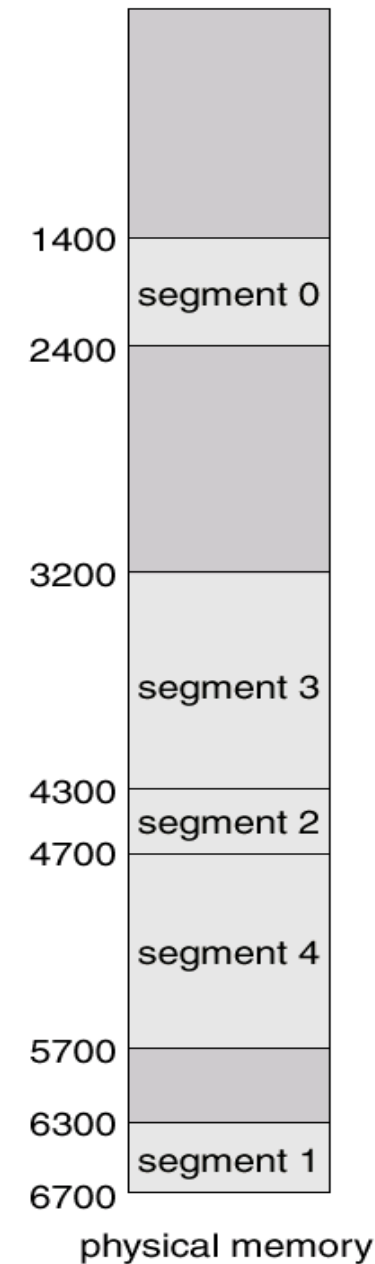
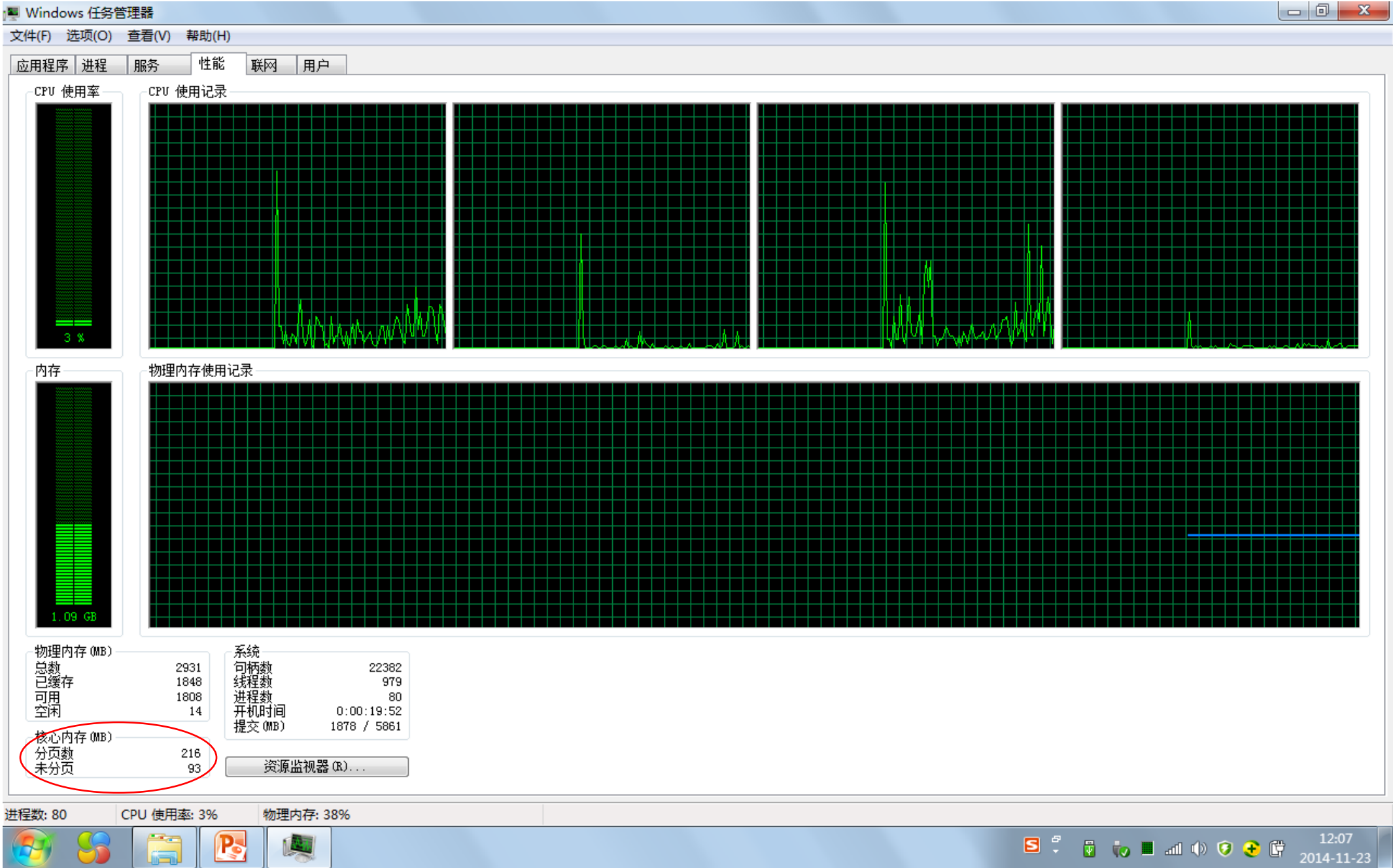


Fig.8.20 An example of segmentation



8.5 Paging

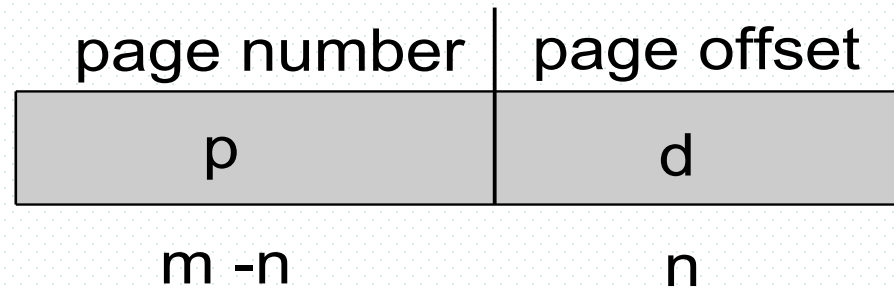
- Paging
 - ❑ a memory management scheme permitting the **physical address space** of processes to be **non contiguous**.
 - ❑ commonly used in most operating systems
 - ❑ advantages,
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- Physical memory is divided into fixed-sized blocks called **frames** (页架, 页框, 帧, 框架), the size of each frame is power of 2, e.g. between 512 bytes and 8192 bytes
- Logical memory is divided into blocks of same size, called **pages** (页)
- For a program of n pages in logical address space, n free frames are needed to load the entire program once at a time, and n frames allocated to the program may be noncontiguous, i.e. physical address space of a process can be noncontiguous.

Paging

- **Page tables** used to implement address binding/mapping for processes in systems 
- **Frame table** used to record memory allocation (which frames are free, and which are allocated) 

Address Binding/Translation Scheme

- Address generated by CPU is divided into:
 - ▣ **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - ▣ **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit



- ▣ E.g. for given logical address space 2^m and page size 2^n , e.g. $n=12$, $m=32$
- /* 对用户而言，逻辑编程空间仍然是一维连续的，由OS/CPU在内部将其转换为二维地址并映射到非连续的物理地址空间！！

Address Binding

- When a process is created and loaded into main memory, OS sets up a **page table** to translate its logical address to the physical address
- Page table 
 - a kernel data structure containing the **base address of** each page in physical space, i.e., the initial address of its corresponding frame, that is
 - page number $\rightarrow$ high bits of the initial address of the corresponding frame
- Address binding based on page table, Fig.8.7
 - **logical address** $\rightarrow$ **physical address**
i.e. $p * 2^n + d \rightarrow f * 2^n + d$
 - physical address: initial frame address + page offset
- Note
 - f is the $(m-n)$ high bits of the initial address of the frame corresponding to the page
 - f is combined /concatenated with n -bit page offset d , thus resulting in the physical address

Paging hardware for address binding

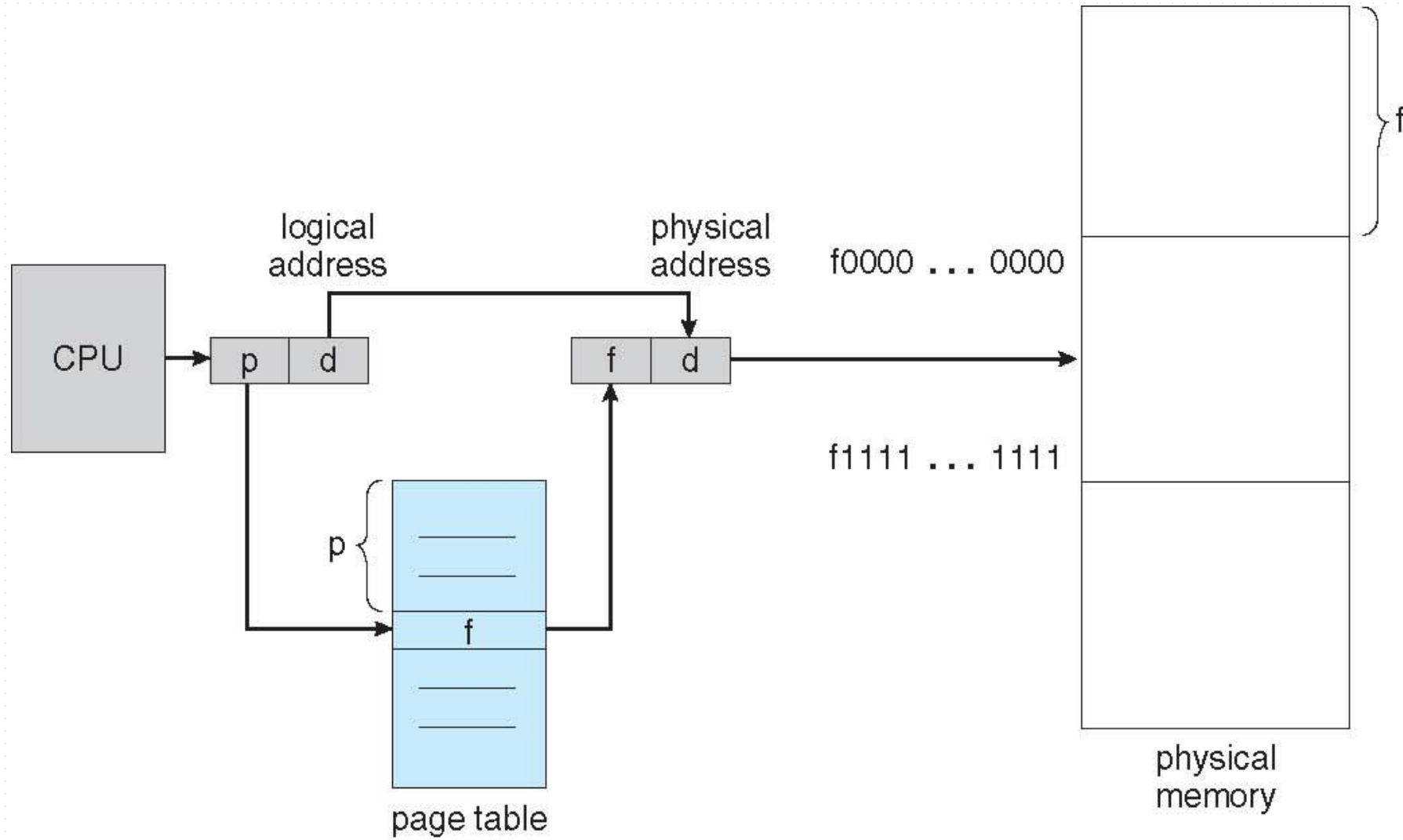


Fig.8.7 Paging hardware for address translation

Paging

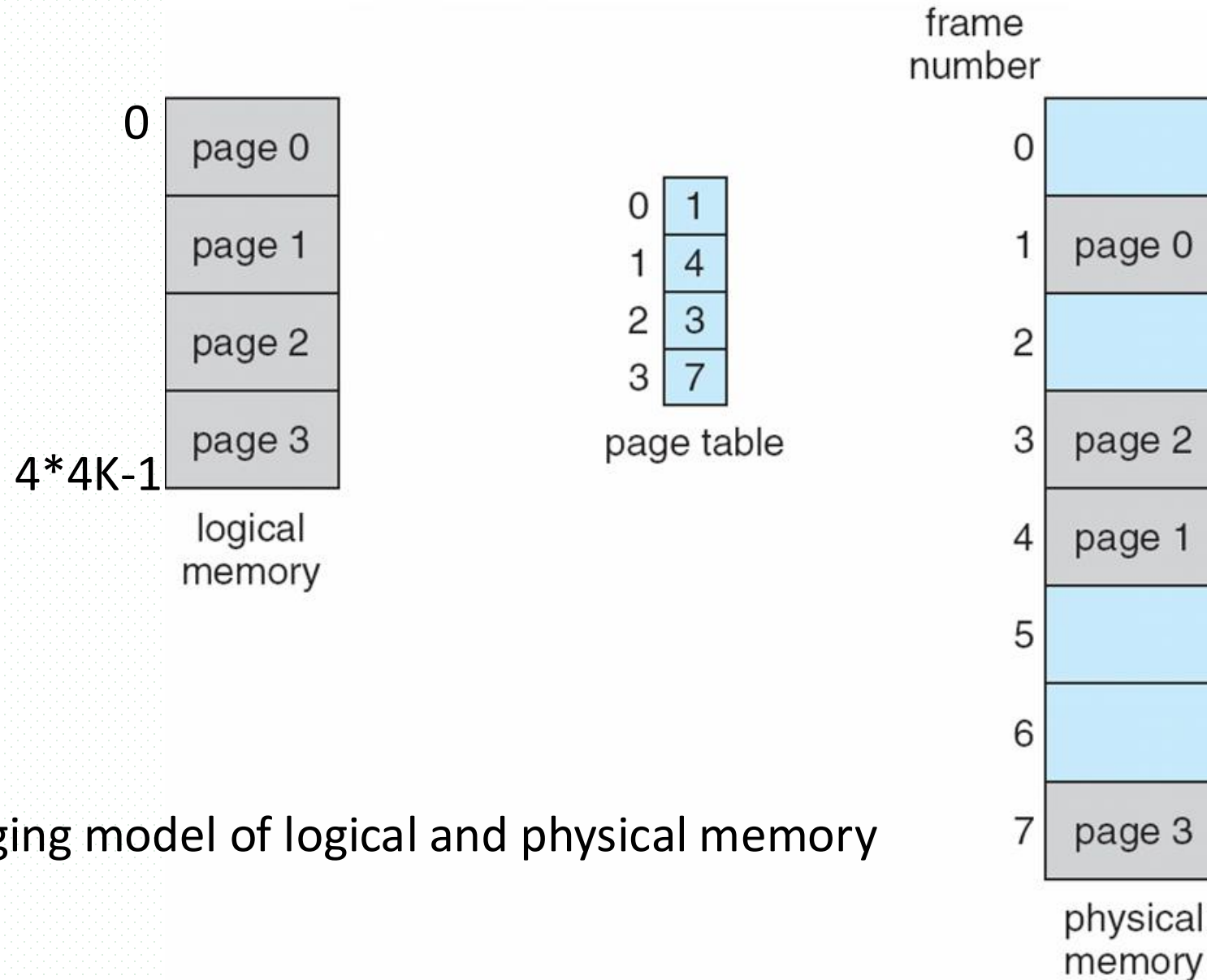


Fig.8.8 Paging model of logical and physical memory

page size:
4 bytes

Logical space:
4 pages
= 2^4 bytes = 16 bytes
logical address: 4 bits

Physical memory :
8 frames
= 2^5 bytes = 32 bytes
physical address: 5 bits

		P# :
0	a	0
1	b	
2	c	
3	d	
4	e	1
5	f	
6	g	
7	h	
8	i	2
9	j	
10	k	
11	l	
12	m	3
13	n	
14	o	
15	p	

logical memory

		f#:
0	5	
1	6	
2	1	
3	2	

page table

e.g. m=4, n=2

		f# :
0	0	
1	4	i j k l
2	8	m n o p
3	12	
4	16	
5	20	a b c d
6	24	e f g h
7	28	

physical memory

Fig.8.9 An paging example

Address mapping in Fig. 8.9

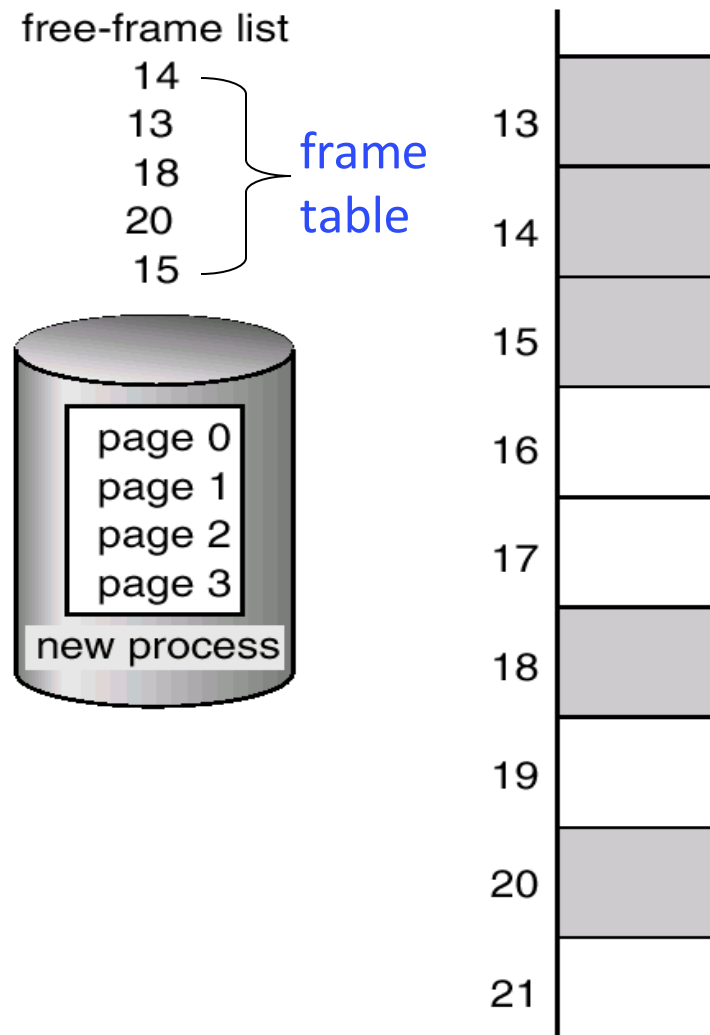
- Logical address consists of $m=4$ bits, i.e. **, **
 $2^4=16$ bytes
- Physical address consists of 5 bits, i.e. ***, **
 $2^5=32$ bytes
- E.g.1 logical address 0, i.e. 00,00
page number=00, page offset=00
page table: page 0 is allocated to frame 5, that is, 101
physical address : 10100, or: $5 * 4 + 0 = 20$
- E.g.2 logical address 9, i.e. 10,01
page number=2, i.e. 10, page offset=1, i.e. 01
page table: page 2 is allocated to frame 1, that is, 001
physical address : $1 * 4 + 1 = 5$, or: 00101

Fragment

- Paging do not result in external fragmentation, but may introduce **internal fragmentation**
- Calculating internal fragmentation
 - ▣ page size = 2,048 bytes
 - ▣ process size = 72,766 bytes
 - ▣ 35 pages + 1,086 bytes
 - ▣ internal fragmentation of $2,048 - 1,086 = 962$ bytes
 - ▣ worst case fragmentation: only 1 byte in the last frame
- On average fragmentation = $1 / 2$ frame size
- Page sizes growing over time
 - ▣ Solaris supports two page sizes – 8 KB and 4 M

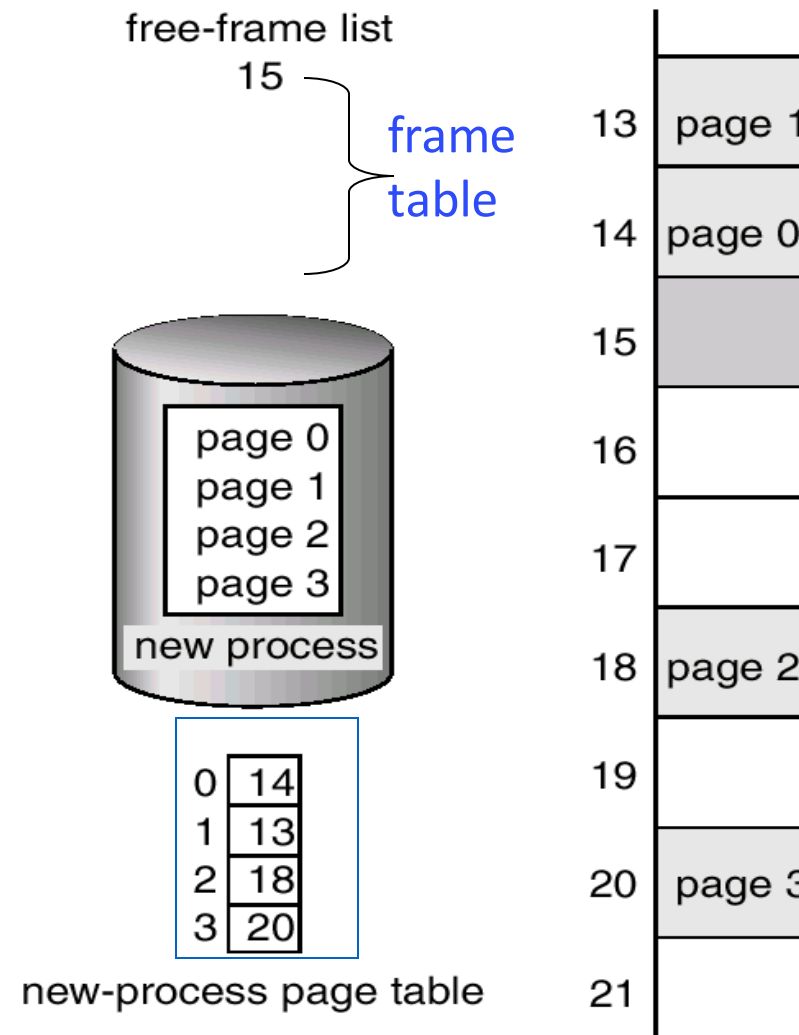
Frame Table

- OS keeps frame table to denote
 - ▣ how many total frames there are
 - ▣ which frames are available, and which are allocated.
- Each page is loaded into the allocated frames, and the frame numbers of these allocated frames are put into the page table for this process.
- e.g. Fig. 8.10



(a)

Before allocation



(b)

After allocation

Fig. 8.10 Free frames (a) before allocation and (b) after allocation

Implementation of Page Table

- The page table is used for address mapping, a process' every access to memory must go through the paging mapping
- The page table is accessed frequently, especially in dynamic relocation, so it must be capable of being addressed quickly enough
- In most systems, each process is allocated a page table; as a kernel data structure, the page table is kept in main memory
- Two registers are used for OS to access page table to do address binding
 - ▣ **Page-table base register (PTBR)** points to the page table //页表基址寄存器
 - the initial address of the page table is recorded in PTBR
 - ▣ **Page-table length register (PTLR)** indicates size of the page table //页表长度寄存器
- In this scheme every data/instruction access requires **two memory accesses**
 - ▣ one for accessing the page table to acquire the physical addresses for data/instruction, and one for access the data / instruction according to the physical addresses

Implementation of Page Table

- The two memory access problem can be solved by the use of a special fast-lookup hardware **cache** called **associative memory** (关联存储器) or **translation look-aside buffers** (转址旁路缓存, TLBs)

- **A**ssociative memory – parallel search
 - ▣ <key, value>, **parallel** search, faster access

Page #	Frame #

- As a type of cache for address binding, TLBs typically small (64 to 1,024 entries), and a part of page table entries (p#, f#) is stored in TLBs
- Address translation (p#, d), *refer to Fig.8.11*
 - ▣ if p# is in associative register, i.e. TLB hitting, get f # out
 - ▣ Otherwise, **TLB missing**, get f# from page table in memory

Paging Hardware With TLB

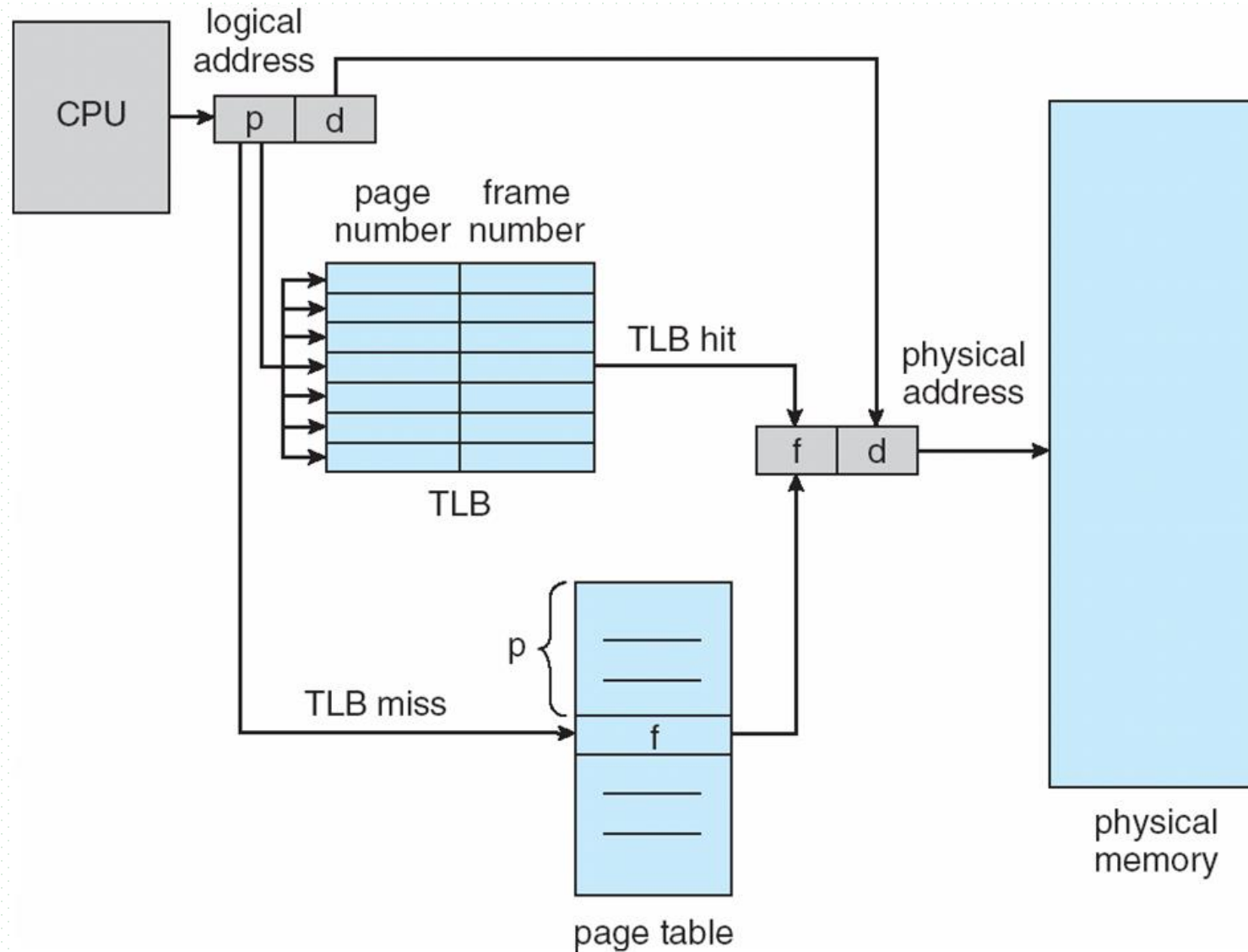


Fig. 8.11 Paging hardware with TLB

Paging Hardware With TLB

- At every context switch, when a page table is selected, TLB should be flushed (刷新) to accept the new page table entries in the new process's page table
//进程上下文切换时，如 P_1 切至 P_2 ，TLB中原有 P_1 的页表项失效，需要重新加载将要运行的进程 P_2 的page table，利用该进程的page table 刷新TLB
- If a TLB miss occurs, the page table entry to be access is loaded from memory into the TLB, for faster access next time
 - ▣ replacement policies must be considered
 - ▣ some entries can be **wired down** for permanent fast access
 - 例如，对OS内核重要进程，为提高地址转换速度，其页表项驻留在TLB中

EAT in TLB-based paging

- **Effective Access Time (EAT)** = time for address translation + time for memory access
 - ▣ time for address translation
 - when TLB hitting, time for address binding in TLB
 - when TLB missing, time for address searching in TLB + time for accessing page table in memory
- Associative Lookup (TLB access) = ε time unit
 - ▣ Can be < 10% of memory access time
- Hit ratio α
 - ▣ percentage of times that a page number is found in the associative registers/TLB; ratio related to number of associative registers

EAT Example

- It is supposed
 - ▣ the hit ratio is 80%
 - ▣ it takes 20 ns to search the TLB and 100ns to access memorythen
- A mapped-memory access takes 120 ns when the page number is in TLB
 - ▣ because it takes 20ns to search TLB and 100 ns to access memory
- If the page number is not in TLB (20 ns), then we must first access the memory for the page table and frame number (100ns), then access the desired page in memory (100ns), for a total 220 ns
- By weighting each case by its probability, we obtain the EAT as
$$\begin{aligned} \text{EAT} &= 0.80 \times (20+100) + 0.20 \times (20+ 100 + 100) \\ &= 140 \text{ ns} \end{aligned}$$

Memory Protection

- To identify **legal memory access**, **valid-invalid** bit attached to each entry in the page table:
 - ▣ "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
 - ▣ "invalid" indicates that the page is not in the process' logical address space
 - ▣ Or use **page-table length register (PTLR)**
- To control **access privileges**, memory protection implemented by associating protection bit with each frame to indicate if **read-only** or **read-write** access is allowed
 - ▣ can also add more bits to indicate page **execute-only**, and so on
- Any violations result in a trap to the kernel

- A system with 14-bit address space, $0-16383=2^3 \times 2^{11}=8 \times 2K$
- page size: 2KB
- process maximal size: 8 pages
- a process's logical address space is only from 0 to 12287, i.e. 6 pages/frames
- process access address in pages 0, 1, 2, 3, 4, 5 are within process's logical space, and are mapped normally
- accesses beyond 12287 are mapped into page 6,7, and are invalid

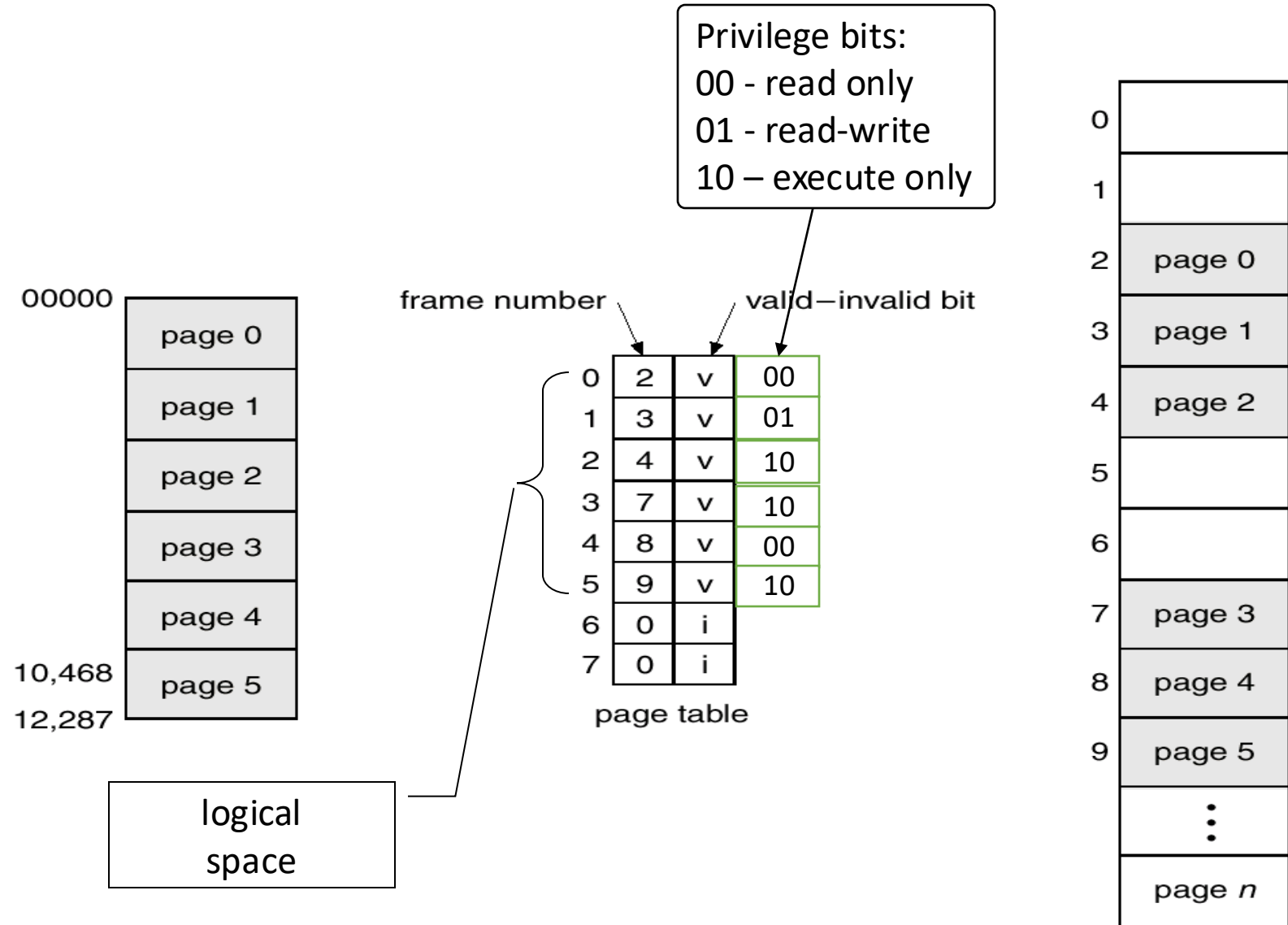
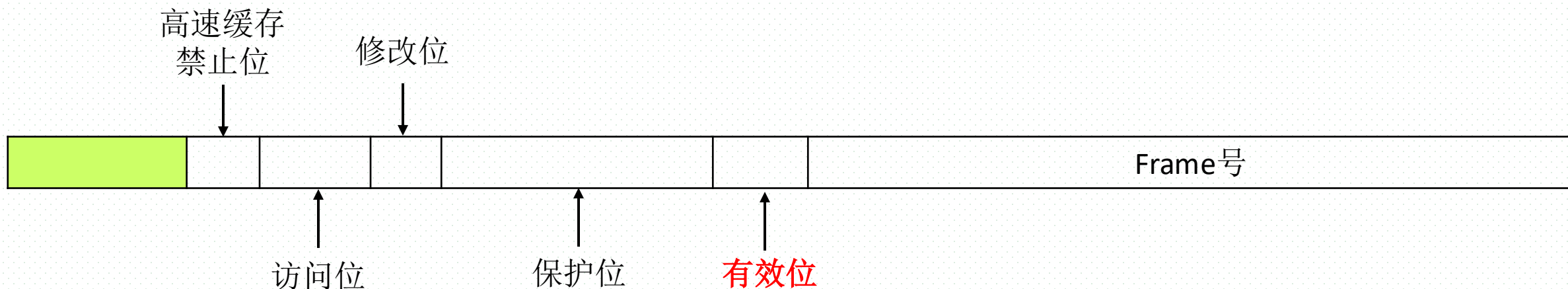


Fig.8.12 Valid (v)/invalid (i) bits and privilege bits in a page table

Linux页表项内容



- **有效位**
 - ▣ 1, 有效, 代表该表项对应的page在内存中; 0, 无效, 代表对应的page不在内存中
- 保护位: 表示页允许的访问类型, 可读、可写、可执行
- **修改位**: 0, 没有被修改过的干净页; 1代表被修改过的脏页, 页置换时, 脏页需要写回磁盘
- 访问位: 最近该页被访问, 置为1, 否则为0, 用于可支持近似LRU等页置换算法
- 高速缓存禁止位: 是否允许该页被cache缓存

LEGEND
P = Present (1=yes, 0=no)
W = Writable (1 = yes, 0 = no)
U = User (1 = yes, 0 = no)
A = Accessed (1 = yes, 0 = no)
D = Dirty (1 = yes, 0 = no)

PWT = Page Write-Through (1=yes, 0 = no)
PCD = Page Cache-Disable (1 = yes, 0 = no)

知乎@极致的内核

PDE 与 PTE 的结构非常相似：

- P 位 — 存在位，表示当前条目是否在物理内存中
- R/W 位 — 读写权限位，为 0 表示只读， 表示可读写
- U/S 位 — 页或一组页的特权级^Q，为 0 表示系统级，对应 CPL 0、1、2，为 1 表示用户级，对应 CPL 3，下文进行详细介绍
- PWT — 页表缓冲写入机制，为 0 表示 write-back 模式，更新页表缓冲区时，只标记为已更新，不同步写内存，只有被新进入的数据取代时才更新到内存，为 1 表示 write-through 模式，更新页表缓冲区时，同步写内存，保证缓冲区与内存一致
- PCD — 是否拒绝被缓冲，为 0 表示可以被缓冲，为 1 表示不可以被缓冲
- A 位 — 是否被访问，CPU 会在访问到页面时将该位置 1，但不会清除，只有软件可以将 A 位复位

Shared Pages

- Process = Private code and data + Shared code
- **Shared code**
 - One copy of read-only code shared among processes (i.e., text editors, compilers, window systems)
 - Similar to multiple threads sharing the same process space
 - Also useful for interprocess communication if sharing of read-write pages is allowed
- **Private code and data**
 - Each process keeps a separate copy of the code and data
 - The pages for the private code and data can appear anywhere in the logical address space
- **Shared code/data** must appear in **the same location in the logical address space of all processes**
 - i.e. with **the same page numbers** in different processes

Example: Shared Pages

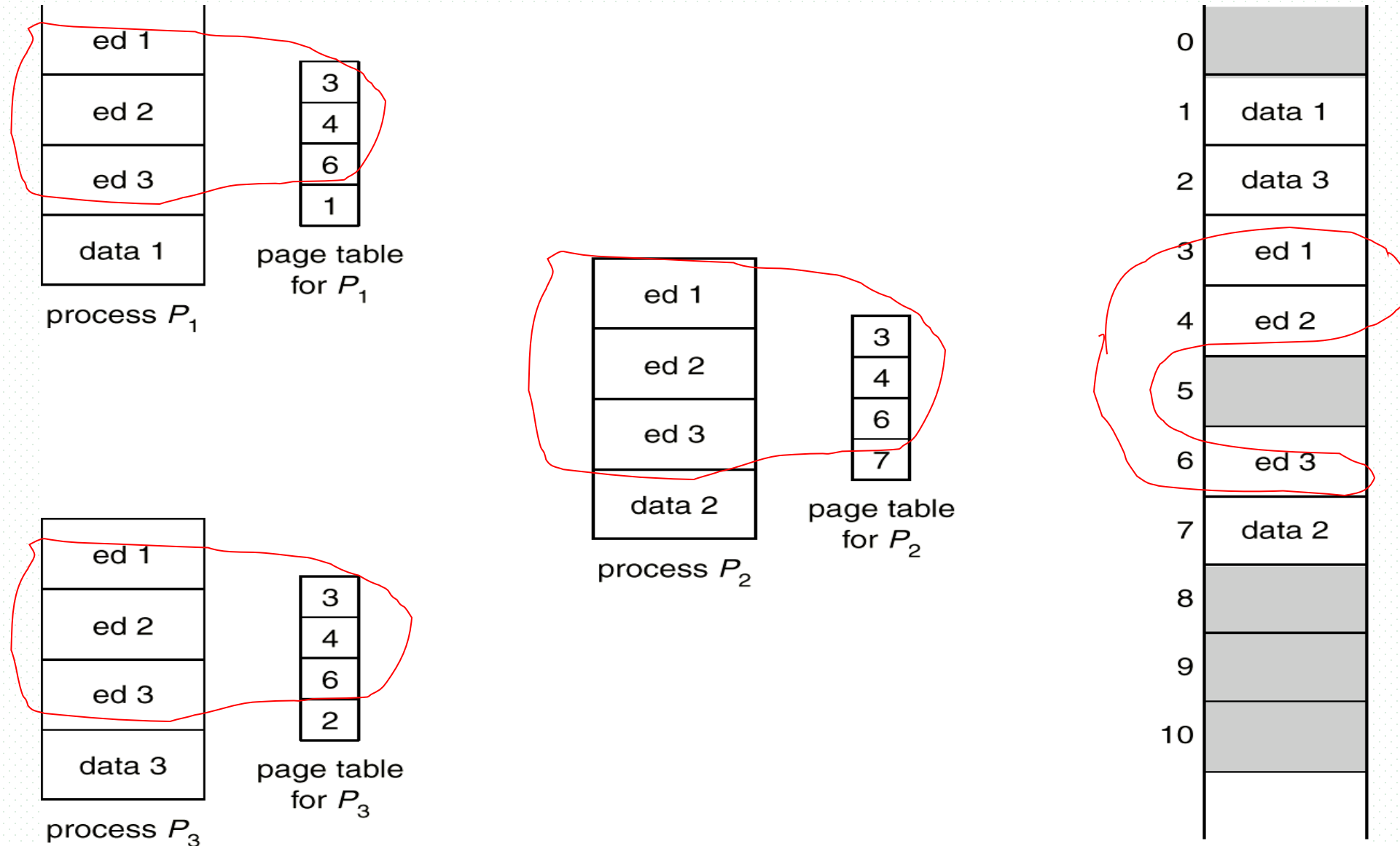


Fig. 8.13 Shared pages

Segmentation vs Paging

- In general, address binding in paging is faster than that in segmenting
- The size of the page table is larger than that of the segmentation table
- In segmentation, the offset must be **added** to the **starting segment address** 
 - ▣ each segment varies in size
- In paging, the page frame number and the offset are concatenated to form the physical address 
 - ▣ concatenating two bit patterns is faster than adding them

8.6 Structure of the Page Table

- Page table: page# $\rightarrow$ frame#, or initial address of frames
- Multi-level/Hierarchical Paging
 - ▣ 解决（单进程）大页表非连续存储，节省页表存储空间
- Hashed Page Tables
 - ▣ 解决基于页表的快速地址变换
- Inverted Page Tables/倒置、逆向页表
 - ▣ 解决（多进程）页表空间占用过大
 - #frame $\rightarrow$ #process, #page
 - 找出共享同一个frame的多个process的pages
 - ▣ e.g. (反向映射) RMAP in Linux, used for
 - 父进程fork()子进程，父子进程共享父进程物理地址空间vma
 - 1个物理内存中的匿名线性映射区被进程使用

Hierarchical Paging

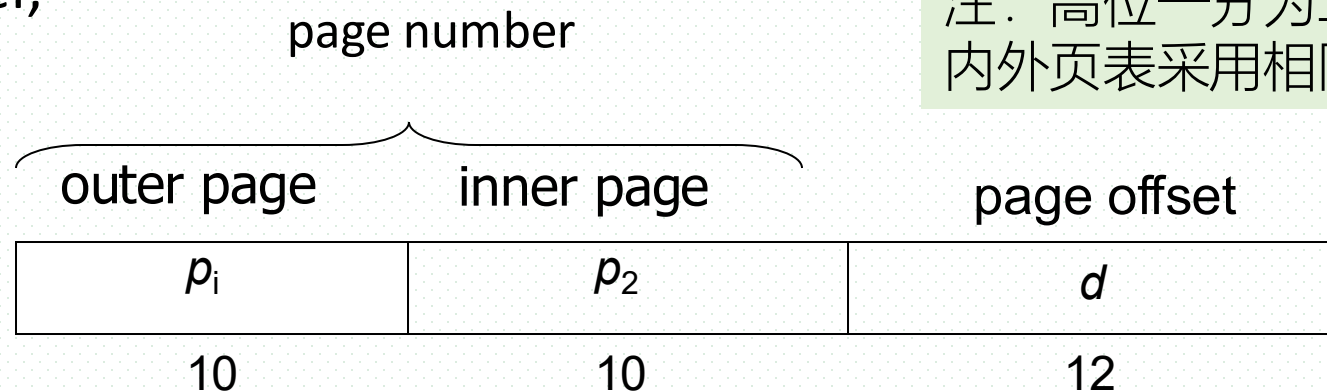
- Page table must be loaded in memory for address binding
- If a process's logical space is very large, its page table may occupy large memory space
- However it is not easy for OS (e.g. Buddy algorithm in Chapter 9) to find a section of **contiguous** main memory of large-size to hold the page table, how to guarantee searching the page table be efficient?
- E.g. In a 32-bit system, with process logical address space 4G
 - ▣ page size of 4 KB= 2^{12} bytes
 - ▣ page table would have $(2^{32} / 2^{12})=1$ million entries
 - ▣ if each entry is 4 bytes, the size of a process's page table is $2^{20} \times 4$ bytes=4MB, and 100 processes' page tables occupy 400M
- To **avoid a single larger contiguous page table**, divide the page table into several pieces, and the logical address space is broken up into multiple page tables

阿里圣经：MySQL
关系表不超过
2000万行， why ?

Hierarchical Paging

- Implementation
 - ▣ a two-level page table, two-level paging
- E.g. A logical address (on 32-bit machine with 4K page size) is divided into
 - ▣ a page number consisting of 20 bits
 - ▣ a page offset consisting of 12 bits
 - ▣ the page table itself is also paged, the page number is further divided into:
 - a 10-bit page number,
 - a 10-bit page offset.

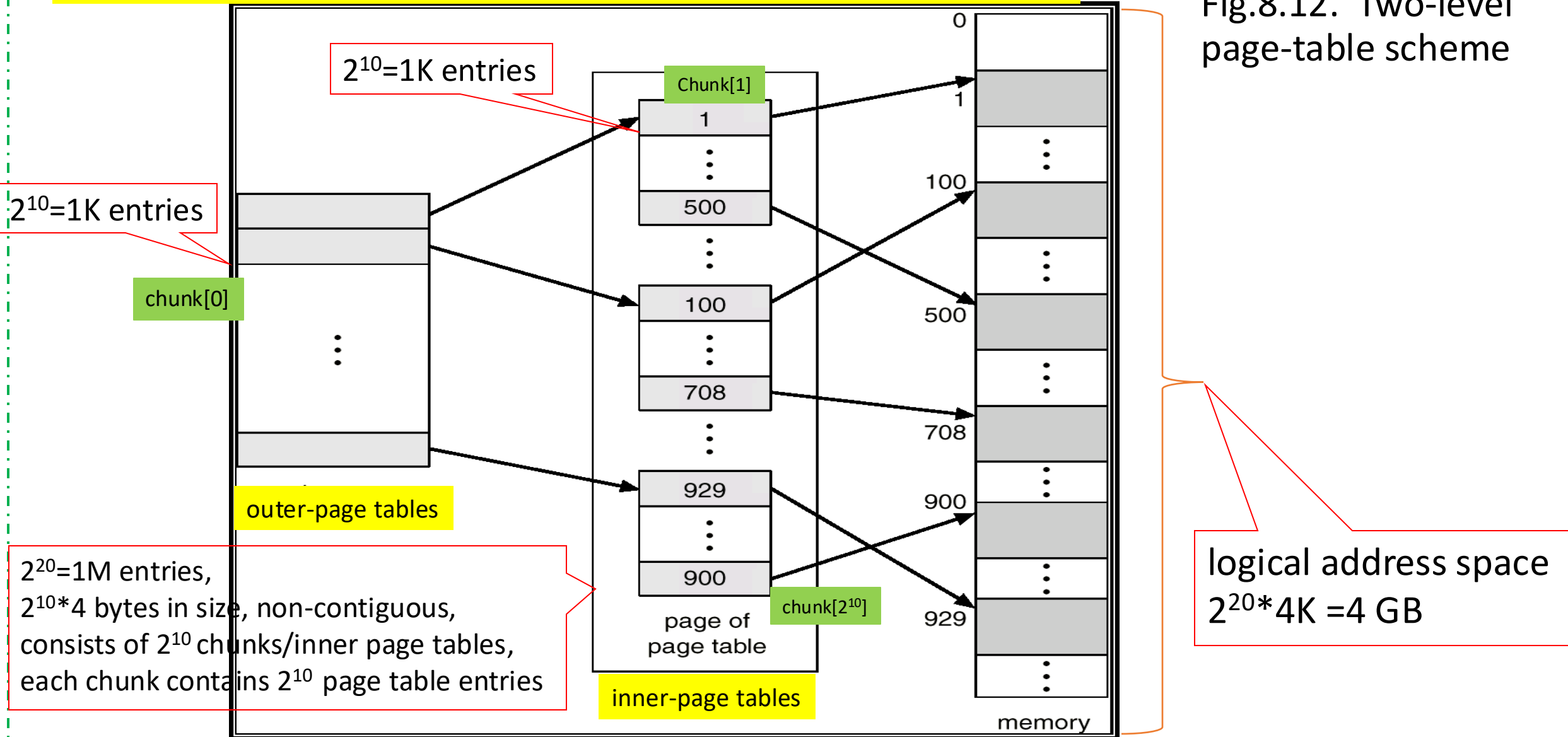
注：高位一分为二，各占10bits，
内外页表采用相同大小的chunk



the number of entries in the table: $2^{20}=1\text{M}=2^{10}*2^{10}=1\text{K}*1\text{K}$
1M= 2^{20} entries in the page table is divided into 2^{10} chunks, and each chunk contains 2^{10} entries and located in a contiguous space

These page tables occupy $(1+2^{10})$ chunks, $(1+2^{10}) * 2^{10} * 4 = (4K + 4M)$ Bytes

Fig.8.12. Two-level page-table scheme



注：内外页表采用相同大小的chunk

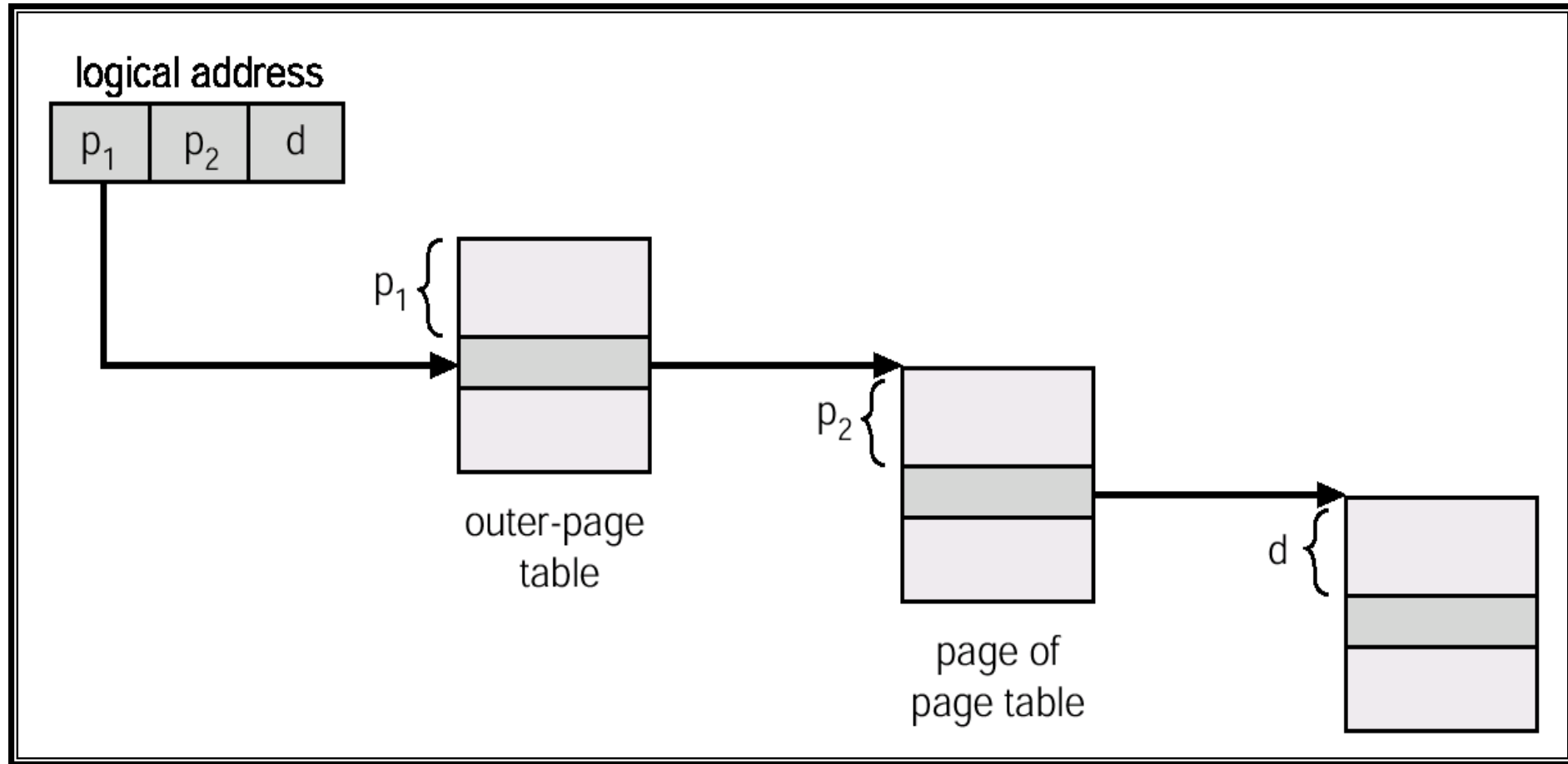
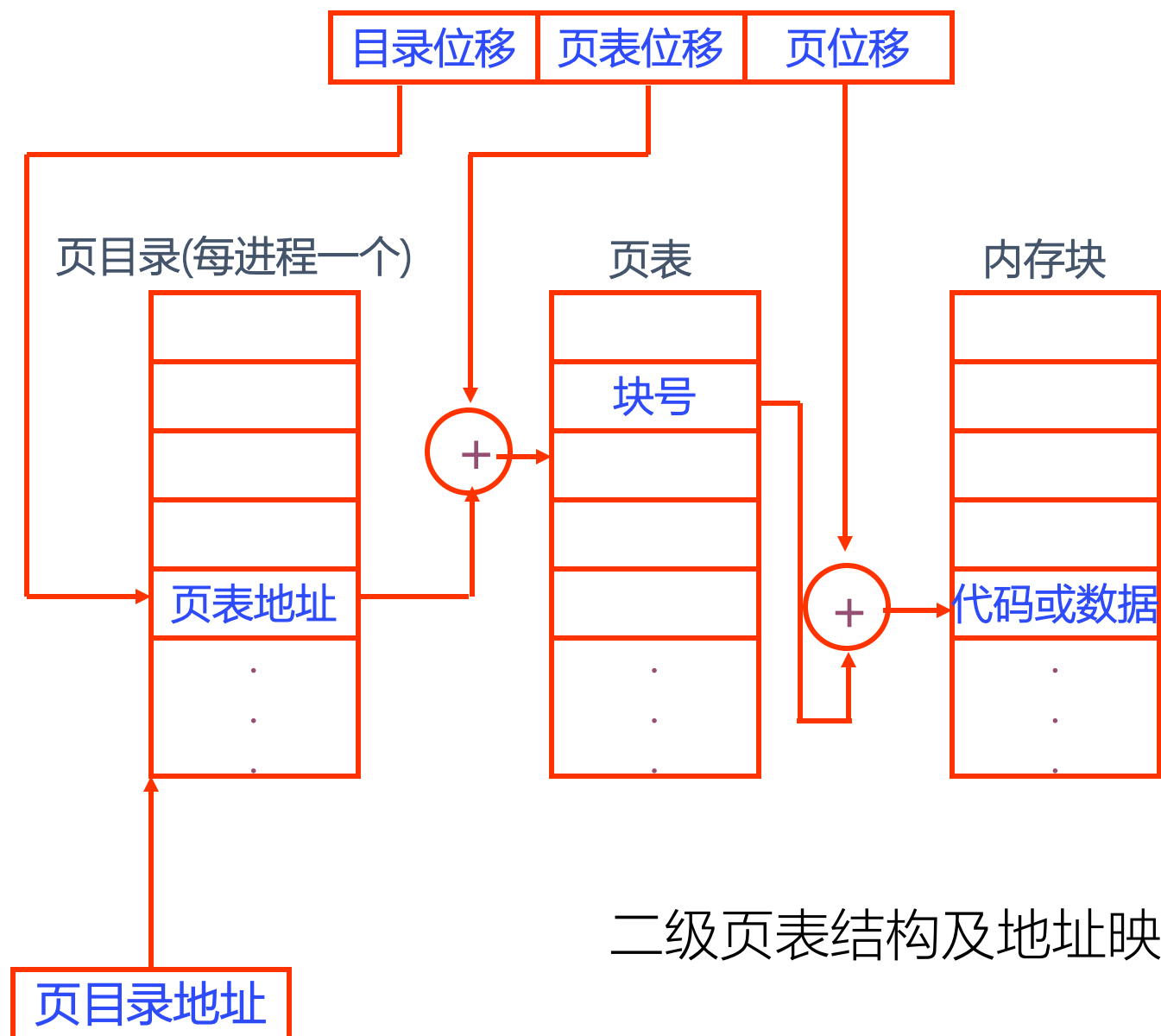


Fig.8.13 Address-translation scheme for a two-level 32-bit paging architecture

虚拟地址

位移: offset



例题 408选择题

- (2021-408) 29. 在采用二级页表的分页系统中，CPU 页表基址寄存器中的内容是
 - A. 当前进程的一级页表的起始虚拟地址
 - B. 当前进程的一级页表的起始物理地址
 - C. 当前进程的二级页表的起始虚拟地址
 - D. 当前进程的二级页表的起始物理地址

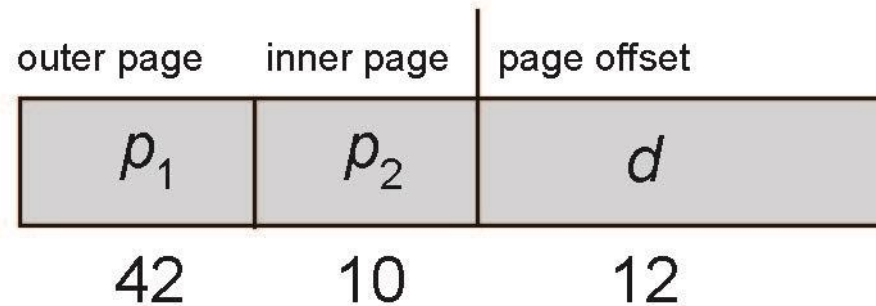
■ 答案： B

- (2019-408)28. 在分段存储管理系统中，用共享段表描述所有被共享的段。若进程 P_1 和 P_2 共享段 S ，下列叙述中，**错误**的是【说明：此题同样适用于页式管理中的页共享】
 - A. 在物理内存中仅保存一份段 S 的内容
 - B. 段 S 在 P_1 和 P_2 中应该具有相同的段号
 - C. P_1 和 P_2 共享段 S 在共享段表中的段表项
 - D. P_1 和 P_2 都不再使用段 S 时才回收段 S 所占的内存空间

答案： B

Multiple-level Paging in 64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - ▣ then page table has $2^{(64-12)} = 2^{52}$ entries
 - ▣ if two level scheme, each inner page table contains 2^{10} 4-byte entries
 - ▣ address would look like

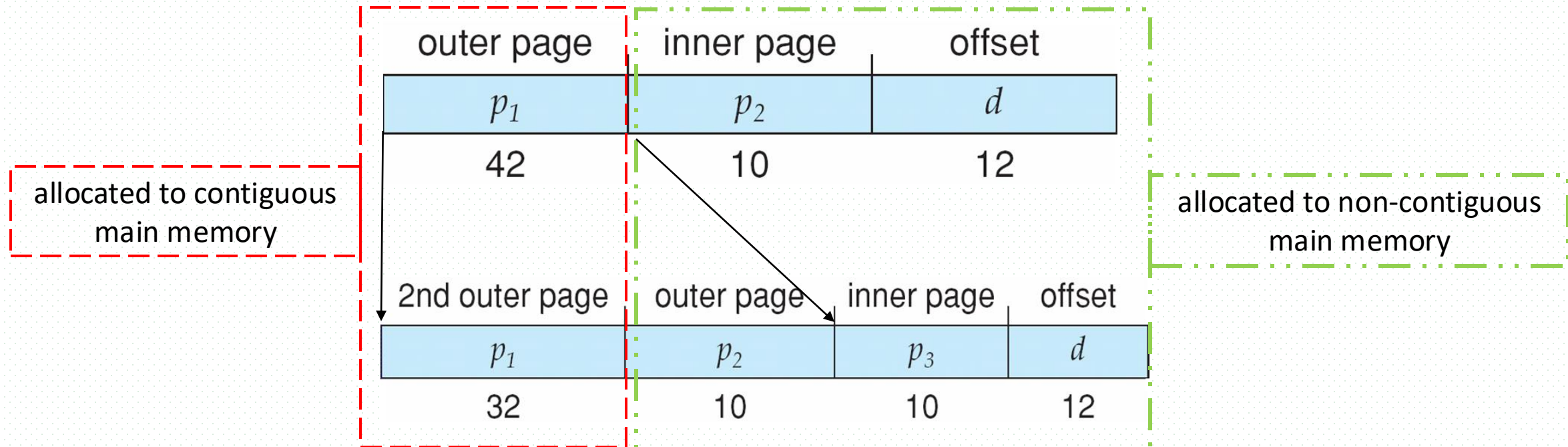


- ▣ the outer page table has $2^{(52-10)} = 2^{42}$ 4-byte entries, occupying 2^{44} bytes = 2^{24} M = 2^{14} G contiguous memory space, too large in size

Three-level Paging in 64-bit Address Space

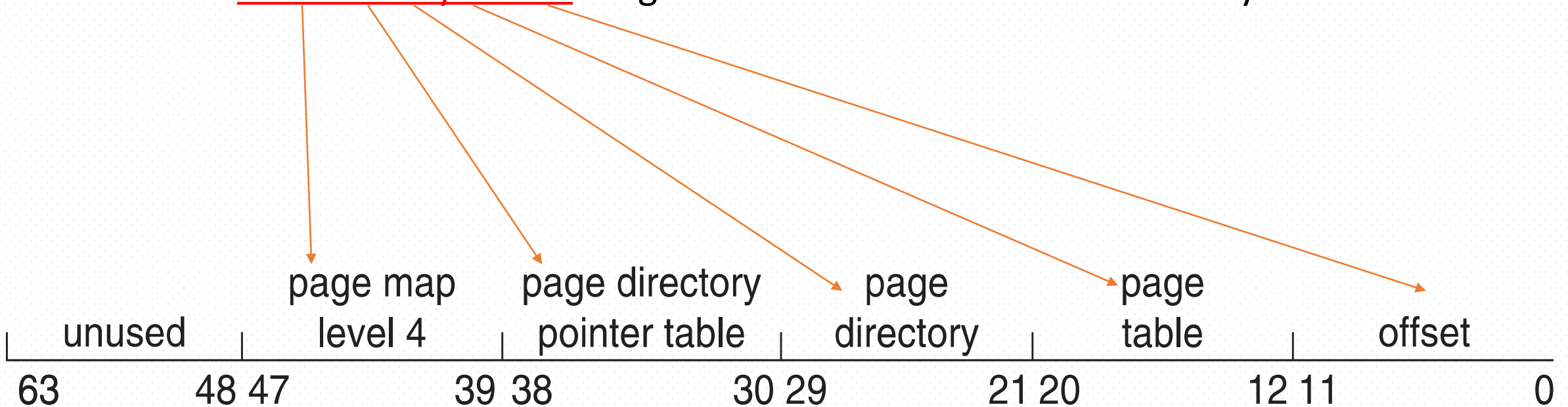
- One solution is to add a 2nd outer page table
 - ▣ but in the following example the 2nd outer page table is still $2^{32} \times 4 \text{ bytes} = 2^{34} \text{ bytes} = 2^4 \text{ G}$ in size
 - and possibly 4 memory access (i.e. p_1, p_2, p_3, d) to get to one physical memory location

// 四次内存访问，访问数据或指令



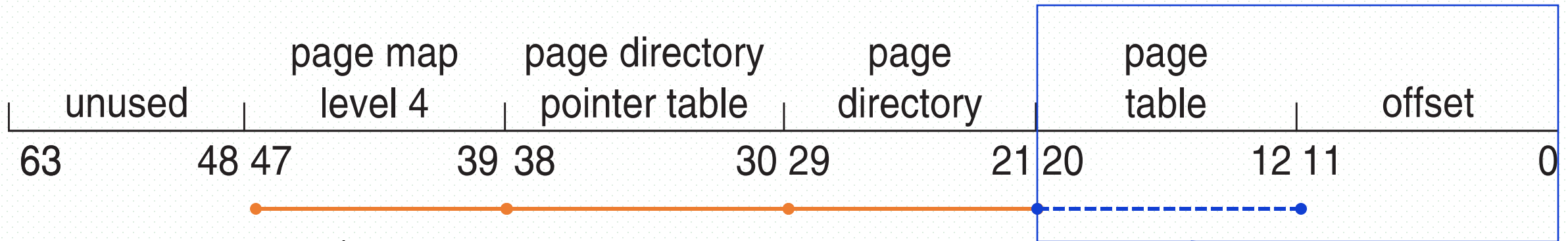
Four-level Paging Scheme

- Approaches to avoid a outer page table with too large size,
 - ▣ four-level paging scheme
 - 48 bit地址线, 48位虚拟地址, $9 + 9 + 9 + 9 + 12$
 - with five memory access to get to instruction or data in memory



Four-level Paging Scheme

- Only 48 bits in address are used as instruction or data addresses
- The size of process address space in 64-bit Linux is 2^{48} bytes=256T, divided into 128T kernel space and 128T user space
- 48 bit地址线, 48位虚拟地址, $9 + 9 + 9 + 9 + 12$



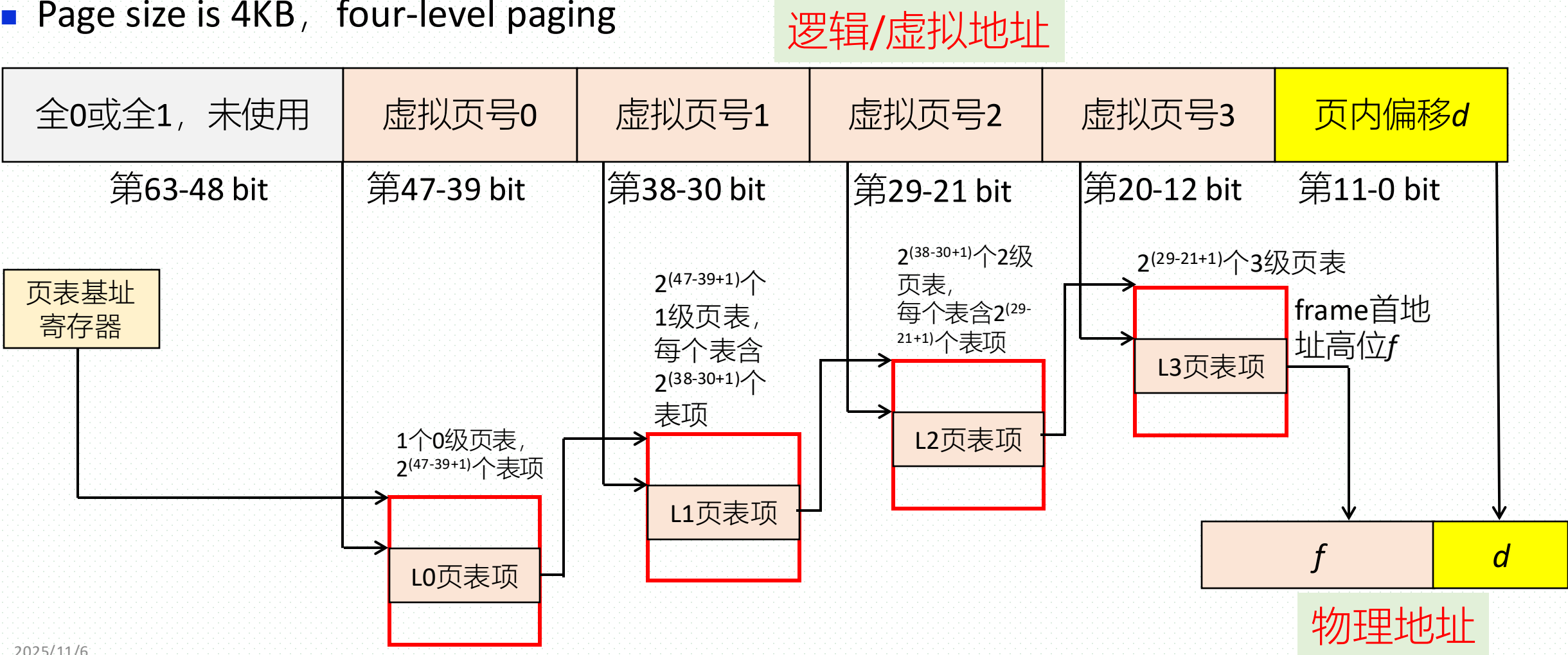
- Large page size-大页, e.g. in Linux, page size can be
 - ▣ four-level, page size=4K, $9 + 9 + 9 + 9 + 12$
 - ▣ three-level, page size=2M, $9 + 9 + 9 + 21$
 - ▣ two-level, page size=1G, $9 + 9 + 30$

Muti-level page tables in main memory

- 相比于单级页表
 - 多级页表解决了大页表无法在内存中连续存储的难点
 - 但多级页表需要的存储空间更大
 - e.g. in the 32-bit two-level page table scheme, these page tables occupy $(1+2^{10})$ chunks = $(1+2^{10}) * 2^{10} * 4 = (4K + 4M)$ bytes, 
 - while one-level page table occupies $2^{20} * 4$ bytes = 4M bytes
- Linux等实际操作系统解决方案
 - 全部多级页表存储在外存
 - 采用虚拟内存VM管理技术，只将一部分页表调入内存，包括
 - 最高层的外页表outer page全部加载进内存
 - 近期需要访问的那部分地址空间对应的各级内页表的表项
 - e.g. 64-bit ARM, 0级页表全部，部分的1、2、3级页表

64-bit ARM Paging Architecture

- 64-bit ARM Paging Architecture
 - ▣ 64-bit虚拟地址，低48=9+9+9+9+12 bits参与paging-based address translation
- Page size is 4KB, four-level paging



- Page table: page number $\rightarrow$ frame number
- 以hash table 替换 page table
 - ▣ hash table, kernel data structure, can not be replaced by hash functions due to hash collision
- Commonly used when logical address spaces larger than 32 bits
 - ▣ the number of entries in a hash table may be lower than that of the pages in logical space, and thus, two pages are mapped to the same one entry in the hash table
 - ▣ e.g. 进程的page数目6, hash table entry数目4, in Fig. 8.12.1
- Method
 - ▣ the (virtual) page number is used as hash value, and hashed into a page table. This page table contains a chain of elements hashing to the same location, i.e. the same frame
 - ▣ virtual page numbers are compared in this chain searching for a match. If a match is found, the corresponding physical frame is extracted

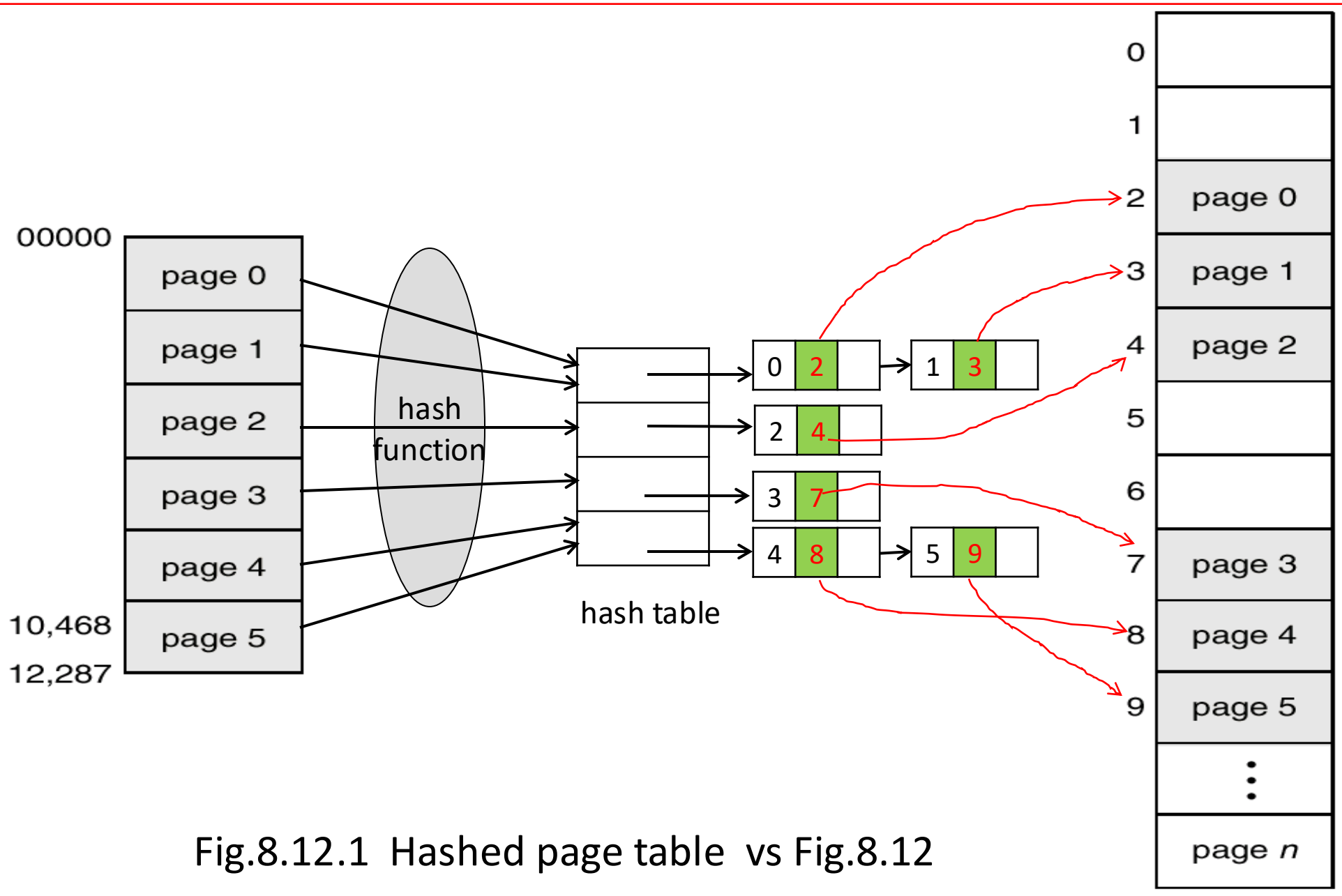


Fig.8.12.1 Hashed page table vs Fig.8.12

Hashed Page Tables

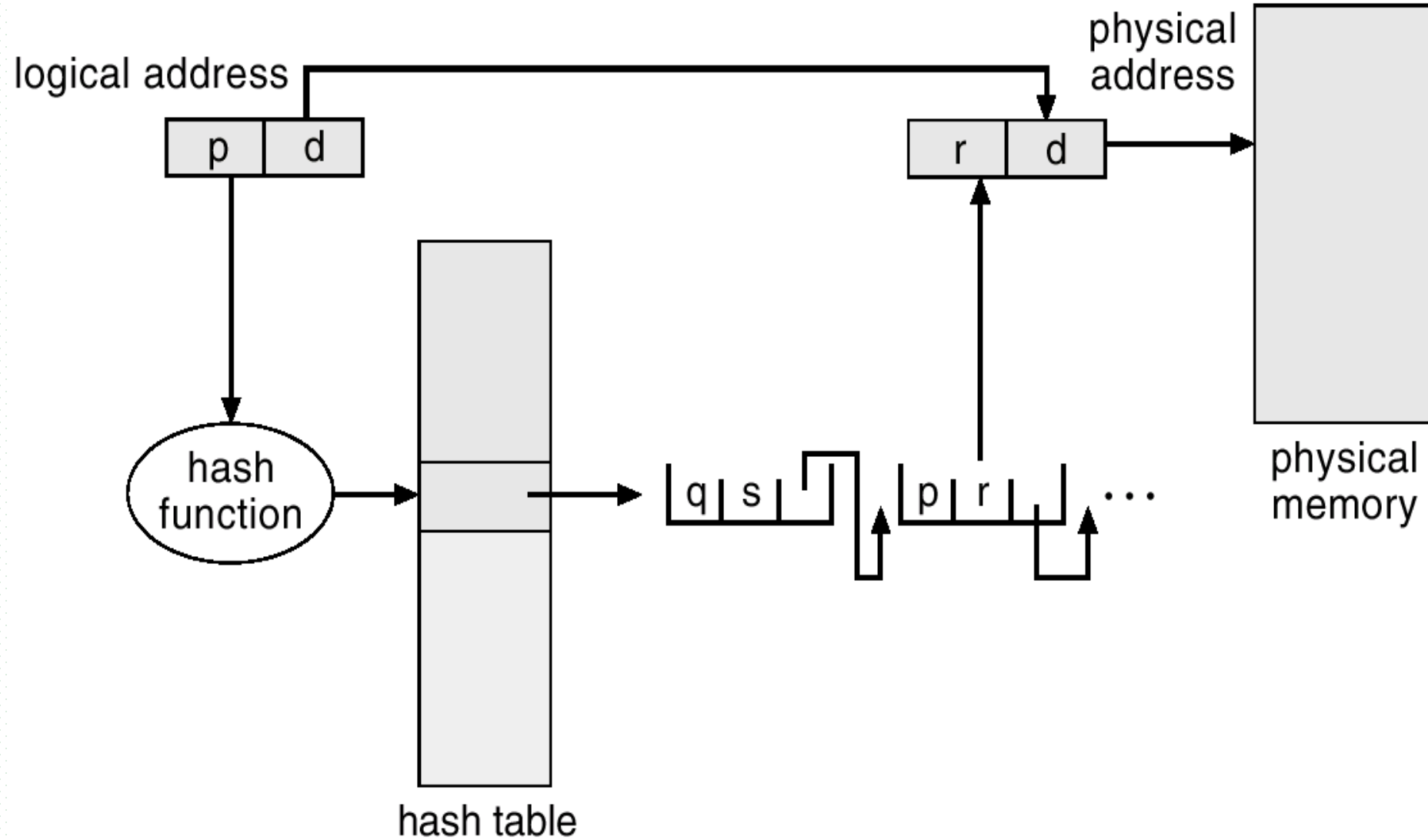


Fig.8.14 Hashed page table

- Why: the demerit for page tables
 - ❑ one page table for each process in system, and in the page table, one entry for each page in the logical address space of the process
 - ❑ two different tables may contain the same physical address, e.g. two processes share one frame, thus the number of this frame occurs in the page tables of these two processes
 - e.g. C++ library function `fork()` shared by two process
 - ❑ so all the tables for the processes in the system may consume large amounts of physical memory.
- Inverted page table, **only one table**
 - ❑ one entry in the table for each real frame of memory, i.e. each frame corresponds to one table entry, mapping to a page in a process!!

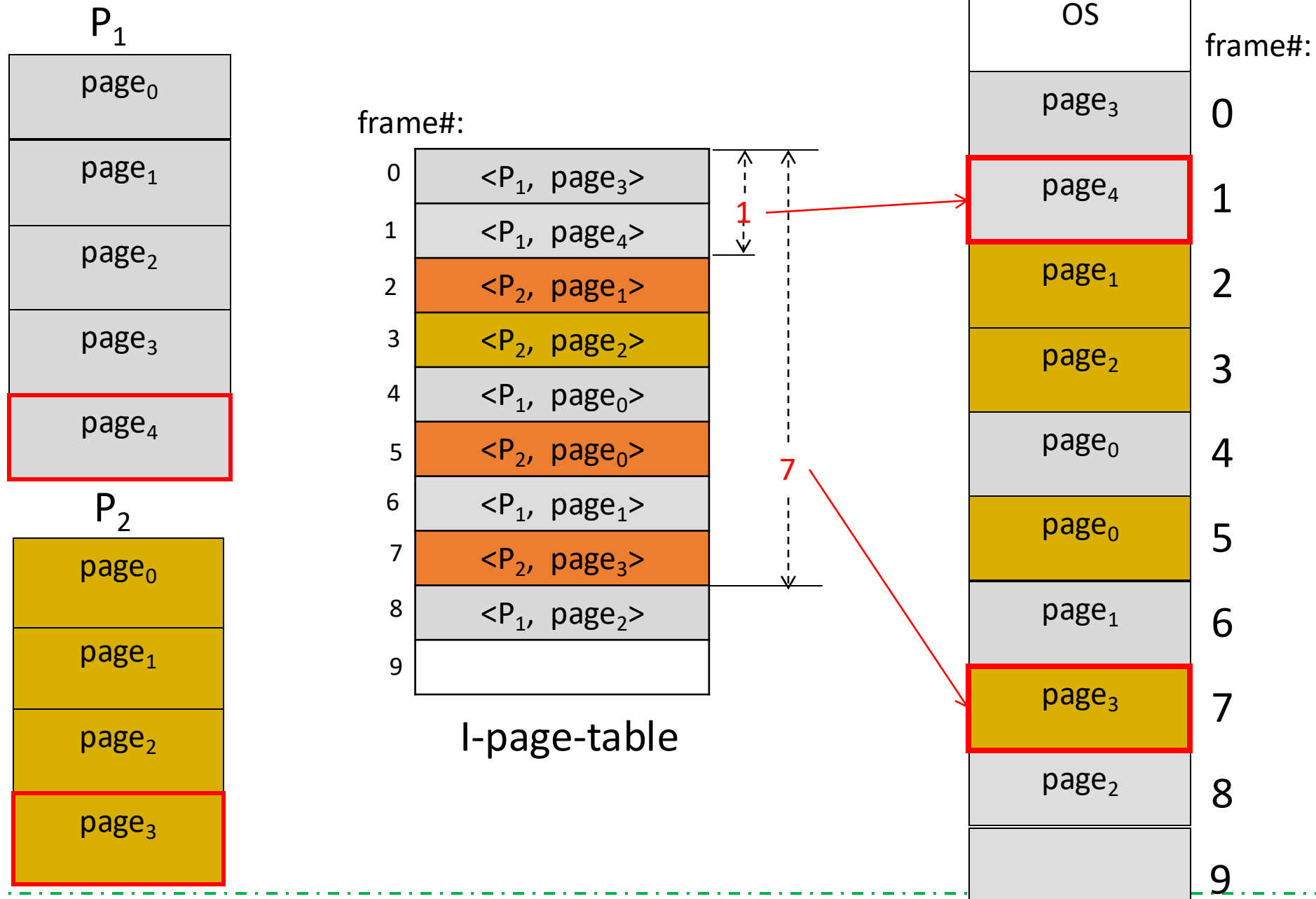
Inverted Page Table

- Table structure:
 - ▣ the table has N entries, and N is the maximum number of frame in main memory, so the table size is N
 - e.g. 16G main memory, $2^{12}=4K$ page/frame, $16G/4K=(2^{24}/2^{12})=2^{12}$ entries in inverted page table
 - ▣ the entry in inverted page table indicates
 - the process that owns that page
 - the virtual address of the page, i.e., **the page number**, stored in each frame (i.e., real memory location)
- Logical/virtual address: **<process id, page-number, offset>**
Invert table: **<process id, page-number>** $\leftrightarrow$ frame#

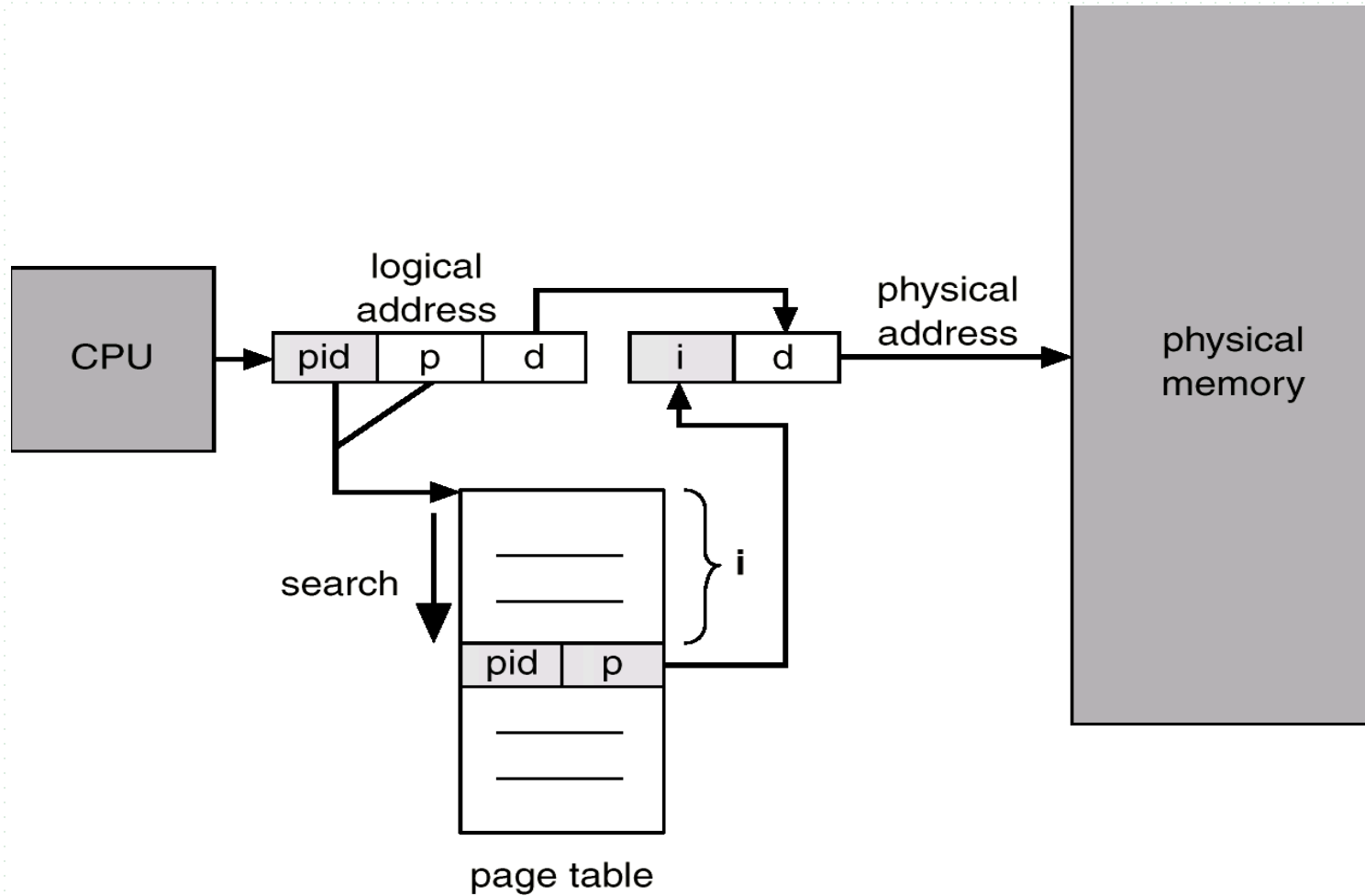
9 pages

10 entries

10 frames



Inverted Page Table



(in associate memory for quick search?)

Fig.8.15 Inverted page table architecture

Page Table vs Inverted Page Table

■ Page table

- 描述1个进程的各个page在物理内存中的frame
page# → frame#
- 每个进程1个page table, 系统内有多个page table
- page table长度/page table 表项数目
= 进程的逻辑地址空间中page数目

■ Inverted page table

- 描述物理内存中的各个frame存放的是哪个进程的哪个page
frame# → P#, page#
- 系统内只有1个inverted page table
- 长度: 物理内存中的frame数目

8.7 The Intel 32 and 64-bit Architectures

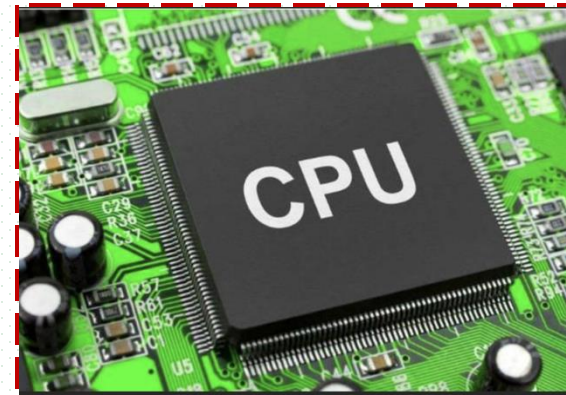
- Segmentation in IA-32
- Paging in IA-32
- Paging in x86-64

CPU架构 (CPU Architecture)

- IA-32, 32-bit CPU
 - e.g. intel Pentium CPUs
- x-86 64,
 - e.g. intel/AMD 64-bit CPU
- ARM 16/32/64
 - ARM, Advanced RISC Machine
 - ARM 16/32-bit CPU, for mobile devices
 - , e.g. Apple CPU, 华为麒麟CPU
 - ARM 64-bit CPU
 - , e.g. 飞腾CPU, 华为鲲鹏服务器
- MIPS
 - e.g. 龙芯CPU
- Alpha
 - e.g. 申威CPU,
 - 神威太湖之光超级计算机
 - 神威海洋之光超级计算机
- PowerPC, IBM
- RISC-V, 开源CPU架构

国产多核/众核CPU

- 申威26010众核处理器， SW26010
 - 基于Alpha架构， 260个运算核心； 频率1.45GHz， 8G DDR3， 25平方厘米芯片
 - 每秒3万+亿次计算能力， 3.06TFLOPS
 - 申威网络交换芯片
 - 神威太湖之光高性能计算机， 装有40,960个芯片， $260 \times 40960 = 10649600$ 个核心
 - 神威E级超算原型机
- 飞腾64核CPU芯片， FT-2000/64 (2016-08)
 - 64核通用处理器， 基于ARM架构
 - 中国天津飞腾信息技术有限公司设计
 - 核心频率 2.0 G， 浮点运算峰值速度每秒 5120 亿次， 典型应用情形下的实测功耗 100 瓦
 - 长宽各为 55 毫米的封装内， 借28 纳米集成电路工艺， 集成 48 亿个晶体管
 - 用于天河超级计算机， 作为管理结点， 4096个？



PHYTIUM 飞腾



高性能服务器 CPU
FT-2000+/64

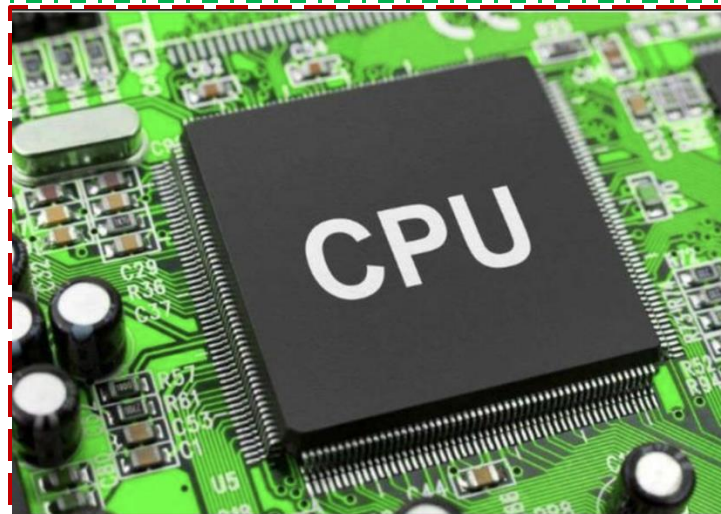
技术参数

- 处理器核：FTC662
- 64 核
- 16nm 工艺
- 2.0~2.3GHz
- 32MB 二级缓存
- 8 个 DDR4-2400 存储接口
- 1 个 x1、2 个 x16 PCIe3.0 接口，每个 x16 可分拆为 2 个 x8
- 功耗 96W
- 峰值计算性能 588.8 GFlops

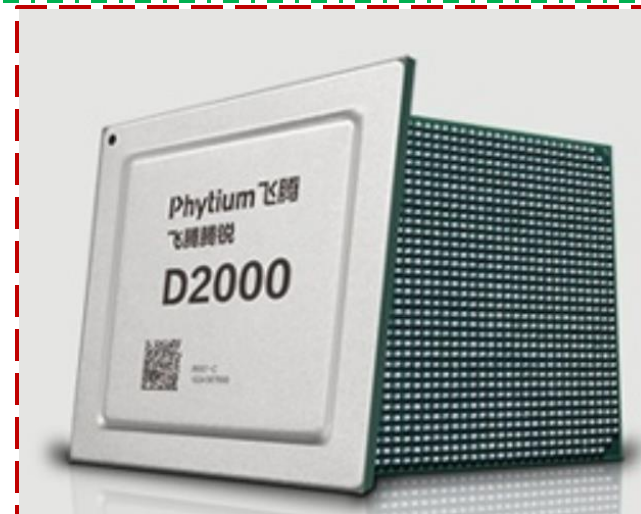
飞腾PHYTIUM



龙芯：
MIPS架构，loongarch指令集



申威：
Alpha架构，sw64指令集



飞腾：
Arm架构/指令集



中科曙光~海光：
x86架构，AMD Zen1微架构



魔都~兆芯：
x86架构/ip内核/指令集



华为~鲲鹏：
ARM架构/指令集



阿里RV~玄铁：
基于开源RISC-V

The Intel 32 and 64-bit Architectures

- Intel公司在早期32位微处理器8086、8088引入segmentation分段机制，并在80286处理器上使用分段机制支持虚拟内存
- 其后，发布IA-32架构的32位CPU芯片，如32-bit 奔腾处理器家族(Pentium CPUs)
 - supports both segmentation and segmentation with paging
- 之后，采用x86-64位架构的64位处理器（Intel、AMD的CPU产品）， paging分页机制的内存成为主流，但出于前向兼容的考虑，硬件上仍支持分段segmentation机制

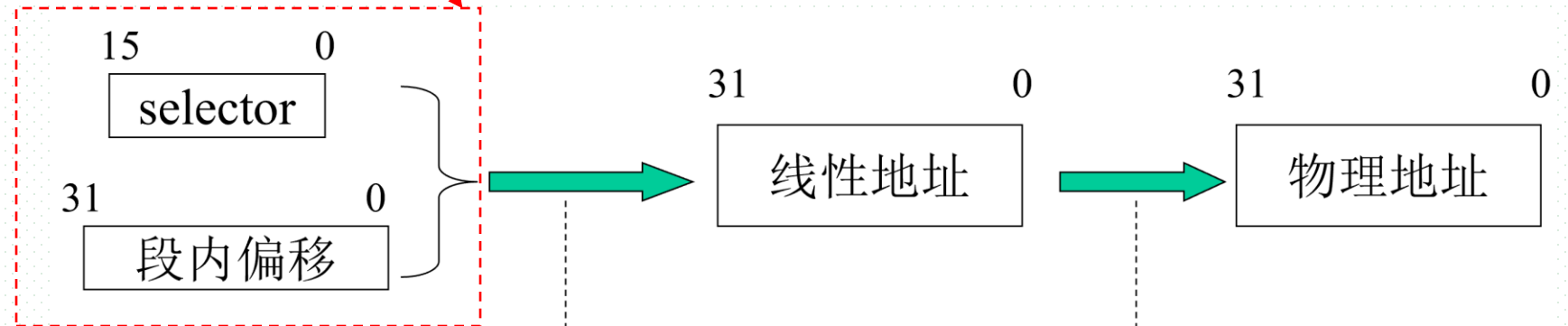
Segmentation with Paging in IA-32

- A process contains up to 16K segments, and each segment can be 4 GB
- Address translation in IA-32: logical address → linear address → physical address

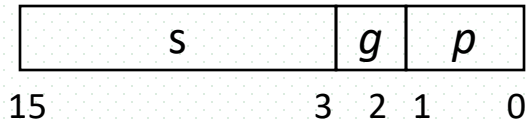
- Logical address

□ <segment number, offset in segment>

=<selector, offset>



16-bit selector



s: 段号, 14 bits, $2^{14}=16K$ 个段

g: 段在LDT、GDT

p: 保护位

逻辑地址空间

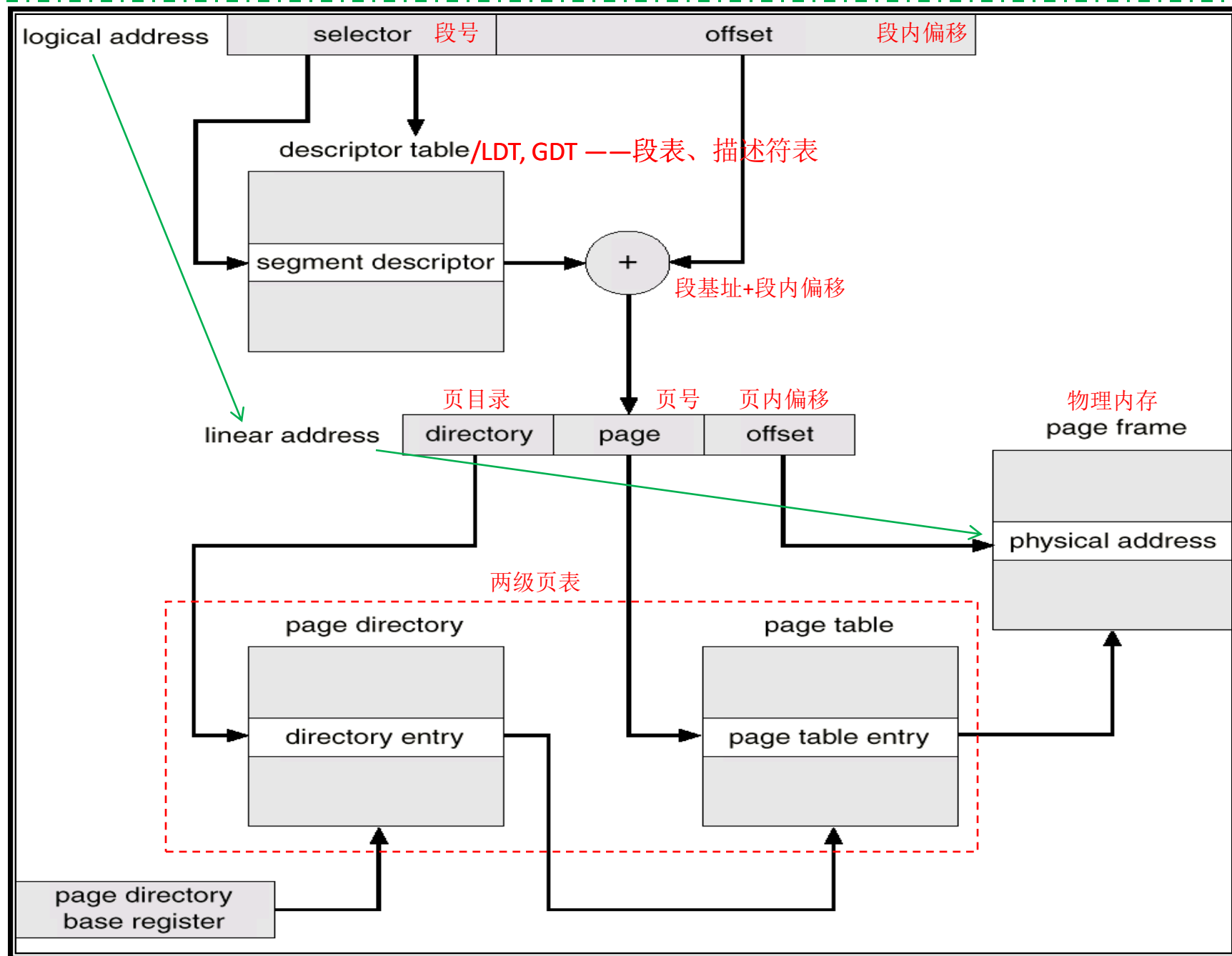
segmentation

线性地址空间
虚拟内存空间

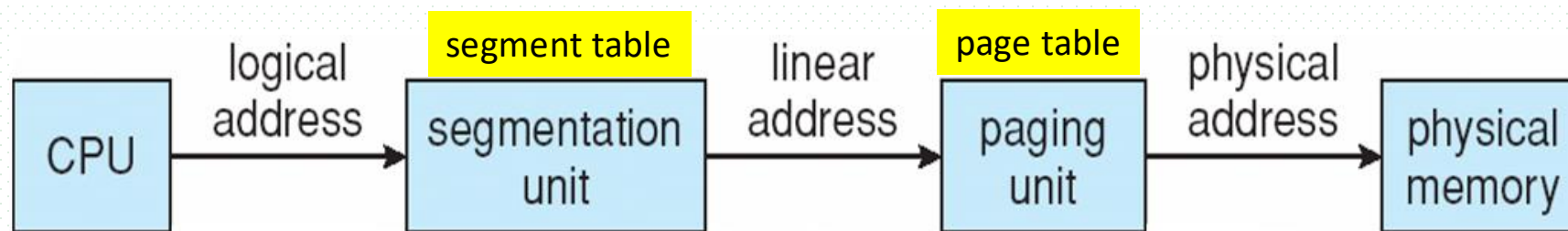
paging

物理地址空间

■ Address Translation



Segmentation with Paging in IA-32 【略】



- The address space of process is divided into two partitions, and kept in two segment tables, i.e. **LDT** and **GDT**
 - ▣ first partition of up to **8 K** segments are private to process, the information about which is kept in **local descriptor table (LDT)**(局部描述符表)
 - 进程私有的code、data段
 - ▣ second partition of up to **8K** segments shared among all processes , the information on which is kept in **global descriptor table (GDT)**
 - 进程间共享的OS内核段、共享库 ▶

Segmentation with Paging in IA-32 【略】

- 每个段有三个描述参数：
 - ▣ 32位的段基地址（Base Address），本段在线性地址空间中的开始地址
 - ▣ 20位段界限（Limit）/段长, 段内最大偏移量.
 - ▣ 12位段属性(Attributes), 如段的可读/可写/可执行属性, 段的特权级等
- 每个段用一个描述符(segment descriptor)记录三个段参数，所有段的段描述符组成段描述符表(descriptor table)，即段表（segment table）。
 - ▣ 每个描述符长 **8 bytes**，描述符表最长为64KB

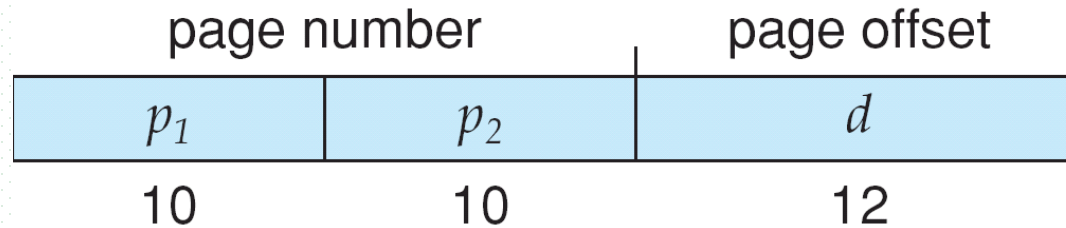
32-bit segment base address	20-bit segment limit	12-bit segment attributes
-----------------------------	----------------------	---------------------------

- 根据逻辑地址中的段的基地址和段内偏移量可生成线性空间内的线性地址, 利用段的界限判定将要生成的线性地址是否为一个段内合法地址

Segmentation with Paging in IA-32 【略】

segment table

page table



- 32-bits linear address(线性地址)
 - ▣ segment base address (段基址) + offset (段内偏移) → 线性地址

Paging in IA-32

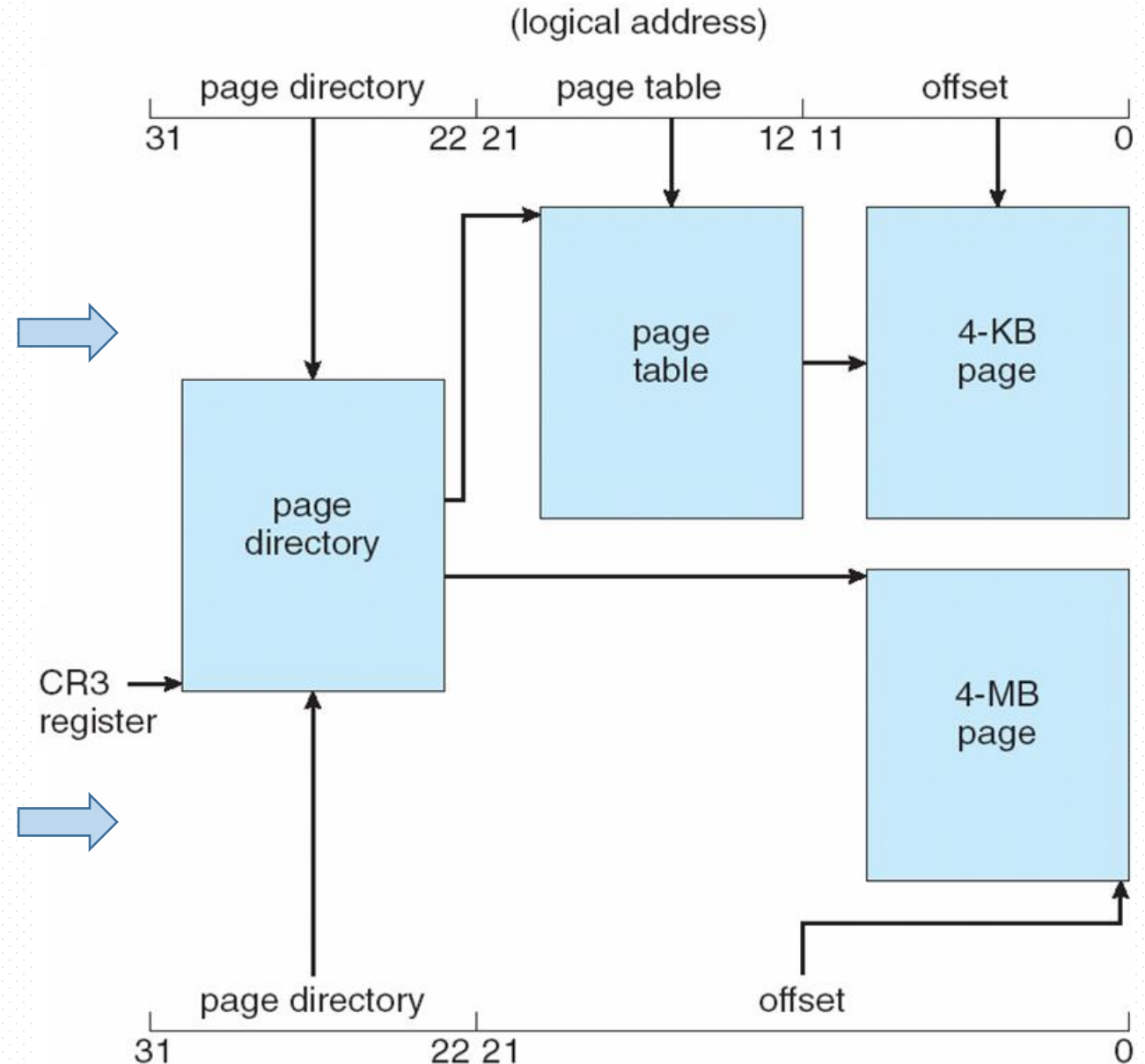
■ Intel IA-32 Paging Architecture

■ When page size is **4KB**, two-level paging

- 2^{20} pages
- 页目录, (内) 页表, 页内偏移

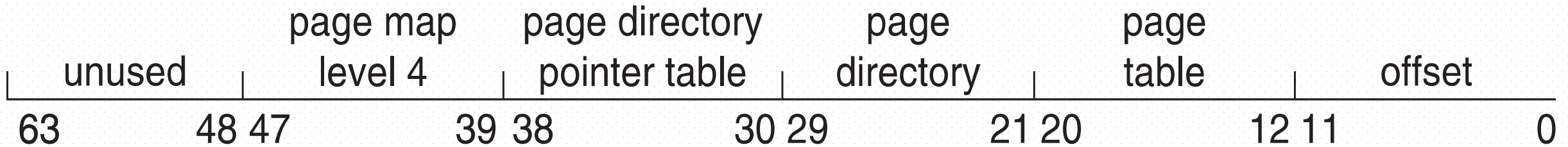
■ When page size is **4MB**, one-level paging

- 2^{10} pages
- 大页, 单级页表



Paging in x86-64

- in Intel x86 64-bits, only **48=9+9+9+9+12 bits** in address are used as instruction or data addresses, and the size of process address space in 64-bit Linux 2^{48} bytes=256T, divided into 128T kernel space and 128T user space
 - ▣ 48 bit地址线, 48位虚拟地址, **$2^{48}=256\text{TB}$ in size**

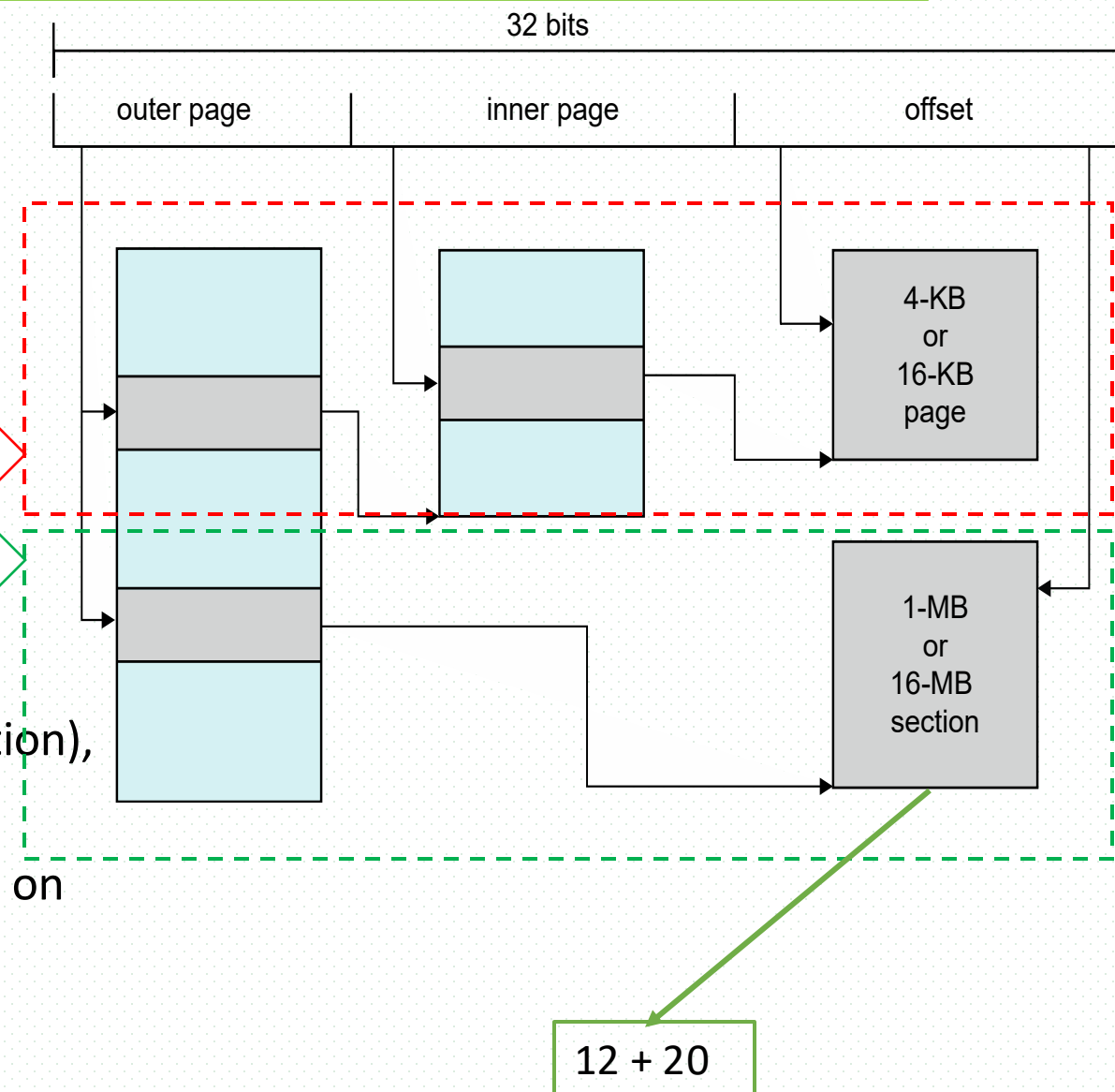


8.8 32-bit and 64-bit ARM Paging Architectures

- 32-bit ARM CPU
 - ▣ dominant mobile platform chip (Apple iOS and Google Android devices for example)
 - ▣ widely used in mobile devices
 - ▣ energy efficient, 32-bit CPU
- 64-bit ARM CPU, be used for desktop and server computer systems
 - ▣ 华为鲲鹏处理器

32-bit ARM Paging Architectures

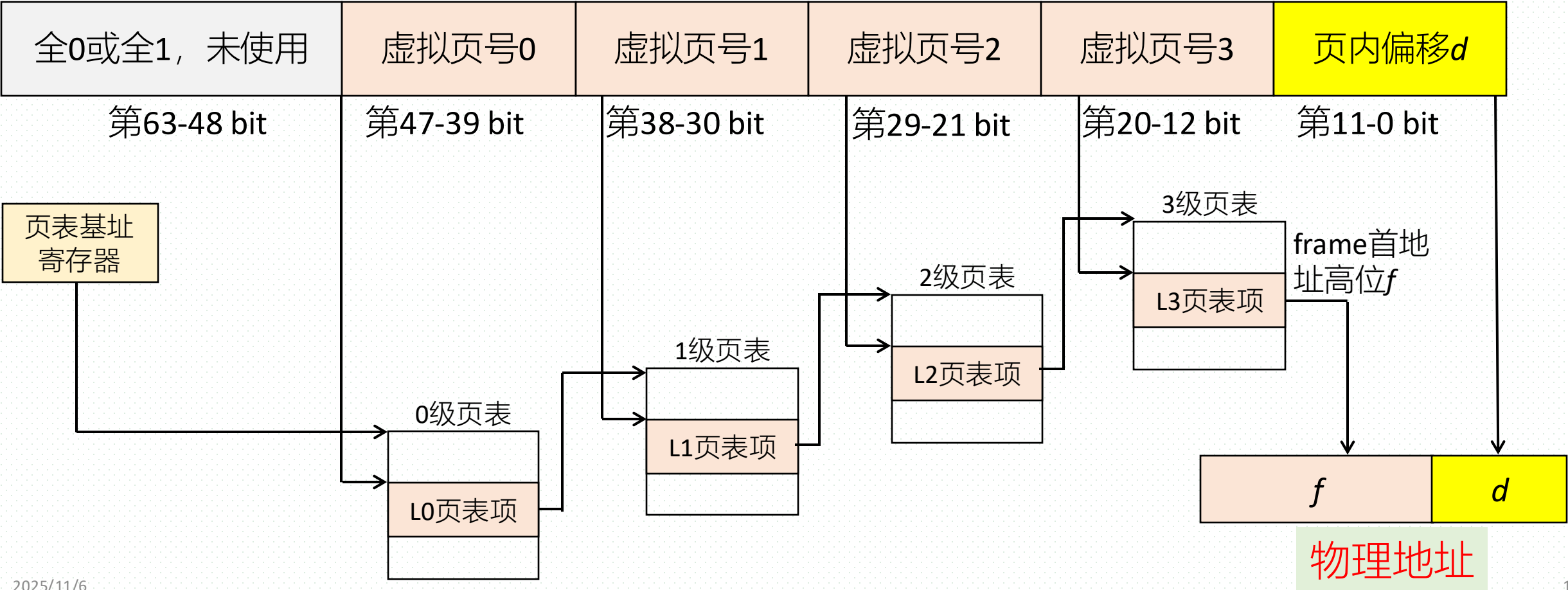
- **Only paging is used**
- Two types of page size
 - ❑ 4 KB and 16 KB pages, termed small pages
 - ❑ 1 MB and 16 MB pages (termed **sections**)
- two-level paging for smaller pages
- one-level for sections
- Two levels of TLBs to speed up address binding
 - ❑ outer level has two micro TLBs (one data, one instruction), and inner is single main TLB
 - ❑ first inner is checked, on miss outers are checked, and on miss page table walk performed by CPU



64-bit ARM Paging Architecture

- 64-bit虚拟地址，低48=9+9+9+9+12 bits参与paging-based address translation
 - ▣ 没有采用segmentation
- Page size is 4KB, four-level paging

逻辑/虚拟地址, $2^{48}=256\text{TB}$ in size



Linux中避免使用段机制

- 针对Intel x86平台，在与体系结构相关的arch子目录下，通过文件segment.h定义了Linux使用的内核段、用户段段，包括
 - ▣ KERNEL_CS, _KERNEL_DS, _USER_CS, _USER_DS
 - ▣ 每段各自从0开始编址，一个地址属于哪一段由CS寄存器指出
- Segmentation with paging需要进行两次地址变换：段地址变换、页地址变换
- 为提高地址映射效率和程序可移植性，Linux尽量避免使用segmentation，使得Intel x86系列的段式映射过程segmentation不起作用
 - ▣ 对KERNEL_CS, _KERNEL_DS, _USER_CS, _USER_DS等4个Linux所使用的段，在全局描述符表GDT初始化时，将这4个段的段基地址全部设置成0x 00000000，段的界限全部为0xFFFFFFFF
 - ▣ 进程的每个段都是从地址0开始，占据全部4G的虚拟地址空间，程序中二维逻辑地址等于虚拟内存空间中的线性地址

Appendix 8-1 Linux/Windows可执行文件格式

- Linux可执行文件ELF (Executable and Linkable Format) 格式
- Windows可执行文件PE (Portable Executable, 可移植可执行) 格式

```
// mymath.c
```

```
int my_add(int a, int b)
{
    return a + b;
}
```

```
// main.c
```

```
#include <stdio.h>
extern int my_add(int a, int b);
```

```
int main()
{
    int i = 1;
    int j = 2;
    int k = my_add(i, j);
    printf("k = %d \n", k);
}
```

编译

ELF文件 on disk

源文件
mymath.c

动态库文件
libmymath.so

源文件
main.c

可重定位文件
main.o

可执行文件
main

加载

链接

进程
in memory

Linux系统可执行文件格式、加载与执行过程

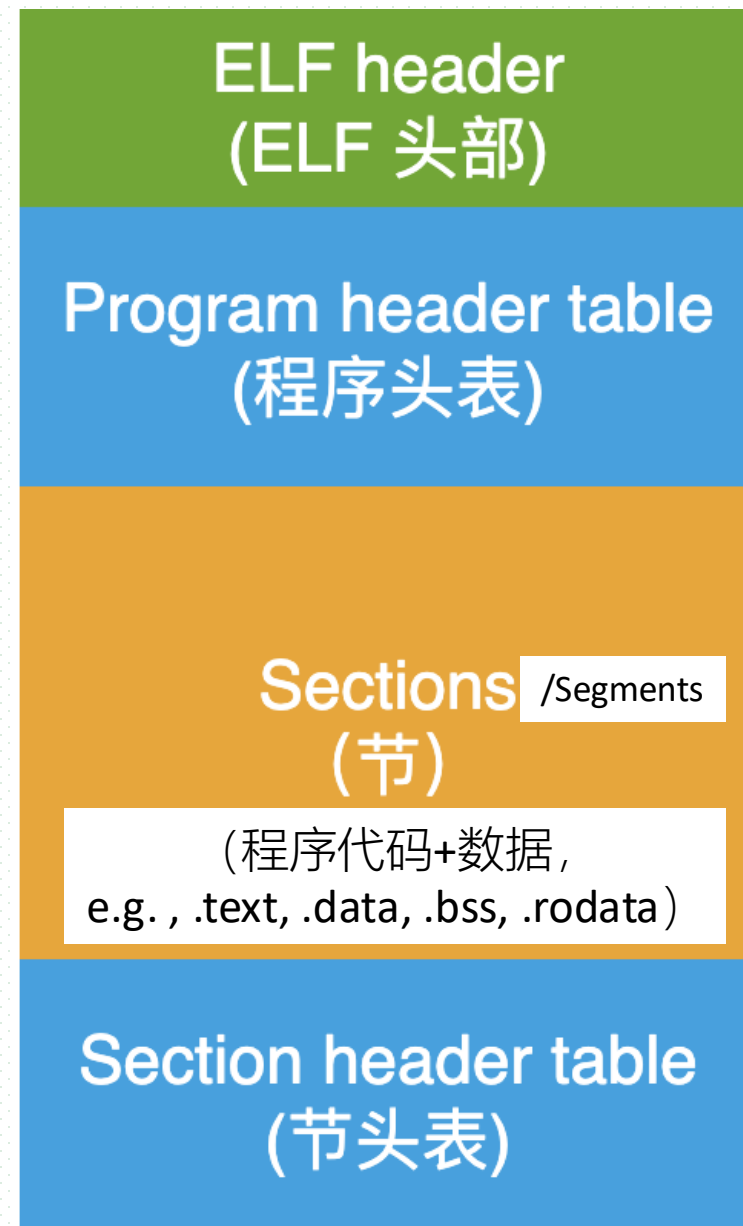
- Linux系统采用文件格式 ELF (Executable and Linkable Format)，表示3种类型文件
 - ▣ 可执行文件(executable file, .out)
 - 可直接加载到内存中执行的二进制程序，包括代码和数据
 - 代码和数据均有固定的物理地址，或相对于基址的偏移量（相对地址）
 - 由OS加载器loader从硬盘读入内存，创建进程映像，进而运行进程
 - 通过链接.o可重定位文件、.so共享目标文件得到
 - ▣ 可重定位文件(relocatable file, .o)
 - 二进制代码和数据，采用可重定位/相对地址
 - 例如，源程序（如主程序main{}）编译后的文件
 - 可以与其它可重定位文件进行链接，生成可执行文件或共享库文件
 - 一般为.o文件

Linux系统可执行文件格式、加载与执行过程

- ❑ 共享目标文件(shared object file)(.so)
 - 一种特殊的可重定位文件，也称为动态库文件，对应于共享库函数等
 - 可在链接时被链接器ld使用，或运行时被动态链接器ld-linux.so.l加载进内存

■ ELF文件组成

- ❑ ELF header (ELF头部)
- ❑ program header table (程序头表)
- ❑ section header table (节区头表)
- ❑ sections/segments (节区)



ELF文件组成

■ ELF header (ELF头部)

- 文件的总体信息，如文件类型
e_type(可执行, 可重定位, 共享目标, ...)、机器类型(Intel80386, ARM, MIPS,...)、版本等
- e_entry: 可执行程序入口地址
- Program header table(程序头表)的开始/偏移地址e_phoff、程序头个数e_phnum、表项大小e_phentsize
- Section的开始/偏移地址e_shoff, 据此可以找到sections开始位置
- Section header table(节头表) 表项大小e_shentsize, 节区section数量e_shnum

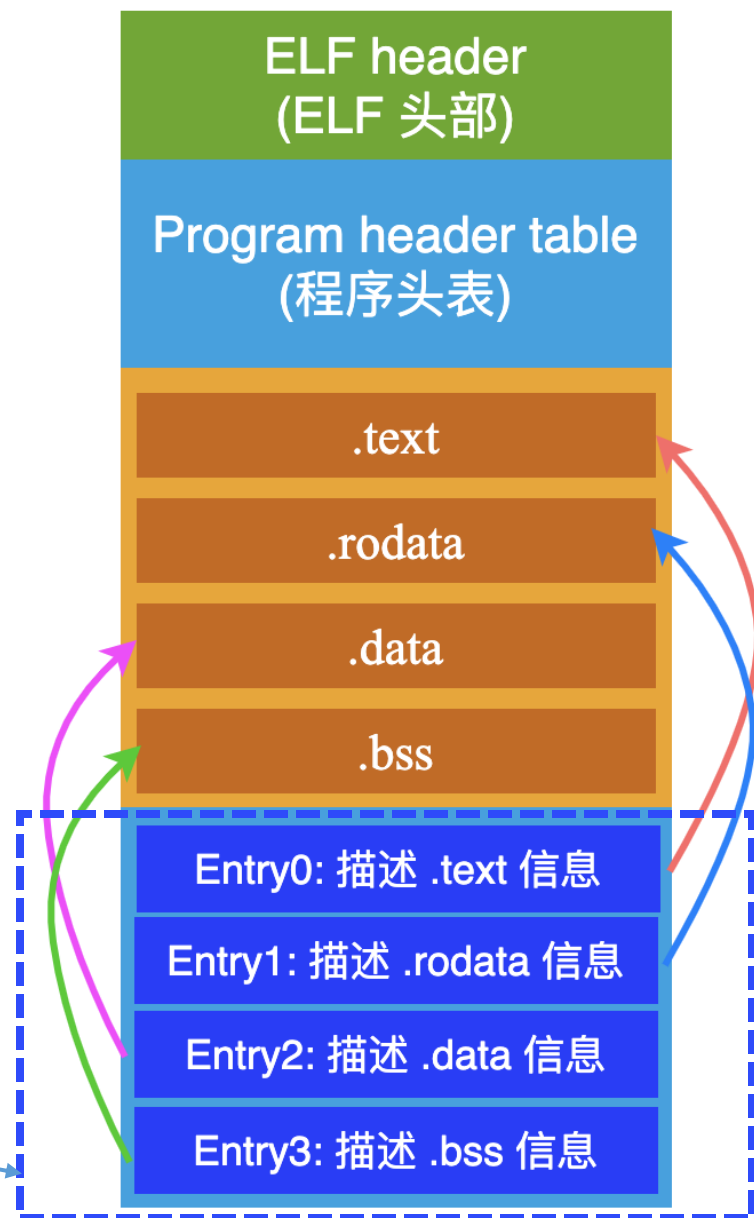
```
#define EI_NIDENT          16

typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half       e_type;
    Elf32_Half       e_machine;
    Elf32_Word       e_version;
    Elf32_Addr       e_entry;
    Elf32_Off        e_phoff;
    Elf32_Off        e_shoff;
    Elf32_Word       e_flags;
    Elf32_Half       e_ehsize;
    Elf32_Half       e_phentsize;
    Elf32_Half       e_phnum;
    Elf32_Half       e_shentsize;
    Elf32_Half       e_shnum;
    Elf32_Half       e_shstrndx;
} Elf32_Ehdr;
```

Linux内核中，描述描述
ELF header 的结构体

ELF文件组成

- Sections/segments (节区)
 - ▣ 存放指令、数据, e.g. , .text, .data, .bss, .rodata
 - ▣ 符号表、重定位表等
- Program header table/Section header table
 - ▣ 存放程序中各类section/segment的位置、大小等信息
 - ▣ 程序中的每一类节区section (代码、数据、辅助信息等), 在表中由一个表项来描述
 - ▣ E.g. 1个 ELF 文件包括.text、.rodata、.data、.bss四个sections, [Section head table](#) 用 4 个 Entry 分别描述这4个Section



Section header table

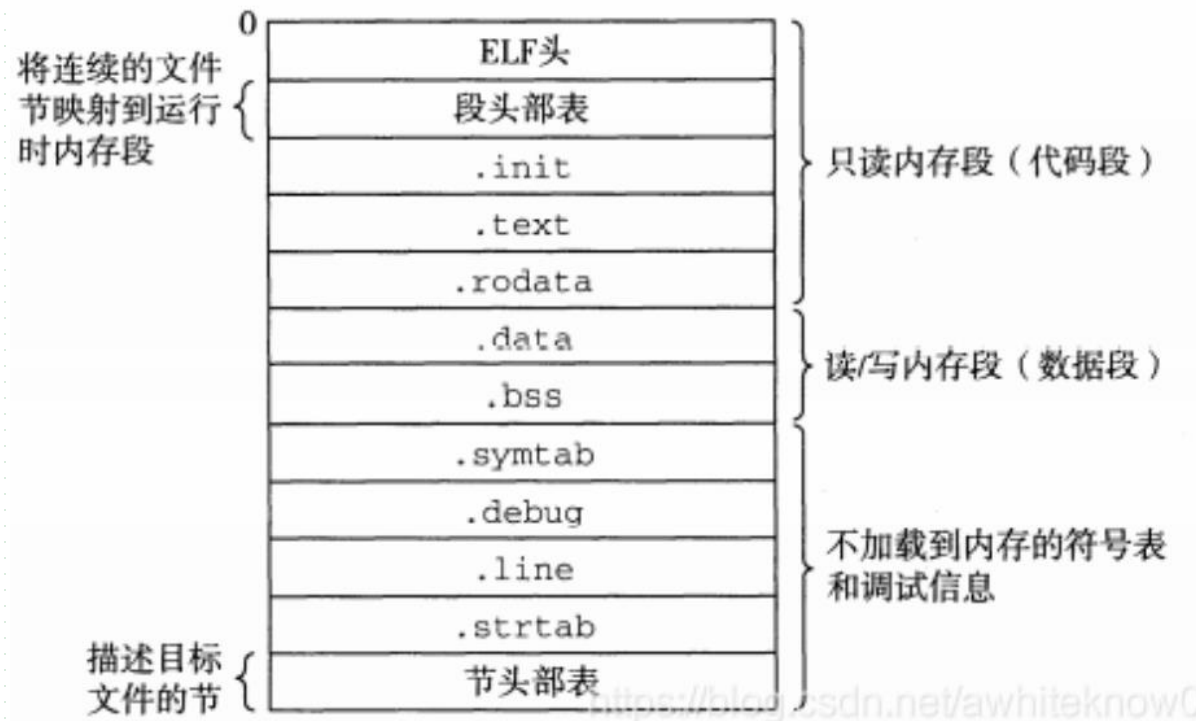
- Section header table(节头表) 包括信息
 - ▣ 节的名称Name, 节的类型Type, 节的大小Size
 - ▣ Addr: 节映射到虚拟空间的位置, 一般不用
 - ▣ Off: 节在ELF文件中的偏移
- 表中还包括程序链接信息, 分为两部分
 - ▣ 库导出的函数/变量
 - 函数/变量的名称和要加载到的内存位置, 保存在类型为DYNsym的节中
 - ▣ 库依赖的其它库的函数/变量。
 - 对于要导入的函数/变量, 在类型为RELA/REL的节中保存了引用其他库的函数/变量在符号表中的索引, 符号表中则保存该函数/变量的名称, 在类型为DYNAMIC的节中保存了程序依赖的库名称
 - 链接器在链接的时候会去依赖的库中根据函数/变量的名称去寻找该函数/变量被加载到内存的位置, 将找到的函数/变量地址写入到类型为PROGBITS (一般名称为.got的节) 节对应的位置, 之后再使用函数/变量就直接从got表中获取地址

Program header table

- Program header table保存如下信息，用于程序加载
 - ▣ Type: 段类型
 - ▣ Offset: 段在ELF文件中的偏移 地址
 - ▣ VirtAddr: 段在虚拟内存中的偏移地址
 - ▣ PhysAddr: 段被加载到内存中的物理地址，在mmu系统下不使用该地址
 - ▣ FileSize: 该段映射的文件大小。从offset $\rightarrow$ (offset+FileSize) 映射到内存的VirtAddr $\rightarrow$ (VirtAddr +FileSize)。
 - ▣ MemSiz: 该段用内存大小， $\text{MemSiz} \geq \text{FileSize}$
 - ▣ Flg: 该段在内存中的读/写/执行权限
 - ▣ Align: 段的对齐要求

ELF可执行文件的节区Sections

- **Init**: 存放初始化函数init, 初始化代码会调用它
- **text**: 已编译程序的机器代码
- **rodata**: 只读数据, 如常量、printf语句中的格式串、开关语句的跳转表
- **data**: 已初始化的全局和静态C变量, 局部变量在运行时保存在栈中
- **bss**: 未初始化的全局和静态C变量, 以及所有被初始化为0的全局或静态变量
 - ❑ 在目标文件中本节不占据实际空间, 仅仅是一个占位符
 - ❑ 目标文件格式区分已初始化和未初始化变量是为了空间效率。在目标文件中, 未初始化变量不需要占据实际磁盘空间。运行时, 在内存中为这些变量分配空间, 并将初值设为0
- **symtab**: 符号表, 存放程序定义和引用的函数和全局变量的信息, 不包含局部变量的条目
- **debug**: 调试符号表, 内容: 程序中定义的局部变量和类型定义, 程序中定义和引用的全局变量, 原始C源文件



- **Line**: C源程序中的行号和text节中机器指令之间的映射
- **strtab**: 字符串表, 其内容包括symtab和debug节中的符号表, 以及节头部中的节名字

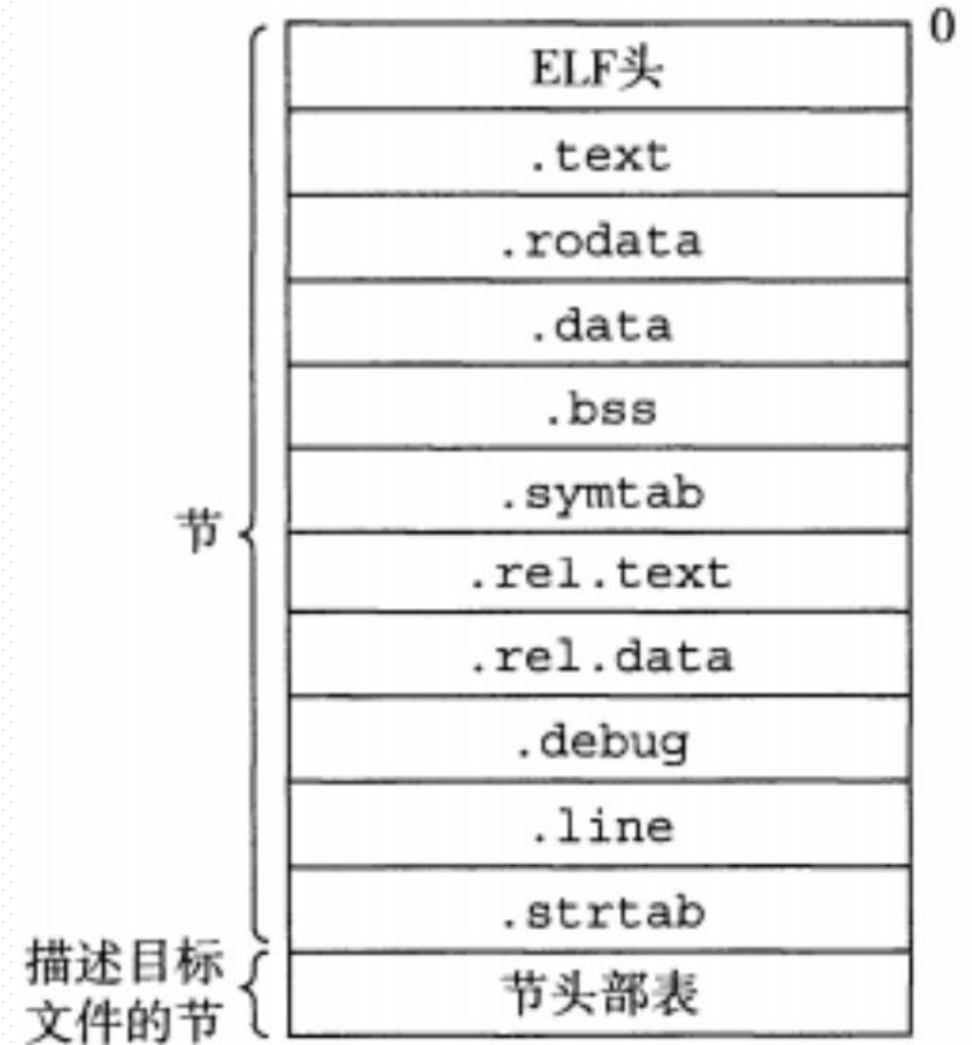
ELF可重定位文件的节区Sections

■ rel.text

- text节中位置的列表，当链接器把这个目标文件和其它文件组合时，需要修改这些位置
- 一般而言，任何调用外部函数或者引用全局变量的指令都需要修改。另一方面，调用本地函数的指令则不需要修改

■ rel.data

- 被模块引用或定义的所有全局变量的重定位信息
- 一般而言，任何已初始化的全局变量，如果其初值是一个全局变量地址或者外部定义函数的地址，都需要被修改

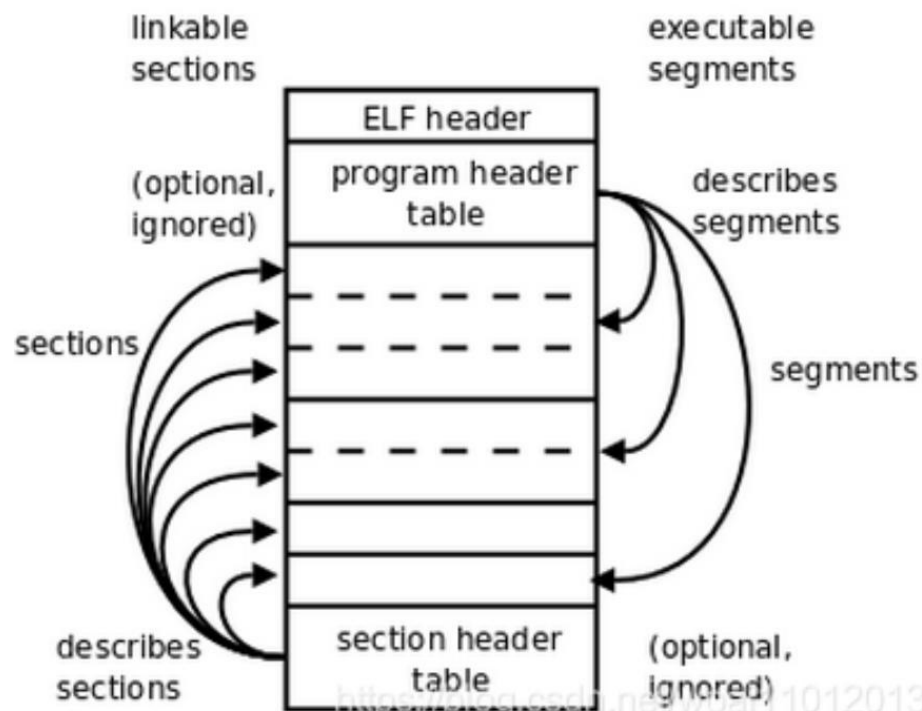


链接与加载

- 1个segment包括1个或多个section
- 链接器linker按照ELF header、Sections、Section header table，将可重定位文件、共享目标文件链接组合成可执行文件
- 加载器Loader则是按照ELF header、Segments、Program header table，将可执行文件加载进内存

Linking View	Execution View
ELF header	ELF header
Program header table <i>optional</i>	Program header table
Section 1	Segment 1
...	
Section <i>n</i>	Segment 2
...	
...	...
Section header table	Section header table <i>optional</i>

Section/linker vs segment/loader



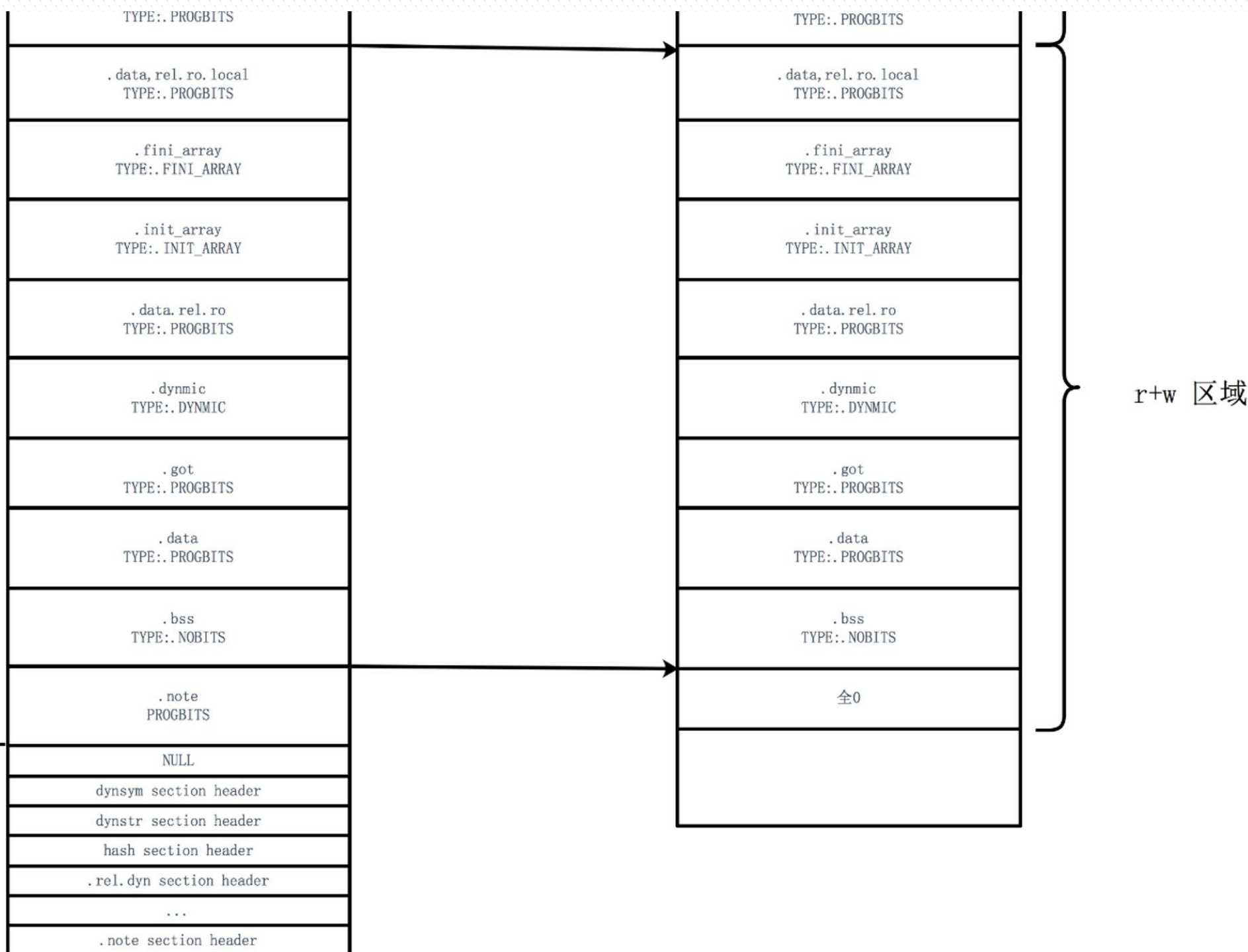
加载器
看到的是 1 个
Segment



ELF文件各部分映射至/安排至进程虚拟地址空间

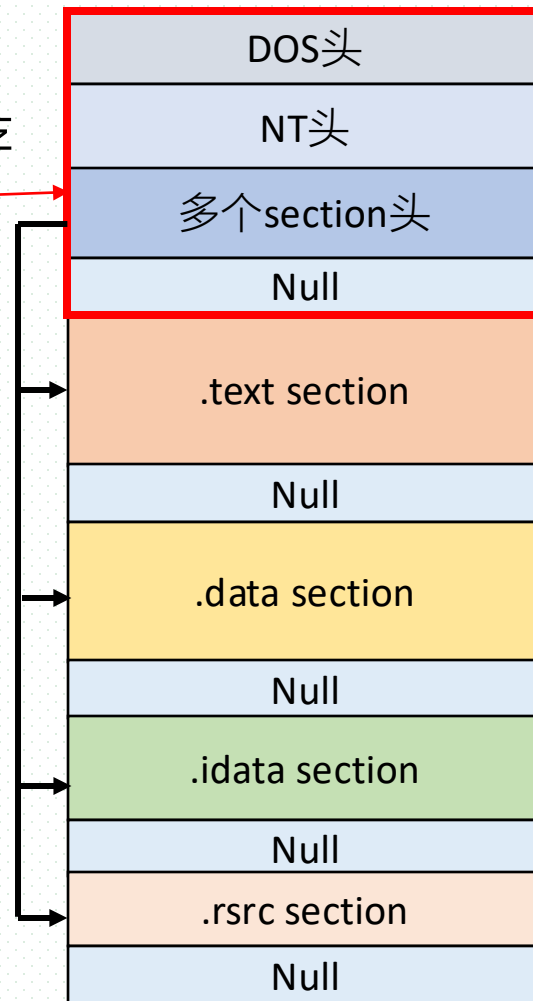


Section Header
table



Windows可执行文件PE格式

- 可执行文件：可执行程序.exe文件，动态链接库.dll文件，.sys,*.ocx等
- 源程序编译后的可执行/目标文件遵循 Windows PE (Portable Executable, 可移植执行体) 结构规范，保存在disk上
- 运行程序时，OS加载程序loader解析程序可执行文件，将其载入内存
- PE结构由PE头结构(head)、PE节区(section) 组成
- PE头结构Head
 - 保存可执行文件的加载基址（虚拟）、入口地址、文件大小、节区位置等信息
 - 程序运行前，OS据此解析可执行文件，将其载入内存
- PE节区Sections
 - text: 二进制代码/指令
 - data: 全局变量等初始化后的数据
 - idata: 使用的动态链接库(.dll)等外接函数和文件信息
 - rsrc: resource table结构和资源数据，描述程序所使用的的资源、图标、菜单



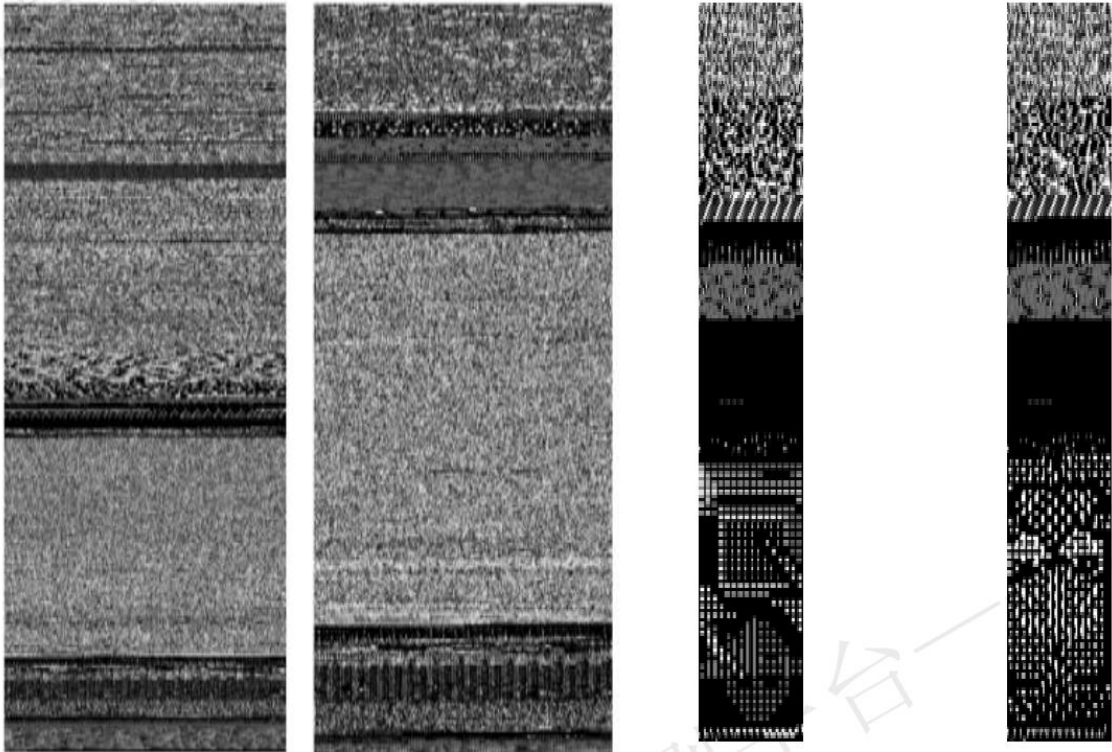
Windows可执行文件可视化：示例

■ 可执行文件机器码及其灰度图

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00
00000080	04	7A	ED	80	40	1B	83	D3	40	1B	83	D3	40	1B	83	D3
00000090	67	DD	EE	D3	4C	1B	83	D3	67	DD	F8	D3	5D	1B	83	D3
000000A0	40	1B	82	D3	81	19	83	D3	49	63	16	D3	04	1B	83	D3
000000B0	49	63	00	D3	A6	1B	83	D3	5E	49	07	D3	43	1B	83	D3
000000C0	67	DD	ED	D3	49	1B	83	D3	49	63	07	D3	14	1A	83	D3
000000D0	5E	49	17	D3	41	1B	83	D3	49	63	12	D3	41	1B	83	D3
000000E0	52	69	63	68	40	1B	83	D3	00	00	00	00	00	00	00	00
000000F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000100	50	45	00	00	4C	01	04	00	F5	45	8C	50	00	00	00	00
00000110	00	00	00	00	E0	00	03	01	0B	01	09	00	00	5A	18	00
00000120	00	32	12	00	00	00	00	00	F4	27	15	00	00	10	00	00
00000130	00	70	18	00	00	00	40	00	00	10	00	00	00	02	00	00
00000140	05	00	00	00	00	00	00	00	05	00	00	00	00	00	00	00
00000150	00	20	2B	00	00	04	00	00	8F	9E	2A	00	02	00	00	80

16 进制编辑器读取的可执行文件的机器码

可视化
→



Ramnit类
Kelihos_Ver1类
不同类别恶意软件(malware)灰度图

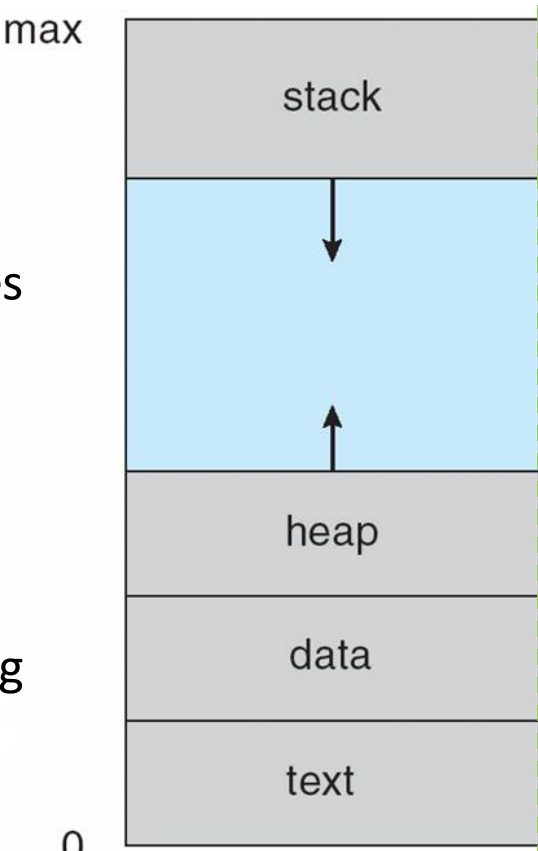
- 可执行文件中每个字节byte作为1个像素，以字节中的8位二进制数作为像素灰度值(0-255)，构造可执行文件的灰度图
- 灰度图特征反映各类可执行文件的程序类别特征，如恶意软件
- 采用图像处理、深度学习，检测恶意/病毒软件

Appendix 8-2 Address Space and Translation in Linux

- 1. Linux进程地址空间
 - ▣ 用户空间
 - ▣ 内核空间
- 2. 32-bit CPU地址空间
- 3. 64-bit CPU地址空间
- 4. 四级页表机制 in 64-bit Linux

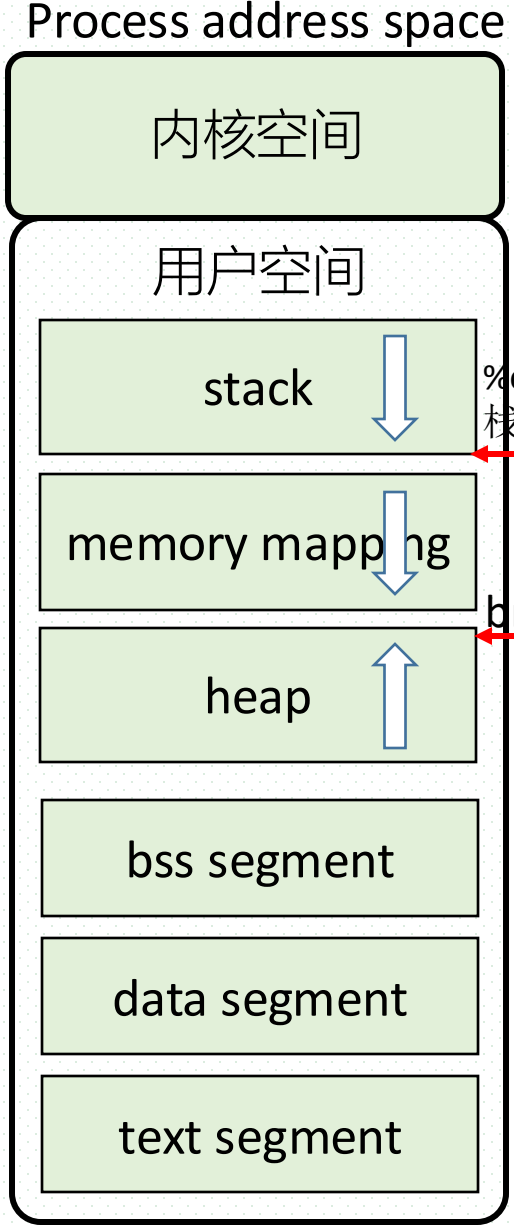
Process (Logical/Virtual) Address Space in Linux

- **Text section**
 - program code
- **Data section**
 - global variables
- **Heap**
 - memory dynamically allocated (e.g. `malloc`) during run time
- **Stack**
 - temporary data, e.g. function parameters, return addresses, local variables



Process User Address Space in Chapter 3

注：text, bss, data段的size在编译时已经确定，可以通过Linux命令查看



- **text segment**: 代码指令/常量, process binary image, ELF格式
- **data segment**: 已初始化的全局或静态变量, e.g. `static char *title = "OS";`
- **bss segment**: 未初始化的全局或静态变量, e.g. `static char *username;`
- **heap**: 堆, 用于动态申请的内存空间, 如 `brk` 或 `malloc` 申请的空间
- **memory mapping**: 1. 进程创建时, 程序用到的共享库、平台、动态链接库加载到该区域. E.g. `/lib/libc.so`; 2. `mmap` 系统调用使用的空间, 用于文件映射到内存或匿名映射 (开辟大块空间), 当 `malloc` 大于 128k 时 (此处依赖于 glibc 的配置), 也使用该区域
- **stack**: 1. 临时创建的局部变量, 不包括 `static` 声明的变量; 2. 函数调用时的参数和函数返回值

C/C++程序内存区域划分

```
int      globalVar      =1;
static int staticGlobalVar =1;
static char *username
```

```
void Test()
```

```
{
    static int staticVar =1;
```

```
int  localVar  = 1;
int  num1[10]  = {1,2,3,4};
```

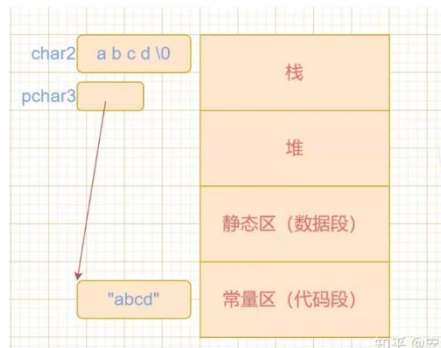
```
char char2[]  = "abcd";
char *pChar3  = "abcd";
```

```
int *ptr1  = (int*) malloc (sizeof(int)*4);
int *ptr2  = (int*) calloc (4, sizeof(int));
int *ptr3  = (int*) realloc (ptr2, sizeof(int)*4);
```

```
free (ptr1);
free (ptr2);
}
```

char2: 字符数组, 存储空间开在栈上, 将常量字符串中的内容拷贝到其中;

pchar3: 字符指针变量, 存储空间开在栈上, 存常量字符串"abcd"的地址, 指向常量字符串"abcd", "abcd"的地址在常量区 (代码段)



Process address space

内核空间

用户空间

stack

mapping space

(文件映射, 动态库, 匿名映射)

heap

bss segment

data segment

(全局, 静态)

text segment

(代码, 常量)

C/C++内存分布

我们先来看一段代码和几个问题：

```
int globalVar = 1;
static int staticGlobalVar = 1;
void Test()
{
    static int staticVar = 1;
    int localVar = 1;
    int num1[10] = { 1, 2, 3, 4 };
    char char2[] = "abcd";
    const char* pChar3 = "abcd";
    int* ptr1 = (int*)malloc(sizeof(int) * 4);
    int* ptr2 = (int*)calloc(4, sizeof(int));
    int* ptr3 = (int*)realloc(ptr2, sizeof(int) * 4);
    free(ptr1);
    free(ptr3);
}
```

选择题:

选项: A.栈 B.堆 C.数据段(静态区) D.代码段(常量区)

globalVar在哪里? ____ staticGlobalVar在哪里? ____

staticVar在哪里? ____ localVar在哪里? ____

num1^Q 在哪里? ____

char2^Q在哪里? ____ *char2在哪里? ____

pChar3在哪里? ____ *pChar3在哪里? ____

ptr1^Q在哪里? ____ *ptr1在哪里? ____

题:

alVar是全局变量, 全局变量存储在静态区, 选C。

cGlobalVar是static修饰的全局变量, 全局变量存储在静态区, 选C。

cVar是static修饰的局部变量, 但由于static修饰的变量属于静态变量, 所以也存储在静态区,

lVar是main函数^Q内部的局部变量, 存储在main函数的栈帧中, 选A。

1是数组名, 它属于局部变量, 储存在main函数的栈帧中, 选A。

2与num1类似, 属于main函数内部的局部变量, 储存在main函数的栈帧中, 选A。

r2, 指向的是数组首元素, 数组在栈上, 选A。

ar3, 属于main函数内部的局部变量, 储存在main函数的栈帧中, 选A。

ar3是一个指向"abcd"的指针, "abcd"是存储在常量区的, *pChar3是 "abcd" 的首元素, 选

ptr1是main函数内部的局部变量, 存储在main函数的栈帧中, 选A。

ptr1指向的是用malloc^Q开辟的空间, malloc从堆中申请空间, *ptr也就是访问的堆上的数据, 选B。

填空题:

sizeof(num1) = ____;

sizeof(char2) = ____; strlen(char2) = ____;

sizeof(pChar3) = ____; strlen(pChar3) = ____;

sizeof(ptr1) = ____;

sizeof 和 strlen^Q 区别?

填空题:

num1是数组名, 数组名单独放在sizeof内部表示整个数组空间的大小, 答案为40字节。

char2类似与num1, 答案为5字节。

strlen(char2)是指的是char指向的数组长度, strlen遇到' \0' 停止, 答案为4。

sizeof(pChar3), 其中pChar3是指针, 答案为4/8字节;

strlen(pChar3), strlen遇到' \0' 停止, 答案为4。

sizeof(ptr1), 其中ptr1是指针, 答案为4/8字节。

对于sizeof和strlen来说, sizeof是求数据所占空间的大小的**关键字**。strlen则是求字符串长度的**函数**。

从操作系统内存管理来说，malloc申请一块内存的背后原理是什么？

linux操作系统的内存管理是通过分页机制来实现的，一个页项4KB，那在C语言程序中，malloc申请一块内存区域是怎样通过分页的机制来管理的呢，操作系统是怎样来进行分配的？就好比申请5KB空间，操作系统去哪里找这5KB的空闲空间，又怎样通过分页的机制进行管理

在操作系统内存管理中，`malloc` 函数的申请内存背后的实现原理通常分为两个阶段：

1. 用户态：`malloc` 函数首先会在用户态的申请内存池中申请合适大小的内存块，并在内部维护一个数据结构来管理已分配的内存块和未分配的内存块。其中，已分配内存块可作为程序的堆内存使用，而未分配的内存块则可以存放新的 `malloc` 请求。
2. 内核态：当用户态没有足够的未分配内存块时，`malloc` 函数调用内核提供的系统调用 `brk` 或 `mmap`，从内核态的虚拟地址空间中分配新的内存物理页，之后再将这些物理页以虚拟内存页的形式映射到该进程的地址空间中。但是要注意，由于分页中一页通常比需要的内存大，因此一般情况下，这些新申请的页可能不止分配了所需的所有字节，还可能分配了一些多余的空间。

对于你提到的问题，当我们使用 `malloc` 申请 5KB 的空间时，通常会在用户态进行内存块的分配，首先会在用户态的申请内存池中查找是否有大小为 5KB 的空闲内存块。如果没有，那么 `malloc` 就会向操作系统发出一个 `mmap` 系统调用，请求内核在虚拟内存地址空间中分配一个新的物理页，并将其映射到进程的地址空间中，操作系统会维护进程的页表维护虚拟地址和物理地址的映射。当 `malloc` 分配出这个空间后，它会将这个空闲内存块维护在内部的未分配内存块链表中，供后续的 `mmap` 或 `brk` 再次使用。

在 Linux 操作系统中，内存管理采用的是虚拟内存管理和分页机制，在每个进程的地址空间中，内存被划分成相等的固定大小的页面，通常为 4KB，每个页面都被映射到物理内存或交换空间中。`malloc` 的内部实现在用户态维护一些数据结构，如堆和链表，来跟踪分配堆内存和空闲内存的情况，并使用系统调用 `mmap` 或 `brk` 在内核态分配和返回内存页。

`malloc` 的申请内存背后是从用户态开始，逐渐深入到内核态，依靠内核提供的系统调用来完成内存页的分配，在地址空间中建立虚拟地址和物理地址的映射，实现了内存的动态分配和管理。

实验：Linux中，一个进程最多可以创建多少个线程？

- https://mp.weixin.qq.com/s/eC599TS_0kfwDK4LGgPa1g
- 使用 **ulimit -a** 命令，查看进程创建线程时默认分配的线程栈空间大小
 - ▣ e.g. 默认分配给线程的栈空间8M，用户空间的物理内存size/8M—>可创建线程数
- 使用ulimit命令，调整栈空间大小，如改为512K，增加可创建的线程总数
 - ▣ **\$ ulimit -s 512**

```
[root@xiaolin ~]# ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 7276
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 65535
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 7276
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
[root@xiaolin ~]#
```


一个进程最多可以创建多少个线程？

- 除物理内存外，以下3个OS内核参数也影响创建线程的上限
 - /proc/sys/kernel/threads-max
 - 表示系统支持的最大线程数，默认值是 14553
 - /proc/sys/kernel/pid_max
 - 表示系统全局的 PID 号数值的限制，每一个进程或线程都有 ID，ID 的值超过这个数，进程或线程就会创建失败，默认值是 32768
 - /proc/sys/vm/max_map_count
 - 表示限制一个进程可以拥有的VMA(虚拟内存区域)的数量，默认值是 65530

```
#include <stdio.h>
#include <string.h>

void *thread ( void *vargp){
    //让线程循环，不退出
    while(1) {
        sleep(1);
    }
}

int main()
{
    int err = 0, count = 0;
    pthread_t tid;

    //持续创建线程，直到创建失败
    while (err == 0)
    {
        //创建线程
        err = pthread_create (&tid, NULL, thread, NULL);
        count++;
        printf("count = %d\n", count);
    }

    printf("create thread error : %s \n",strerror(errno));
    printf("Maximum number of thread within a Process" is : %d\n", count);

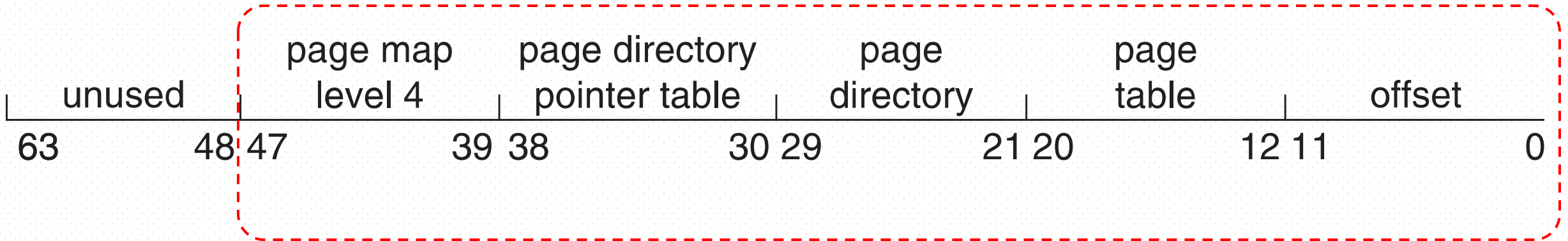
    //输入回车键，程序才退出，可以方便观察创建完线程后，进程的top情况
    getchar();

    return 0;
}
```

测试程序

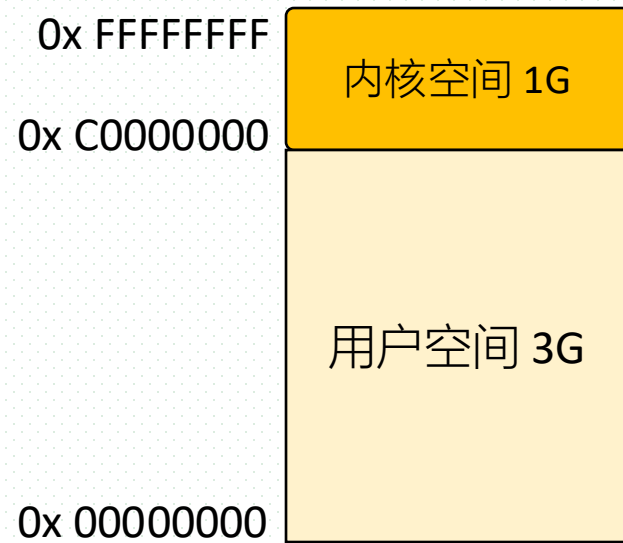
Intel x86-64

- Current generation Intel x86 architecture
- 64 bits is ginormous (> 16 exabytes)
- In practice only implement 48 bit addressing, $2^{48}=256\text{TB}$ in size
 - page sizes of 4 KB, 2 MB, 1 GB
 - four-levels of paging hierarchy, for 4KB pages
 - 48 bit地址线, 48位虚拟地址, 四级页表
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits

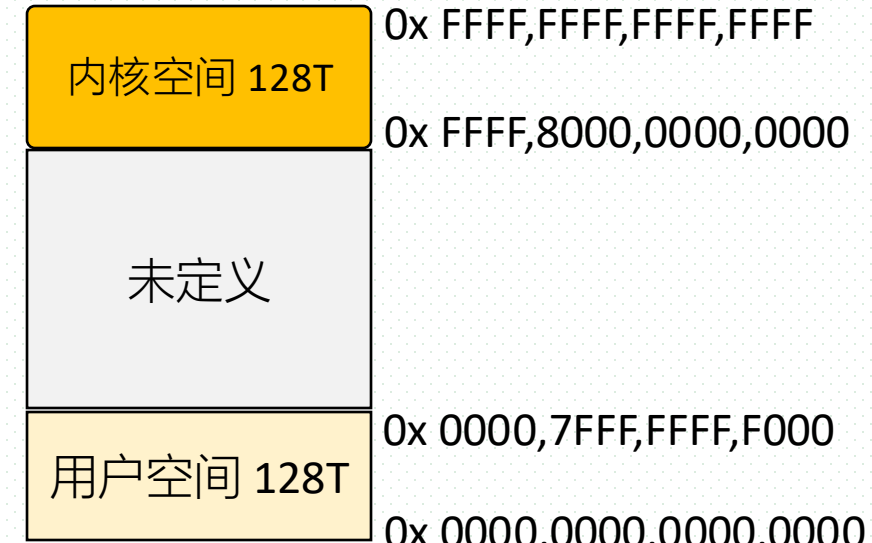


Process Address Space in Linux/Intel x86

- The size of process address space in 32-bit Linux is 2^{32} bytes=4GB, divided into 1G kernel space and 3G user space
 - ▣ 将操作系统内核映射到进程的内存地址空间，允许用户进程访问操作系统内核提供的系统调用等功能模块



32-bit地址空间，大小 2^{32} bytes=4G



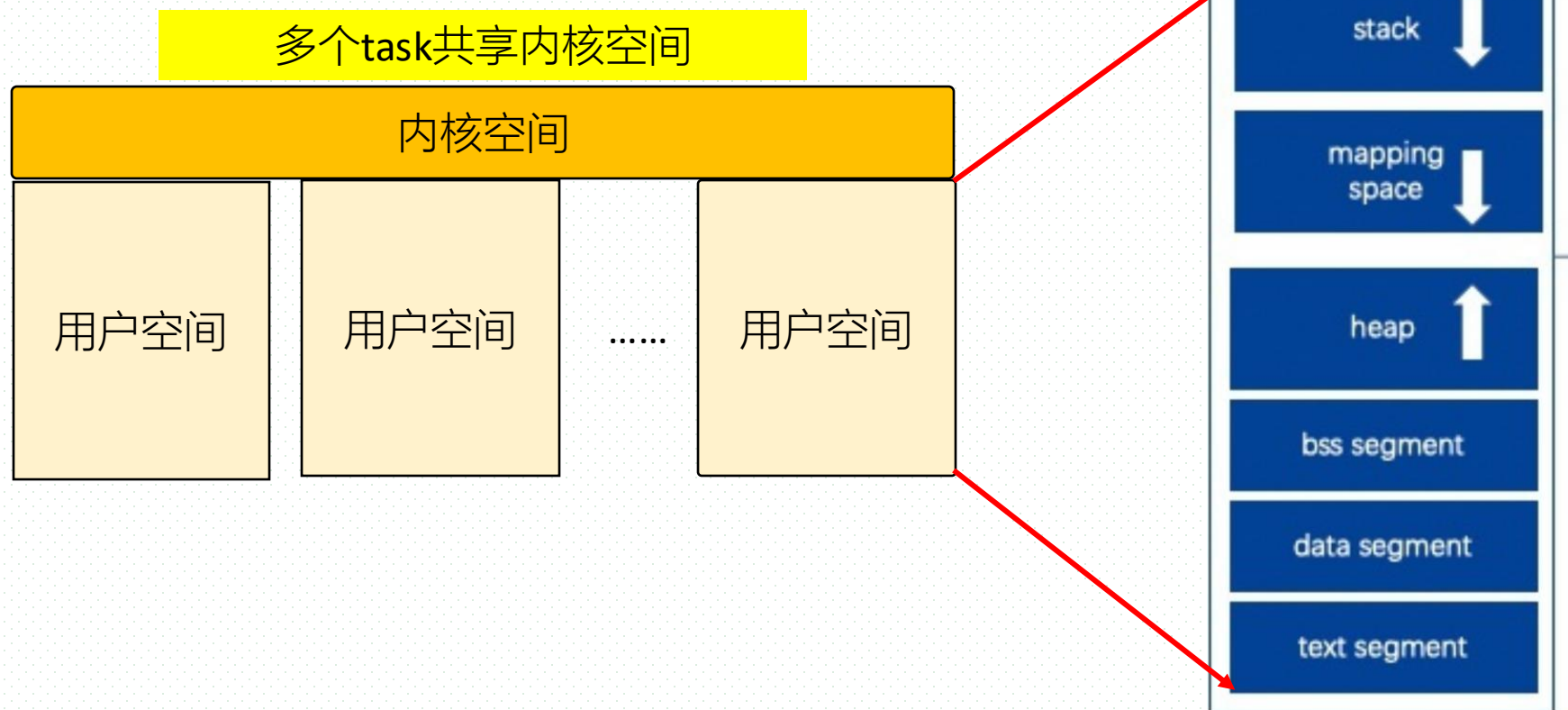
64-bit地址空间，地址线宽度48bits（48位虚拟地址），大小 2^{48} bytes=256T

Process (Logical) Address Space in Linux/Intel x86

- Kernel space in process

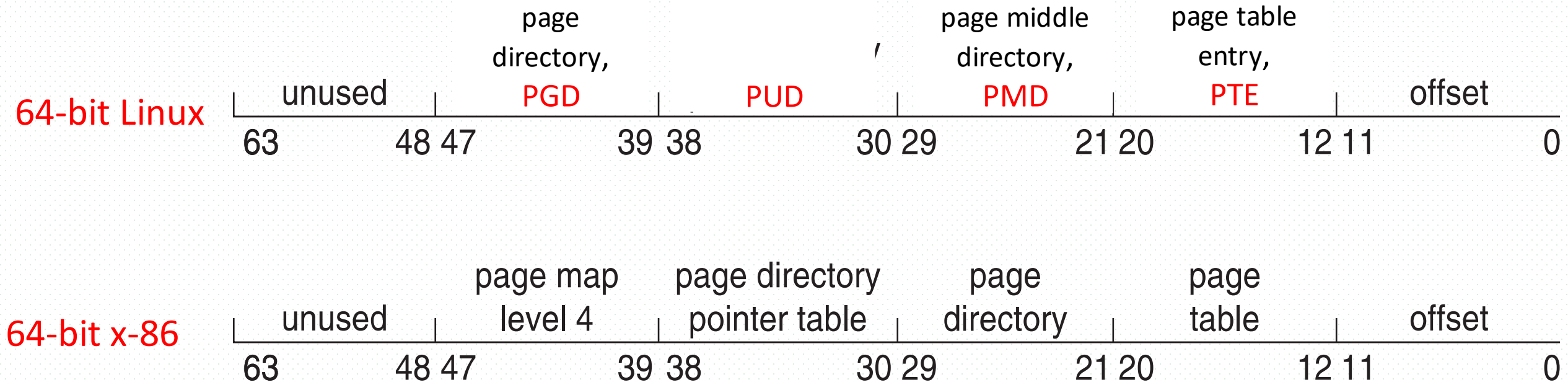
- 将操作系统内核映射到进程的内存地址空间，允许用户进程访问操作系统内核提供的系统调用等功能

- 所有进程共享内核空间



Process Address Space in 64-bit Linux/x86

- In 64-bit Linux/Intel x86 64-bits, only 48 bits in address are used as instruction or data addresses, and the size of process address space in 64-bit Linux 2^{48} bytes=256T, divided into 128T kernel space and 128T user space
 - ▣ a four-level paging scheme is used to organize the logical address space, and page size is 4K, 2M, 1G
 - 48 bit地址线, 48位虚拟地址, 四级页表



- 64-bit采用48位虚拟地址空间，40位物理地址空间，可用使用命令cat/proc/cpuinfo来查看

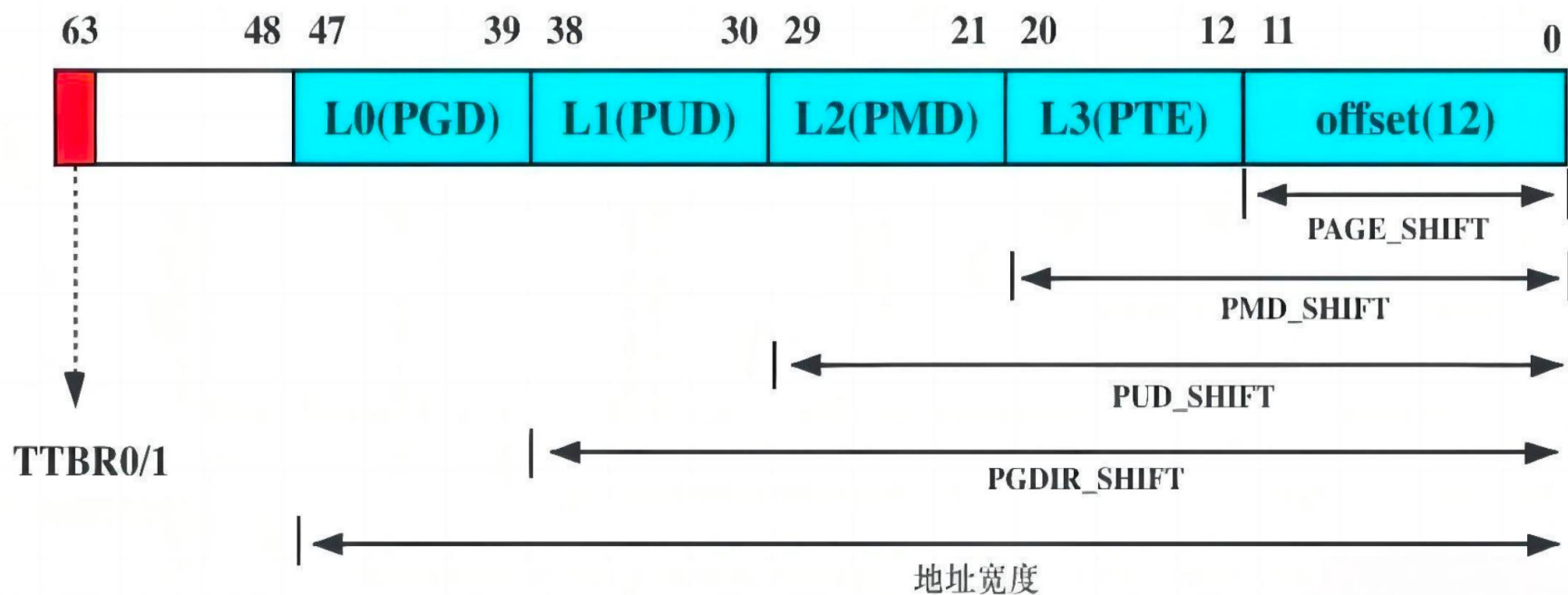
```
[root@localhost ~]# cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 44
model name     : Intel(R) Xeon(R) CPU           E5606  @ 2.13GHz
stepping       : 2
microcode      : 0x13
cpu MHz        : 2133.458
cache size     : 8192 KB
physical id    : 0
siblings       : 4
core id        : 0
cpu cores      : 1
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 11
wp             : yes
flags          : fpu de tsc msr pae cx8 apic sep cmov pat clflush mmx fxsr s
bugs           :
bogomips       : 4266.91
clflush size   : 64
cache_alignment : 64
address sizes   : 40 bits physical, 48 bits virtual
power management:
```

Address Space in Linux kernel V5.9-rc2/64-bit ARM

- Linux新版本，与CPU寻址和机制配合，可支持的用户空间、内核空间分别扩展到

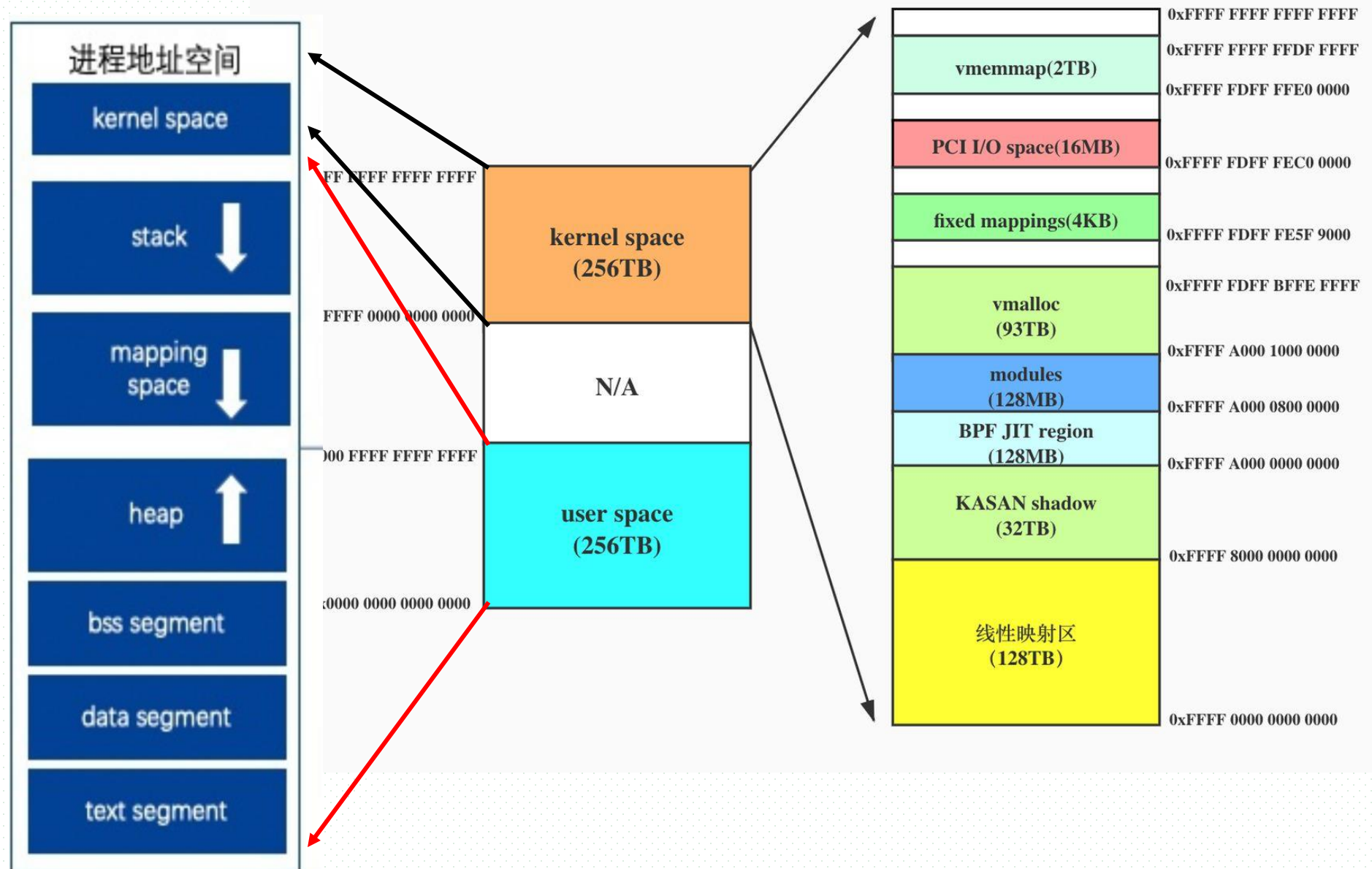
256T

- 48bit地址总线
- Page size 4KB
- 4级页表



Address Space in Linux kernel V5.9-rc2/64-bit ARM

- User space 256T
- Kernel space 256T



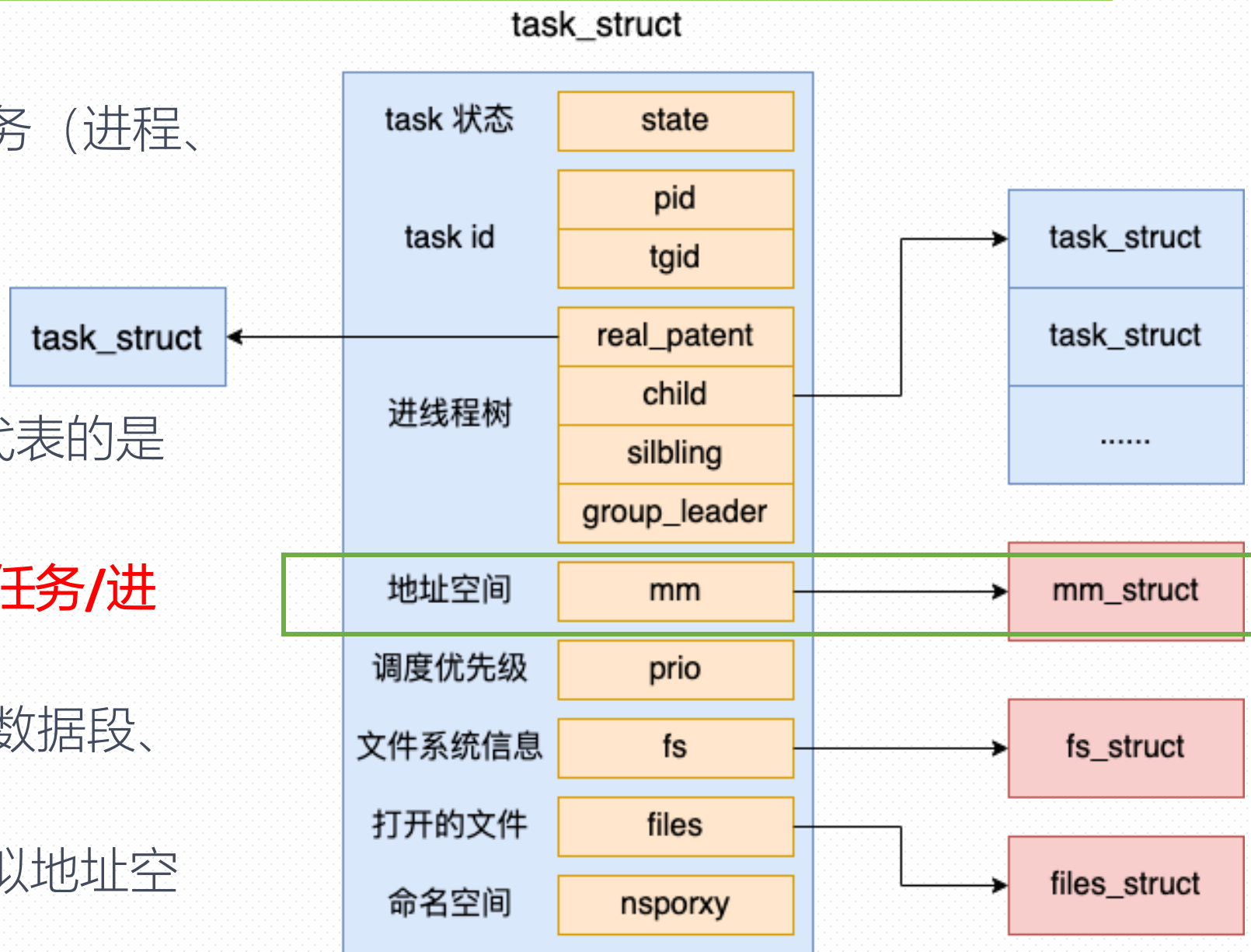
Address Space Information in Linux task_struct

■ task_struct

- 相当于PCB, 记录Linux 任务（进程、线程）信息
- 位于include/linux/sched.h

■ 内存描述符 mm_struct(mm 代表的是 memory descriptor)

- 描述运行在用户态的用户任务/进程的虚拟/逻辑地址空间
 - 包括代码段、全局变量数据段、mmap 内存映射段等
- 进程通过 mm_struct 对虚拟地址空间中各段进行寻址和访问




```

//file:include/linux/sched.h

struct task_struct {
    //2.1 进程状态
    volatile long state;

    //2.2 进程线程的pid
    pid_t pid;
    pid_t tgid;

    //2.3 进程树关系: 父进程、子进程、兄弟进程
    struct task_struct __rcu *parent;
    struct list_head children;
    struct list_head sibling;
    struct task_struct *group_leader;

    //2.4 进程调度优先级
    int prio, static_prio, normal_prio;
    unsigned int rt_priority;

    //2.5 进程地址空间
    struct mm_struct *mm, *active_mm;

    //2.6 进程文件系统信息 (当前目录等)
    struct fs_struct *fs;

    //2.7 进程打开的文件信息
    struct files_struct *files;

    //2.8 namespaces
    struct nsproxy *nsproxy;
}

```

```

//file:include/linux/mm_types.h

struct mm_struct {
    struct vm_area_struct * mmap; /* List of VMAs */
    struct rb_root mm_rb;

    unsigned long mmap_base; /* base of mmap area */
    unsigned long task_size; /* size of task vm space */
    unsigned long start_code, end_code, start_data, end_data;
    unsigned long start_brk, brk, start_stack;
    unsigned long arg_start, arg_end, env_start, env_end;
}

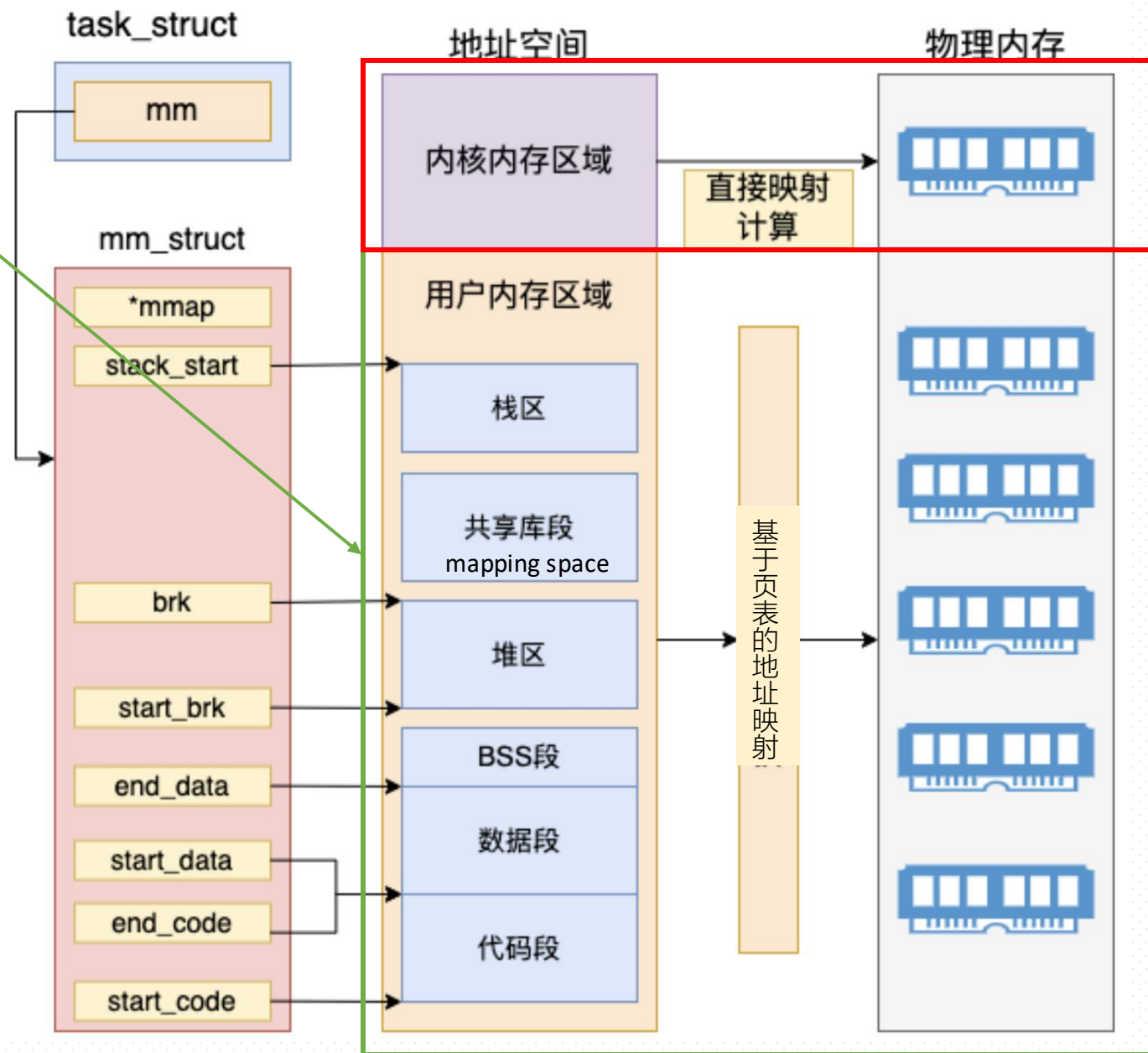
```

- start_code、end_code: 代码段的开始与结尾
- start_data 和 end_data: 决定数据段的区域
- start_brk 和 brk: 决定堆在内存中的位置
- start_stack: 用户态堆栈的起始地址
- mmap_base: 内存映射区基址

Address Space Information in Linux task_struct

对用户态下的进程/任务

- start_code, end_code
 - ▣ 代码段的开始与结尾
- start_data, end_data
 - ▣ 数据段的区域
- start_brk, brk
 - ▣ 堆在内存中的位置
- start_stack
 - ▣ 用户态堆栈的起始地址
- mmap_base
 - ▣ 内存映射区基址



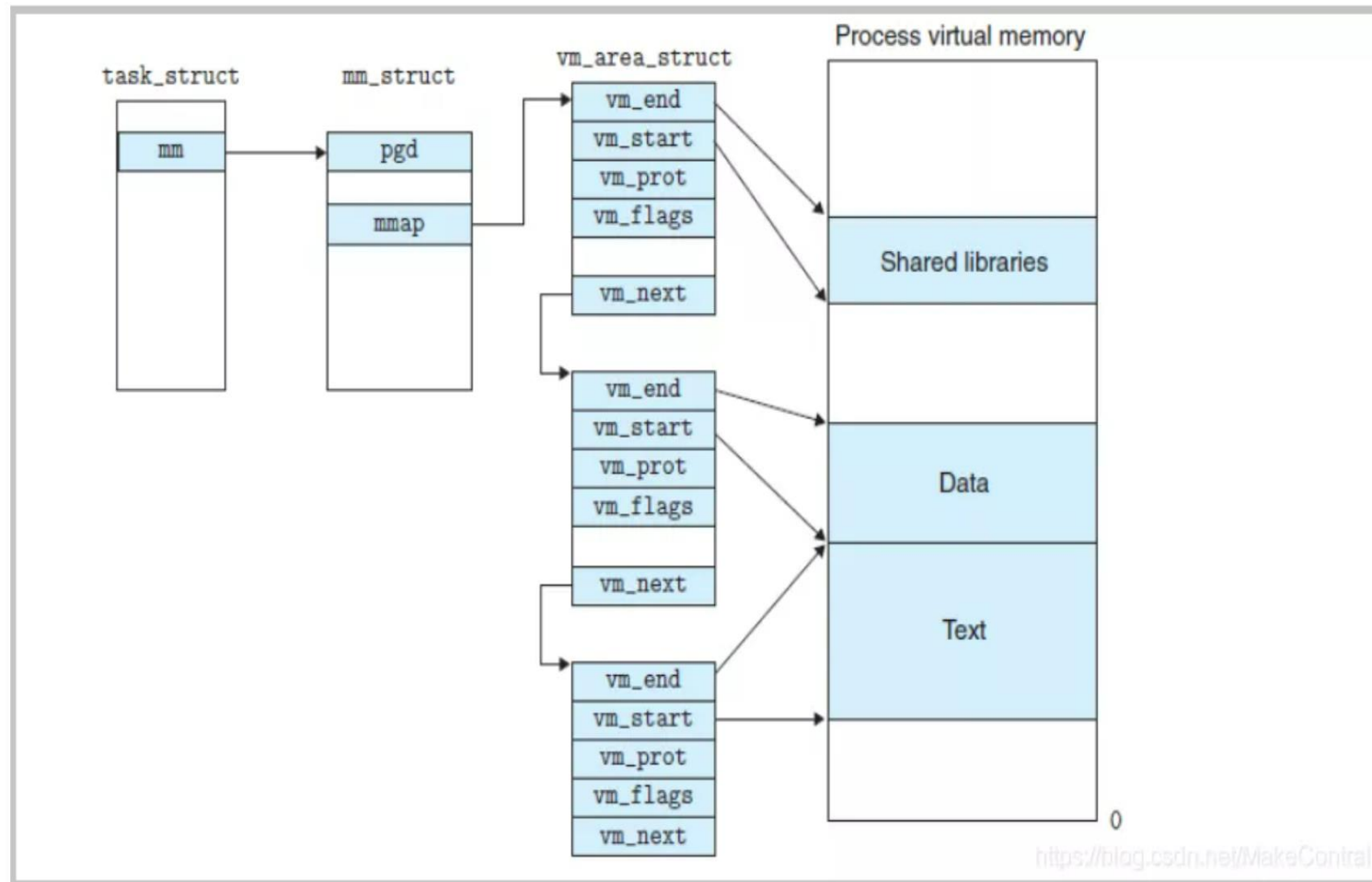
Address Space Information in Linux task_struct

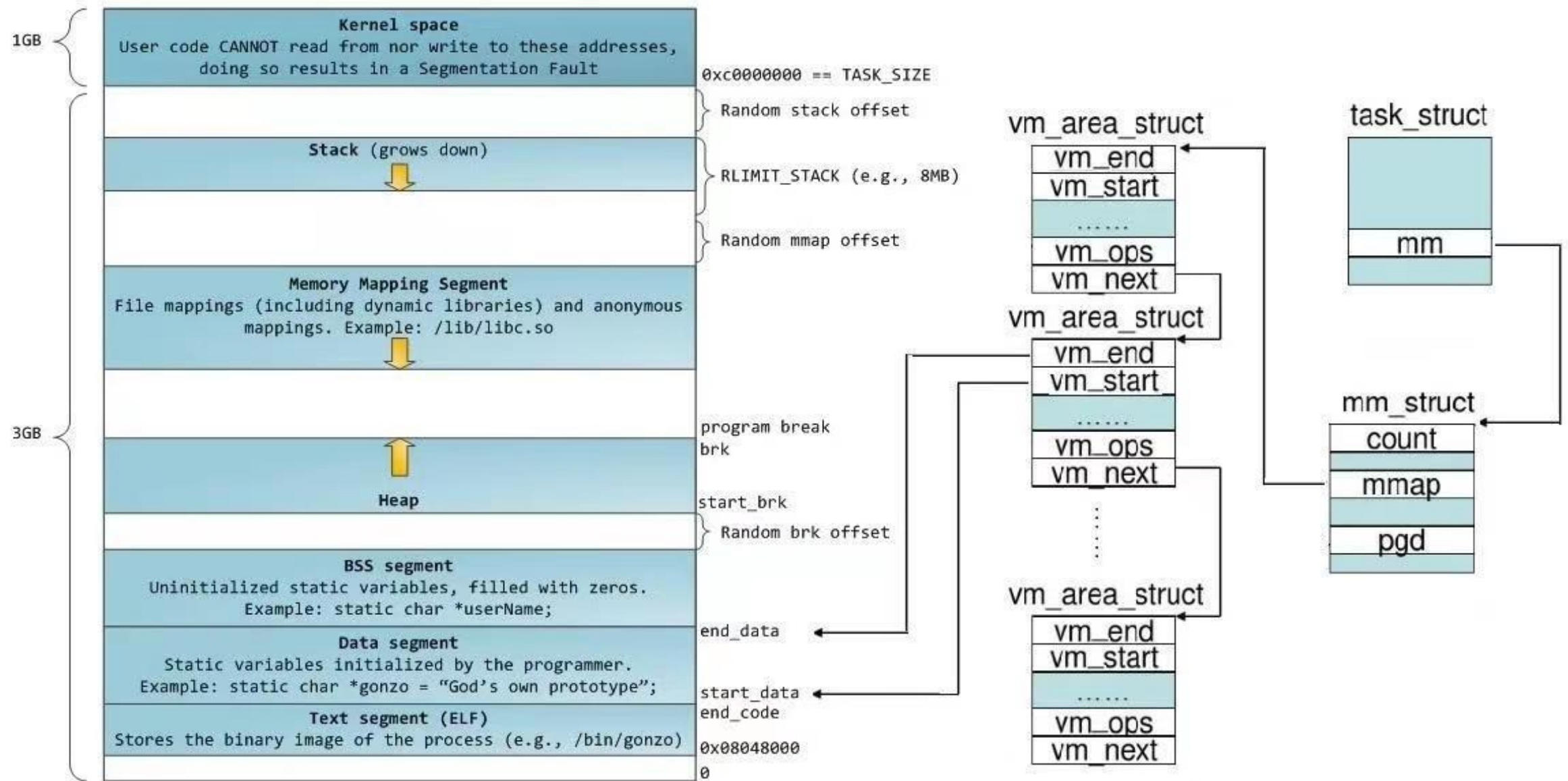
- 对运行在用户态下的用户进程，虚拟/逻辑地址空间中的地址通过paging机制，映射到物理内存中实际的物理地址
- 对用户进程所共享的运行在内核态下的OS内核线程
 - ▣ 没有用户态的虚拟地址空间
 - ▣ 需要访问的物理内存区域和物理地址，通过直接计算得出，不需要复杂、基于页表的地址转换
 - ▣ 所有用户进程共享这部分内核态下的地址空间 ▶

Address Space Information in Linux task_struct 【老版本？！】

- Linux 内核使用 **vm_area_struct** 结构表示一个 task 的虚拟地址空间中各段，vm_area_struct 包含以下字段

- vm_start: 指向段起始处
- vm_end: 指向段结束处
- vm_prot: 段内所有页的读写权限
- vm_flags: 段是私有的，还是可共享的





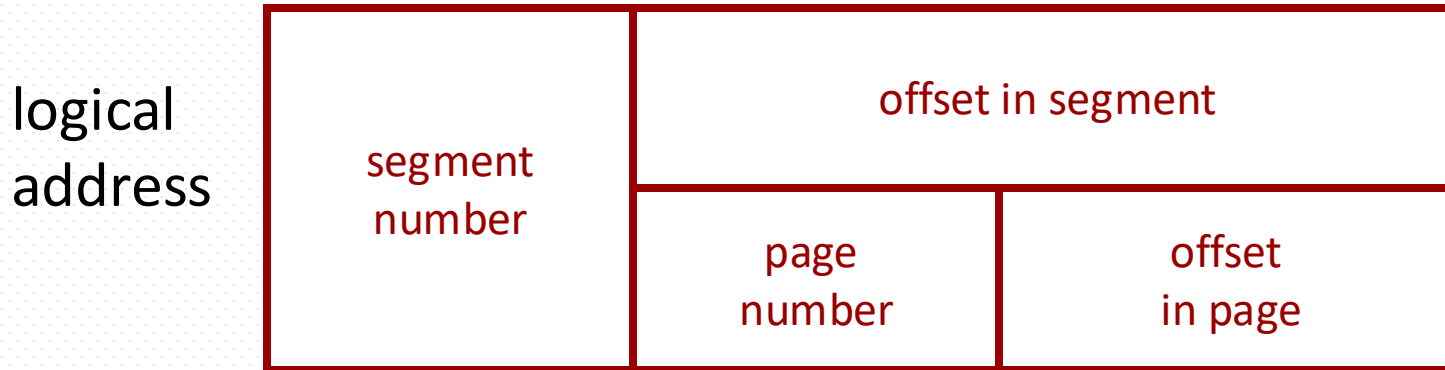
Appendix 8-3 Segmentation with Paging (段页式, out of date) 【略】

- Both paging and segmentation have advantages and disadvantages
 - ▣ paging: internal fragmentation, too many pages may results in search times for pages
 - ▣ segmentation: external fragmentation
- Two-dimensional segmentation structure is more suitable for programming
- Segmentation with paging (段页式内存管理)
 - ▣ the logical address space is defined as <segment number, offset>
 - ▣ each segment can be allocated to noncontiguous frames, in each segment, the physical address is defined by the page number and the offset in pages
 - ▣ one segmentation table and several page tables (each one for a segment) are used for address binding

华为openGauss数据库的数据库文件逻辑结构:

1. 将DB file存储空间逻辑上组织为段页式结构
2. 关系表中的元组（即数据库文件记录）存储在各个页中
3. B+树索引的叶结点为page，每个page中存放多个元组/记录

Segmentation with Paging

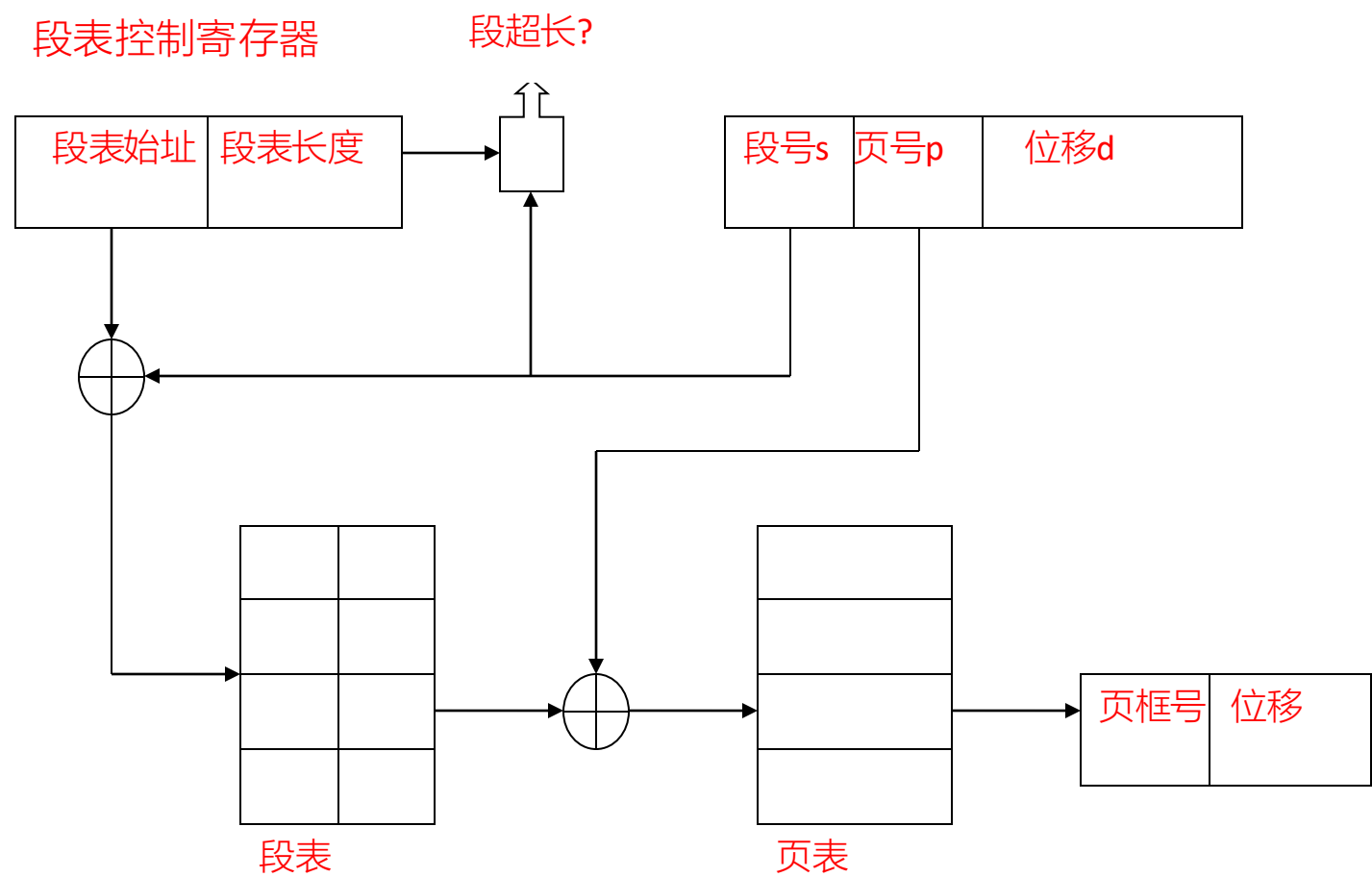


- Differing from pure segmentation, the segment-table entry does not contain the base address of the segment, but the base address of a *page table* for this segment
- For more details, refer to Chapter 9

■ Address binding in segmentation with paging

逻辑地址形式

段号(s)	段内页号 (p)	页内位移(d)
-------	----------	---------





Thanks for your
attention



北京邮电大学